PEARL: SPATIO-TEMPORAL GRAPH LEARNING FOR PREDICTIVE & EXPLORATORY ANALYTICS IN ESPORTS: A ROCKET LEAGUE CASE STUDY

by

Oğuz Arslan

B.S., Mechanical Engineering, Boğaziçi University, 2022

Submitted to the Institute for Graduate Studies in
Science and Engineering in partial fulfillment of
the requirements for the degree of
Master of Science

Graduate Program in Computer Engineering

Boğaziçi University

2026

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my thesis advisor, Assist. Prof. Suzan Üsküdarlı, for her invaluable mentorship. She has been an exceptional teacher, providing not only academic guidance but also profound wisdom regarding life. I am truly grateful for her support and the broader perspective she has instilled in me throughout this journey.

I extend my sincere thanks to Enis Simsar, Oğuz Kaan Yüksel, DJ Sterling, and Peter Xenopoulos. Their combined experience and insightful discussions were instrumental in shaping the project's direction. Their technical guidance and feedback played a crucial role in the growth and maturation of this thesis.

I also wish to acknowledge the Rocket League developer community as well as the individuals behind the professional esports scene. It is their passion and the robust data ecosystem they have cultivated that made this research possible.

Finally, I express my heartfelt thanks to my family for their unwavering support at every step of the way. They have been my greatest source of motivation. I dedicate this thesis to them, with love and gratitude.

ABSTRACT

PEARL: SPATIO-TEMPORAL GRAPH LEARNING FOR PREDICTIVE & EXPLORATORY ANALYTICS IN ESPORTS: A ROCKET LEAGUE CASE STUDY

Improving strategy and player development is a central objective in sports and esports. Given the abundance of data, analytics tools are increasingly used to find insights. However, existing tools for team-based sports largely focus on post-hoc descriptive statistics and fail to capture their dynamic nature. Using Rocket League as a representative case study, this thesis introduces peaRL, a comprehensive framework that combines a constructed dataset of nearly 10 million game states from professional matches with an ML pipeline for scoring prediction. A baseline is first established via a statistical ledger that is based on spatial parcelization to map historical goal probabilities. On top of this, multiple architectures ranging from standard baselines (XGB, MLP) to GNN models and temporal variants (GAT-LSTM) were evaluated. All evaluated architectures outperform both primitive heuristics and the statistical baseline. The GAT models showed better probability calibration over other architectures. Ablation studies on GAT models show that representing the ball as a distinct node improves model predictions, whereas explicitly encoding kinematic edge features yields diminishing returns. Additional experiments suggest that these models are generalizable for lower-skilled gameplay. A stratified threat analysis shows that the model performances increase towards higher-threat zones. To address the requirement for visualization of complex spatial data, a custom interactive 3D Analysis Portal was developed using the Unity engine. To support real-time inference, this tool connects the ML models to the user interface. The system enables users to manipulate game states and create counterfactual scenarios to receive quantitative feedback on specific strategic decisions.

ÖZET

PEARL: ESPORDA ÖNGÖRÜCÜ VE KEŞİFSEL VERİ ANALİTİĞİ İÇİN UZAMSAL-ZAMANSAL GRAF ÖĞRENİMİ: BİR ROCKET LEAGUE VAKA ÇALIŞMASI

E-sporda strateji oluşturma ve oyuncu gelişimini iyileştirmek, takımlar için kritik bir hedeftir. Mevcut tekrar verilerinin bolluğu göz önüne alındığında, içgörü elde etmek amacıyla analitik araçların kullanımı giderek yaygınlaşmaktadır. Ancak Rocket League özelinde bu araçlar, genellikle geriye dönük betimsel analizlere odaklanmakta ve oyunun karmaşık, dinamik doğasını yakalamakta yetersiz kalmaktadır. Bu sorunu gidermek amacıyla; 10 milyona yakın oyun durumunu içeren yüksek frekanslı bir veri setini, tahmine dayalı bir makine öğrenmesi hattı ile birleştiren peaRL sistemini sunuyoruz. Çalışmada ilk olarak, mekansal ayrıklaştırmaya dayalı istatistiksel bir kayıt sistemi oluşturulmuştur. Ardından, basit taban modellerden zamansal graf modellerine kadar uzanan bir tahmine dayalı mimariler hiyerarşisi uygulanmış ve karşılaştırılmıştır. Ayrıca geliştirilen bu hat, daha düşük seviyeli oyun verileri üzerinde de doğrulanmıştır. Değerlendirme sonuçları, veri dağılımının dengesiz olduğu bu alanda önerilen graf tabanlı mimarilerin, tehlikeli durumları sıralama başarısında istatistiksel taban modelleri önemli ölçüde geride bıraktığını doğrulamaktadır. Son olarak, bu modeller sürükleyici bir 3 boyutlu etkileşimli portal içerisine entegre edilerek; sistemin, kullanıcıların oyun durumlarını manipüle etmelerine ve stratejik kararlarına ilişkin anlık, nicel geri bildirim almalarına olanak tanıyan bir deneysel keşif aracı olarak nasıl işlediği ortaya konulmuştur.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS . . . iii
ABSTRACT . . . iv
ÖZET . . . v
LIST OF FIGURES . . . xi
LIST OF TABLES . . . xiii
LIST OF SYMBOLS . . . xiv
LIST OF ACRONYMS/ABBREVIATIONS . . . xvi
1. INTRODUCTION . . . 2
1.1. High-level Description of Esports . . . 2
1.2. Need for a Solution . . . 2
1.3. Motivation . . . 2
1.4. Overview of the Method . . . 3
1.5. Main Contributions . . . 3
1.6. Thesis Organization . . . 4
2. BACKGROUND . . . 6
2.1. The World of Esports . . . 6
2.1.1. The Rise of Esports . . . 6
2.1.2. The Esports Landscape . . . 7
2.1.3. The Professional Ecosystem . . . 8
2.2. Rocket League . . . 9
2.2.1. Cars and Hitboxes . . . 10
2.2.2. Ranking System . . . 10
2.2.3. Replay System and Data Collection . . . 11
2.2.4. Unity and Unreal Game Engines . . . 13
3. RELATED WORK . . . 14
3.1. Descriptive and Statistical Sports Analytics . . . 14
3.1.1. Foundational Surveys and Frameworks . . . 14
3.1.2. Publicly Available Tools for Rocket League . . . 14

3.1.3. Statistical and Computational Modeling 15
3.1.4. Methodologies for Descriptive Analysis 16
3.1.5. Data Engineering and Preparation 16
3.2. Machine Learning for Prediction 16
3.2.1. Predictive Modeling in Rocket League 17
3.2.2. Outcome and Performance Prediction in Other Sports and Esports 17
3.2.2.1. Goal and Win Probability 17
3.2.2.2. Player and Action Valuation 17
3.2.3. Meta-Game and Strategic Prediction 18
3.2.4. Advanced Methodologies for Prediction 18
3.2.4.1. Graph-Based Modeling for Sports 18
3.2.4.2. Reinforcement Learning for Automation (AI Bots) . . 19
3.2.4.3. Generative Modeling for Behavioral Stylometry 19
3.2.4.4. Foundational Theory for Spatio-Temporal GNNs . . . 19
3.3. Interactive Systems and Counterfactual Analysis 20
3.3.1. Interactive Systems for "What-If" Analysis 21
3.3.2. The Foundation of Counterfactual Exploration and XAI 21
3.3.3. Interactive Systems for Exploratory Game Analytics 22
3.3.4. User-Centered Design and Communication 22
3.3.5. Interactive Systems for "What-If" Analysis 23
3.3.6. The Foundation of Counterfactual Exploration and XAI 24
3.3.7. Interactive Systems for Exploratory Game Analytics 24
3.3.8. User-Centered Design and Communication 25
4. METHODS 27
4.1. Data Preprocessing Pipeline 27
4.1.1. Data Sourcing and Collection 27
4.1.2. Replay Parsing and Player Standardization 29
4.1.3. State Assembly and Feature Engineering 29
4.1.3.1. Demolition-Aware Null Imputation 30
4.1.3.2. Feature Engineering 31
4.1.3.3. Dataset Schema and Final Feature Vector 32

4.1.4. Gameplay Filtering and Ground Truth Labeling 33
4.1.5. Downsampling and Dataset Aggregation 34
4.1.6. Dataset Characteristics and Splitting 35
4.1.6.1. Data Integrity and Volume 36
4.1.6.2. Primary Dataset Class Imbalance 36
4.1.6.3. Train-Validation-Test Split 37
4.1.6.4. Ranked Generalization Dataset 37
4.2. State Tokenization and Historical Probabilities 37
4.2.1. State Tokenization Rules 38
4.2.1.1. Spatial Discretization (The Grid) 38
4.2.1.2. Permutation Invariance 38
4.2.2. Canonicalization and Symmetry Validation 40
4.2.2.1. The Resolution Trade-off with Sparsity 42
4.2.3. Predictive Performance on Test Set 42
4.3. Model Architectures 44
4.3.1. Experimental Framework 44
4.3.1.1. Input Representation and Normalization 44
4.3.1.2. Handling Imbalance: Weighted Loss 45
4.3.2. Primitive and Shallow Baselines 46
4.3.2.1. Trivial Heuristics Definitions 46
4.3.3. Feature-Driven Baseline: Logistic Regression 47
4.3.4. Gradient Boosted Decision Trees: XGBoost 48
4.3.4.1. Data Processing and Architecture 48
4.3.4.2. Training Configuration 48
4.3.5. Multi-Layer Perceptron (MLP) 49
4.3.5.1. Data Processing and Architecture 49
4.3.6. Spatially-Aware Graph Neural Networks (GNNs) 50
4.3.6.1. Graph Construction and Topology 50
4.3.6.2. Graph Convolutional Networks (GCN) 52
4.3.6.3. Graph Attention Networks (GAT) with Relational Edge Features 52

4.3.7. Temporal Sequence Models (LSTM) . . . 55
4.3.7.1. Sequential Data Engineering: The Boundary Problem 56
4.3.7.2. Baseline LSTM Architecture . . . 56
4.3.7.3. The Spatio-Temporal GNN-LSTM . . . 57
4.4. User Interface Portal . . . 58
4.4.1. System Architecture and Data Ingestion . . . 59
4.4.1.1. Coordinate System Transformation . . . 59
4.4.1.2. Replay State Management . . . 60
4.4.2. The Analytic Replay Viewer . . . 60
4.4.2.1. Temporal Visualization . . . 60
4.4.2.2. Temporal Threat Visualization . . . 60
4.4.3. Real-Time Inference and Counterfactuals . . . 61
4.4.3.1. Counterfactual Interaction . . . 61
4.4.3.2. The Inference Engine (Unity Sentis) . . . 62
4.4.4. Historical Data Integration . . . 62
4.4.5. Human Benchmarking Module . . . 63
5. EVALUATION . . . 64
5.1. Experimental Setup . . . 64
5.1.1. Implementation and Training Details . . . 64
5.1.1.1. Optimization Strategy . . . 64
5.1.1.2. Model Checkpointing . . . 65
5.1.2. Standardization of Evaluation Metrics . . . 65
5.2. Baseline Models . . . 66
5.2.1. Baseline Results and Analysis . . . 66
5.2.1.1. The Accuracy Paradox . . . 66
5.2.1.2. The Sensitivity of Weighted Loss . . . 67
5.2.1.3. The AUPRC Baseline . . . 68
5.2.2. Logistic Regression Performance Analysis . . . 68
5.3. Evaluation of Full State-Space Machine Learning Architectures . . . 69
5.3.1. Performance Analysis: Generalization vs. Calibration . . . 69

5.3.1.1. The Calibration Divergence (XGBoost vs. Neural Networks) . . . 70
5.3.1.2. Head-Wise Performance Asymmetry . . . 71
5.3.2. The Failure of Isotropic Graph Convolutions (GCN) . . . 72
5.3.3. Topology Impact: The "Ball-as-Node" Dilemma . . . 72
5.3.4. Ablation of Edge Features . . . 72
5.3.5. The Hybrid GAT-LSTM and the Interpretability Trade-off . . . 73
5.4. Stratified Performance and Interpretability Analysis . . . 74
5.4.1. Interpretability: The Physics of Prediction . . . 74
5.4.1.1. Possession as the State Switch . . . 74
5.4.1.2. Kinematic and Role-Based Prioritization . . . 75
5.4.1.3. The Structural Engine: Attention Distribution . . . 76
5.4.2. Stratified Performance: Threat Zone Analysis . . . 78
5.4.2.1. Zone Definition and Distribution . . . 78
5.4.2.2. Comparative Performance Analysis . . . 78
5.4.2.3. Ranked Generalization Dataset . . . 81
6. DISCUSSION AND FUTURE WORK . . . 84
6.1. Critique of Model Performance and Methodological Choices . . . 84
6.1.1. Feature Engineering and State Representation . . . 84
6.1.2. Temporal Horizons and Goal Definitions . . . 86
6.1.3. Data Resolution and Scalability . . . 86
6.2. Future Work and Broader Implications . . . 87
6.2.1. Advanced Architectural Paradigms . . . 87
6.2.2. Interactive and Immersive Analytics . . . 88
6.2.3. Operational Viability and Scalability . . . 89
6.2.4. Rocket League as a Proxy for Sports Analytics . . . 91
7. CONCLUSION . . . 93
REFERENCES . . . 96
APPENDIX A: EXTENDED RANKED EVALUATION BY THREAT CHARACTERIZATION . . . 107

LIST OF FIGURES

Figure 2.1 A Rocket League field (left) and the top-down view of the field (right) with the boost pads marked, six big boost pads with red and twenty-eight small boost pads with white. 10
Figure 2.2 Rocket League 3v3 ranked season 17 skill rating distribution. . . . 11
Figure 4.1 High-level system architecture of the peaRL framework. 27
Figure 4.2 Data preprocessing pipeline from raw replay data to tabular output. 30
Figure 4.3 The Rocket League field coordinate system and dimensions. 33
Figure 4.4 One second of gameplay with a 5Hz sampling rate, showing 6 consecutive states. 35
Figure 4.5 Ball, blue team, and orange team heatmaps made out of all data points from the primary model dataset, with a resolution of 200 cells along the longer axis. 36
Figure 4.6 Tokenization logic with a parcel size of 1024. 39
Figure 4.7 Token mirroring and canonicalization logic on parcel size of 1024. . 41
Figure 4.8 The structure of the 6-node (left) and the 7-node (right) graph structure and the respective PyG data objects to be used in GNN model trainings. 51
Figure 4.9 The unified GNN architecture used for ablation studies. 53
Figure 4.10 Architecture of the baseline LSTM model. 57
Figure 4.11 Schematic of the spatio-temporal GNN-LSTM architecture. 57
Figure 4.12 The peaRL user inteface portal. 59
Figure 4.13 Detailed view of the synchronized goal probability graphs. 61
Figure 4.14 Counterfactual modification tools for the user interface portal. . . 62
Figure 4.15 The human benchmarking module for the user interface portal. . . 63
Figure 5.1 Top 30 feature importances (gains) for the XGBoost dual-models. 75
Figure 5.2 Average attention weight distribution by tactical role for the GAT-7n-att-e2 model. 77
Figure 5.3 Threat characterization logic visualization. 78

Figure 5.4 Stratified AUPRC performance across threat zones for XGBoost, GAT-7n-avg-e2 (GAT-Avg), GAT-7n-att-e2 (GAT-Att), and GAT-LSTM hybrid models for the test set of the primary model dataset. 80

Figure 5.5 Model performance (AUPRC) across competitive ranks from the ranked generalization dataset for XGBoost, GAT-7n-avg-e2 (GAT-Avg), GAT-7n-att-e2 (GAT-Att), and GAT-LSTM hybrid models. 82

Figure A.1 Detailed performance evolution across the ranked generalization dataset, segmented by threat category, for XGBoost, GAT-7n-avg-e2 (GAT-Avg), GAT-7n-att-e2 (GAT-Att), and GAT-LSTM hybrid models. 107

LIST OF TABLES

Table 4.1	Summary of the 92-dimensional input feature vector.	34
Table 4.2	Statistical breakdown of the primary model dataset splits.	37
Table 4.3	Distribution of canonical transformations for validation of symmetry.	41
Table 4.4	Sparsity profile of the tokenized state space for primary model dataset (original) and addition of historical augmentation dataset (augmented).	42
Table 4.5	Performance of the historical baseline on the test set with statistical ledgers using original and augmented datasets.	43
Table 5.1	Hyperparameter configuration and computational cost by model family.	65
Table 5.2	Performance metrics for the trivial baselines on the test set for the primary model dataset.	67
Table 5.3	Performance metrics for the Logistic Regression baselines (univariate and multivariate models) on the test set for the primary model dataset.	68
Table 5.4	Primary model dataset test results for the full state-space ML models.	70
Table 5.5	Distribution of test set states and positive labels across threat zones for primary model dataset.	79

LIST OF SYMBOLS

b	Bias term
B	Ball node
$\mathcal{C}$	Configuration
d_{ij}	Euclidean distance between entity i and entity j
$deg(i)$	Degree of node i
$\mathcal{E}$	Set of edges
$\mathbf{e}_{ij}$	Edge feature vector connecting node i to node j
$\vec{f}_{fwd}$	Unit forward vector for vehicle orientation
f_{gate}	Learnable linear transformation
$\mathbf{g}$	Global context feature vector
$\mathcal{G}$	Graph topology
$\mathbf{h}_i$	Latent feature vector of node i
$\mathbf{h}'_i$	Updated node feature vector after message passing
$\mathbf{h}_j$	Input feature vector of a neighbor node j
$\mathbf{h}_G$	Pooled graph-level embedding vector
$h_{temporal}$	Hidden state vector of the LSTM network
$\mathbb{I}(\cdot)$	Indicator function
k	Number of distinguishable parcels
$\mathcal{L}$	Loss Value of Weighted Binary Cross-Entropy
n	Number of indistinguishable players within a team
N	Total number of nodes in a graph
N_b	Total number of samples in a batch
N_{neg}	Count of negative samples inside the training set
$N_{parcels}$	Total number of discrete parcels in the spatial grid
N_{pos}	Count of positive samples inside the training set
N_{team}	Number of possible unique team configurations
$\mathcal{N}(i)$	Set of immediate neighbor nodes for node i
p	Predicted probability score or dropout

$\mathbf{p}_{ij}$	Relative position vector from node i to node j
P_i	Player i node
$\mathbf{p}_{raw}$	Raw positions
$\mathbf{p}_{unity}$	Unity vectors
p_x, p_y, p_z	Discrete indices of a spatial parcel in the grid
q	Quaternion orientation vector
$\|S\|$	Cardinality of the set of all possible unique state tokens
t	Time step index
T	Transpose operation
$\mathbf{v}_i$	Velocity vector of entity i
v_{max}	Maximum achievable velocity
$\mathcal{V}$	Set of nodes
w	Real component of a quaternion orientation vector
w_{pos}	Positive class weight coefficient
$\mathbf{w}$	Learnable weight vector
$\mathbf{W}$	Learnable weight matrix
$\mathbf{x}$	Input feature vector
$\mathbf{x}_{poly}$	Feature vector expanded with polynomial terms
x, y, z	Cartesian coordinates in 3D space
x_{norm}	Feature value after Min-Max normalization
$\hat{y}$	Model prediction output
$\hat{y}_i$	Raw logit output for sample i
y_i	Ground truth binary label for sample i
α_i	Attention coefficient for node i
Δs	Size of the discretization parcel
ϵ	Constant ($1e^{-8}$)
η	Learning rate optimization parameter
Φ	Discretization floor function
$\sigma(\cdot)$	Sigmoid function

LIST OF ACRONYMS/ABBREVIATIONS

2D	Two Dimensional
3D	Three Dimensional
AI	Artificial Intelligence
AUPRC	Area Under Precision-Recall Curve
BCE	Binary Cross-Entropy
CICD	Continuous Integration and Continuous Delivery
CM	Center of Mass
CPU	Central Processing Unit
CS:GO	Counter-Strike: Global Offensive
CSV	Comma-Separated Value
Dota	Defense of the Ancients
DRL	Deep Reinforcement Learning
FPS	First-Person Shooter
GAT	Graph Attention Network
GBDT	Gradient Boosted Decision Tree
GCN	Graph Convolutional Network
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
ID	Identifier
JSON	JavaScript Object Notation
LLM	Large Language Model
LoL	League of Legends
LR	Learning Rate
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multilayer Perceptron
MMR	Matchmaking Ranking

MOBA	Multiplayer Online Battle Arena
MTL	Multi-Task Learning
NaN	Not a Number
NHL	National Hockey League
ONNX	Open Neural Network Exchange
peaRL	Predictive & Exploratory Analysis in Rocket League
PyG	PyTorch Geometric
ReLU	Rectified Linear Unit
RL	Rocket League
RLCS	Rocket League Championship Series
SOTA	State-of-the-Art
SSL	Supersonic Legend
STRN	Spatial-Temporal Relation Networks
UI	User Interface
USD	United States Dollar
uu	Unreal Units
UX	User Experience
WPA	Win Probability Added
XAI	Explainable Artificial Intelligence
xG	Expected Goals
XGB	Extreme Gradient Boosting
XR	Extended Reality

1. INTRODUCTION

1.1. High-level Description of Esports

The rapid professionalization of esports has established a multi-billion dollar industry where competitive margins are exceptionally thin [1, 2]. Consequently, esports organizations are increasingly turning to data analytics to gain a competitive edge, mirroring a practice long established in traditional sports [3, 4]. The data-rich nature of esports, with game engines capable of recording every player action and positional change, presents a fertile ground for sophisticated analytical techniques.

1.2. Need for a Solution

However, the tools available to players and coaches, particularly in high-speed, physically complex games like Rocket League (RL), have largely remained in the realm of descriptive, post-hoc analysis. Existing systems, such as ballchasing [5] or CARL2 [6], provide valuable statistics and replay-viewing capabilities, but they primarily show what happened. This leaves the deeper analysis (why it happened, what could have been done differently, and what are their consequences are) as a time-consuming, manual, and intuitive process of replay or video review.

1.3. Motivation

While there have been significant strides made in developing predictive models for traditional sports and esports (e.g., forecasting match outcomes [7–11] or valuing player actions [12–14]), this work often results in a static metric or a research finding rather than a tool for practical application. A critical gap therefore exists between the potential of predictive analytics and the practical needs of players and coaches [15]. There is a distinct lack of systems that bridge this gap, moving beyond descriptive reports and static predictions to enable a new paradigm of prescriptive and interactive

analysis, which is a gap explicitly identified [16]. Players and coaches require tools that not only identify past mistake but also provide an interactive "sandbox" to explore potential solutions, thereby building a deeper, data-driven intuition for complex game states [15, 17, 18].

1.4. Overview of the Method

To address this gap, we present peaRL (Predictive & Exploratory Analysis in Rocket League), an interactive decision-support system for the analysis of Rocket League gameplay. Our platform integrates a high-fidelity 3D replay viewer, built in the Unity engine, with a real-time machine learning (ML) inference engine. The core of our system is a predictive model trained on 6,299 professional matches to calculate, for any given moment, the probability of a goal being scored by either team within the next five seconds.

1.5. Main Contributions

This thesis makes the following primary contributions to the field of esports analytics and interactive systems:

- A Systematic Comparison of Predictive Models: We present a large-scale empirical comparison of predictive models for Rocket League, spanning heuristic, neural, and graph-based approaches, including spatio-temporal architectures. While learned models substantially outperform statistical baselines, their similar performance reveals nuanced trade-offs between spatial and temporal inductive biases.
- An Adaptation of Heuristic State Tokenization for an Interpretable Baseline: We adapt and benchmark an interpretable non-ML baseline, inspired by similar state-discretization methods in traditional sports, that maps the complex 3D game state into tokens to derive historical probabilities. This provides a strong, explainable baseline for validating our machine learning approaches and offers rule-based insights into the high-speed dynamics of Rocket League.

- A Novel Interactive System for Prescriptive and Counterfactual Analysis: We design and implement a complete, user-centric platform that moves beyond descriptive analysis. Its "What-If" feature provides a unique environment for counterfactual exploration, which enables players and coaches to quantitatively assess the impact of tactical decisions and rehearse improved strategies.
- An Open-Source Platform for Reproducible Research: We release the entire platform, including the replay analysis pipeline, the implementation of our predictive models, and the Unity-based interactive portal, as an open-source project. This provides the community with an extensible framework and a reproducible benchmark for future research into prescriptive analytics and human-AI coaching systems in esports.

1.6. Thesis Organization

The remainder of this thesis is organized as follows:

- Chapter 2 provides background information on key concepts relevant to this work. It covers the esports domain and Rocket League's place in it, and an overview of Rocket League as a game, including its mechanics, ranking system, and replay data structure.
- Chapter 3 reviews prior research in areas relevant to this thesis. It includes descriptive and statistical sports analytics, existing research on machine learning for prediction, and interactive systems for counterfactual analysis techniques used in similar domains.
- Chapter 4 details the technical implementation of the proposed system. It describes the Data Preprocessing Pipeline used to standardize raw replay logs, the State Tokenization framework for historical probabilities, and the design of the various model architectures. The chapter concludes with an overview of the User Interface Portal.
- Chapter 5 presents the experimental results and analysis. It begins with the experimental setup and standardization of metrics, followed by an assessment

of trivial baseline models. The core predictive performance is analyzed in the ML Models section. The chapter concludes with a stratified performance and interpretability analysis.

- Chapter 6 discusses key findings, limitations of the current approach, and potential directions for future research in traditional sports and esports analytics and interactive predictive modeling.
- Chapter 7 summarizes the contributions of this thesis, emphasizing its impact on analytics and the feasibility of predictive tools for Rocket League.

2. BACKGROUND

This chapter provides background information on the broader domain of esports, situates Rocket League within this context, and presents a foundational overview of the game itself.

2.1. The World of Esports

Esports, or electronic sports, refers to organized competitive gaming at professional and amateur levels. It has rapidly grown into a global phenomenon with significant cultural and economic impact. This section introduces the broader esports landscape to establish the context for analyzing Rocket League within this domain.

2.1.1. The Rise of Esports

Esports is a form of competition using video games, most commonly in organized, multiplayer formats between professional players and teams. While casual video gaming is a leisure activity, esports is distinguished by its focus on structured competition, often within leagues and tournaments, with the goal of entertaining a viewership, promoting rivalry, and achieving professional-level performance. What began as informal gatherings in arcades and internet cafes has been transformed by high-speed internet and advanced gaming technology into a full-blown international industry [19].

The scale of this transformation is significant. The global esports market was valued at approximately USD 2.13 billion in 2024 and is projected to grow to USD 7.46 billion by 2030 [20]. This economic expansion is fueled by a massive and rapidly growing global audience, which is expected to surpass 640 million people by 2025 [21]. This growth has been driven by several key factors, including the proliferation of live-streaming platforms like Twitch, increased investment in the infrastructure for professional league tournaments, and a rise in engagement activities that foster strong

fan communities [22].

This rapid professionalization has spurred a commensurate demand for sophisticated, data-driven performance analysis, mirroring the "analytical revolution" that has reshaped traditional sports over the past two decades [23]. Just as baseball's "Moneyball" era demonstrated that data could provide a competitive edge by uncovering undervalued assets and optimizing strategy, esports organizations are now increasingly investing in analytics to gain a similar advantage [24, 25]. Teams and analysts now leverage vast quantities of in-game data to refine strategies, enhance player performance, and make more informed decisions. This establishes sports analytics as a new and critical discipline within the competitive gaming industry [26].

2.1.2. The Esports Landscape

Just as "sports" is an umbrella term for a wide range of athletic contests like swimming and basketball, "esports" encompasses a diverse landscape of competitive video games, each with its own dedicated player base, strategic depth, and professional ecosystem. The most prominent of these are the Multiplayer Online Battle Arenas (MOBAs) like League of Legends and Dota 2, complex team games that are often likened to immense strategic planning. Equally popular are the First-Person Shooters (FPS) such as Counter-Strike and Valorant, which test players', aiming precision, and team tactics in objective-based scenarios.

Beyond these titans, the landscape includes a wide array of other competitive titles. Battle Royales like Fortnite and Apex Legends focus on survival and improvisation on a massive scale, while fighting games like Street Fighter offer a pure, one-on-one test of execution and psychological strategy. The spectrum of esports further extends to sports simulation titles, high-speed racing games, and even unconventional competitions centered on games like GeoGuessr. This proves that nearly any game with a high skill ceiling can foster a competitive scene.

Among this diverse landscape, Rocket League has carved out its own unique and highly successful niche. Unlike any other title, it blends the familiar concepts of soccer with a unique, physics-based mechanics which creates a game that is simple to understand yet almost limitless in its strategic and mechanical depth. Its standing as a major global esport is demonstrated by quantifiable metrics. As of October 2025, it ranked 9th among all esports in total prize money awarded for the year (over USD 6.5 million) and 14th in peak viewership (just under 400,000) for a single event [27]. This indicates a stable, well-established, and highly competitive professional scene.

2.1.3. The Professional Ecosystem

A key factor that elevates a competitive video game to the status of a premier esport is the development of a stable and structured professional ecosystem. This mirrors the organization of traditional sports, consisting of several core components. Esports organizations act as the teams, signing top players to professional contracts and providing salaries, coaching, and support staff. These teams then compete in scheduled leagues and tournaments, often culminating in a world championship event with significant financial stakes.

The health and significance of a game's professional scene are often quantified by two key metrics: the total prize money awarded in its tournaments and the viewership numbers for its major events. A large prize pool indicates a high level of investment from developers and sponsors (and the community), while high viewership demonstrates strong public engagement. These factors create a high-stakes environment where peak performance is paramount.

Rocket League exemplifies a mature competitive ecosystem, anchored by its long-running official league, the Rocket League Championship Series (RLCS). For over a decade, RLCS has consistently offered a regular season of online league play for teams across multiple regions, culminating in live international tournaments and a definitive World Championship. While the amount of team members varies across esports titles,

most of the Rocket League tournaments are played in a 3-versus-3 format [28].

Crucially for this research, the existence of this professional circuit provides a consistent and invaluable source of high-quality replay data. The game states saved after these RLCS matches represent the pinnacle of human strategic play under immense pressure and makes this dataset the ideal foundation for training a predictive model aimed at understanding and analyzing expert-level gameplay.

2.2. Rocket League

Rocket League is a physics-based vehicular soccer video game developed and released by Psyonix in 2015. Rocket League runs on Unreal Engine 3, a game engine developed by Epic Games, with extensive modifications by Psyonix to support advanced physics-based vehicle movement and real-time multiplayer interactions. In a standard match, six players are split into two teams, each represented by a distinct color, blue or orange. Players control rocket-powered cars to hit a large ball and score goals within a symmetrical arena, which features a goal on each side, as seen in section 2.1. Matches last five minutes, with golden goal overtime if the score is tied at the end of regulation. A kickoff is set at the beginning of the game, after each goal, and at the beginning of overtime. All of the entities in the field reset at a kickoff, and all players are placed into predefined symmetrical kickoff positions with the ball in the middle of the field. This means that every gameplay sequence separated by a kickoff is completely unrelated; everything other than the scoreline and the time resets.

The game offers a high degree of mechanical skill, as cars can drive, jump, dodge, and even fly using their built-in rocket boost. Each vehicle can store up to 100 boost points obtained by driving over 34 boost pads scattered throughout the field, shown in section 2.1. These pads are categorized into small (providing 12 boost points) and large (fully refilling 100 boost). Players can maneuver in midair by rotating their car, and by combining midair rotation with boost usage, they can achieve controlled aerial movement. Additionally, players can demolish opponents by colliding with them at



Figure 2.1. A Rocket League field (left) and the top-down view of the field (right) with the boost pads marked, six big boost pads with red and twenty-eight small boost pads with white.

high speeds, forcing them to respawn after a three-second delay in their defensive zone. The complex interplay between the physics interactions and finite resource management (boost) creates a highly dynamic state space. It is this moment-to-moment complexity that makes manual analysis difficult and presents an ideal challenge for a data-driven predictive model.

2.2.1. Cars and Hitboxes

While Rocket League offers over 100 different car models, the game's physics engine simplifies car interactions by using six distinct hitbox types. These hitboxes vary in dimensions (length, width, and height), handling characteristics, ground clearance, and inclination angles. While Psyonix has not officially disclosed the exact physical parameters of each hitbox, the community has extensively tested and analyzed their differences to optimize competitive play. It is also notable that the dynamic data, such as position, velocity, and orientation, are universal to all car types, and the key statistics, like top speed and acceleration, are the same for every car.

2.2.2. Ranking System

Rocket League employs a competitive ranking system to assess and match players based on skill level in online ranked play. A player's skill rating is quantified by a

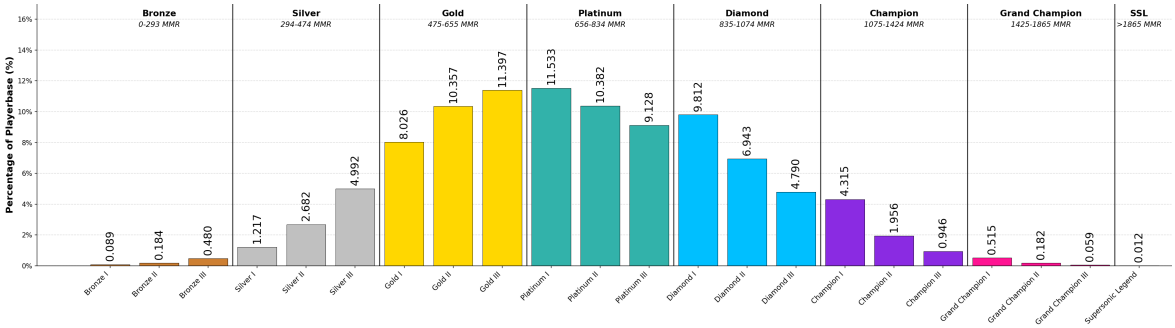


Figure 2.2. Rocket League 3v3 ranked season 17 skill rating distribution.

Matchmaking Ranking (MMR), a numerical value that determines their placement in the ranking tiers. Each rank, except for Supersonic Legend (SSL), consists of three sub-ranks, such as Bronze I, Bronze II, and Bronze III. Furthermore, each sub-rank is divided into four divisions (e.g., Bronze I Division 1 through Bronze I Division 4), creating a structured progression system, seen in subsection 2.2. The data seen in this figure is collected from an online tracker repository [29]. The dataset used for this training of the predictive models is sourced exclusively from professional players who compete at SSL and beyond, which represents the absolute peak of this skill distribution.

The ranking system dynamically adjusts a player's MMR based on match outcomes, with wins increasing MMR and losses decreasing it. Additionally, different game modes (such as 1v1, 2v2, and 3v3) maintain separate MMR values, allowing players to hold distinct ranks across various modes. subsection 2.2 illustrates the approximate distribution of the 6.8 million players' ranks in the 3v3 game mode on the last day of Season 17. A season usually goes on for four months, after which the ranks of the players go under a soft reset.

2.2.3. Replay System and Data Collection

Rocket League's replay system allows players to review past matches by storing gameplay data in replay files. These files do not contain pre-rendered video but instead store structured match data that the game engine uses to reconstruct the match when

played in replay mode. This approach enables a dynamic, interactive replay experience where users can change camera angles, slow down or speed up playback, and jump between in-game events.

The replay files contain various forms of structured data, including:

- Player actions: Position, velocity, boosts used, jumps, dodges, and demolitions.
- Ball physics: Position, speed, spin, and interactions with cars.
- Game events: Goals, assists, shots, saves, and other notable events.
- Boost pad states: Locations of boost pads and their recharge timings.
- Match metadata: Players' ranks, match duration, and team compositions.

Since the replay files store raw game data, they need to be parsed using specialized tools to extract meaningful insights. Several open-source parsers exist to turn Rocket League replays into a human-readable format, such as:

- Boxcars [30]: A Rust-based fast parser optimized for bulk replay analysis.
- CPPRP [31]: A C++ parser designed for integration into game analysis tools.
- RocketLeagueReplayParser [32]: A Python-based tool for extracting structured data from replays.
- Rattletrap [33]: A parser written in Haskell that supports detailed replay breakdowns.
- carball [34]: A Python-based framework for parsing and analyzing Rocket League replays that provides high-level event reconstruction and detailed statistical data extraction.

These parsers enable researchers and developers to extract data such as player positioning, ball trajectories, and team coordination patterns. Services like ballchasing.com [5] rely on these parsers to provide visual and statistical analysis of matches.

These parsers are essential for enabling research and development of analytical

tools. However, the replay analysis process is not without its challenges. The Rocket League replay format is proprietary and is frequently updated by the developers alongside major game patches. This format volatility requires constant maintenance and debugging from the open-source community to ensure that parsers remain compatible and can accurately extract data from the latest version of the game. Despite these hurdles, the continuous improvement of these tools provides the foundation for the gameplay analysis undertaken in this thesis.

2.2.4. Unity and Unreal Game Engines

Rocket League itself runs on Unreal Engine, while the interactive portal for this thesis is implemented using the Unity game engine. Unity and Unreal rank among the most widely used real-time, cross-platform 3D development environments. Though often associated primarily with games, their capabilities extend into architectural visualization, engineering simulations, and interactive data analysis: both provide mature physics engines, rendering pipelines, and scripting systems [35, 36].

What makes Unity particularly well-suited for this project is its flexibility in integrating Python within its development pipeline, specifically for the replay conversion and ML model inference tasks. While Unity's Python support is not typically for runtime gameplay logic, it enables automation and data processing workflows in the Editor or as external processes, which aligns well with the needs of pre-processing match replays. Moreover, Unity has a broad ecosystem of online resources, plugins, and community support, which makes development and debugging faster and more accessible. In contrast, Unreal Engine, while extremely powerful for high-fidelity, graphically complex scenarios, has a steeper learning curve (C++/Blueprint) and may not offer the same convenience for tooling [37].

3. RELATED WORK

3.1. Descriptive and Statistical Sports Analytics

The pursuit of understanding what happened in a sports match has created a rich field of descriptive and statistical analysis. This field spans from foundational spatio-temporal theory to the practical and data-rich tools. It also includes diverse computational models, specific methods for pattern discovery, and data engineering work.

3.1.1. Foundational Surveys and Frameworks

To ground our work, we first review the foundational surveys that have defined and structured the field of spatio-temporal analytics in both traditional and digital sports (esports). The analysis of traditional team-based invasion sports (such as soccer and basketball) has been formalized in comprehensive surveys [4]. This work provides a coherent framework for the field. It categorizes research efforts that utilize high-frequency spatio-temporal data into key areas such as playing area subdivision, network-based analysis of team structure, data mining for tactical patterns, and the development of novel performance metrics. Parallel to this, the field of game analytics has developed its own specializations for digital games with foundational reviews that outline techniques such as heat maps and trajectory analysis for unconventional playing fields and dimensions [38]. These surveys define the interdisciplinary mix of spatio-temporal theory and data-driven analysis that our work builds upon.

3.1.2. Publicly Available Tools for Rocket League

The practical analytics for the Rocket League community began with data-rich platforms for post-hoc descriptive analysis. The most prominent is the web-based repository ballchasing.com [5], which parses user-uploaded replays to provide an exten-

sive variety of statistics alongside a simplified 3D replay viewer. This is complemented by CARL2 [6], which offer a suite of tools for players and coaches, including detailed statistical breakdowns, pointing out data extremes, and live-mode data tracking.

Recently, this descriptive analysis has been integrated into AI-driven coaching platforms. Tools like RL DataCoach [39] and trophi.ai [40] ingest replay data to benchmark a player's performance against others of similar rank. Their AI models analyze these descriptive stats to identify skill gaps and provide prescriptive training modules. While these platforms are invaluable for showing what happened and prescribing what to practice, they lack the in-place, interactive, and counterfactual simulation capabilities for exploring why a specific tactical moment failed and what-if a different decision had been made.

3.1.3. Statistical and Computational Modeling

Beyond descriptive tools, formal computational and statistical models have been used to analyze gameplay. In the Rocket League domain, traditional collective behavior metrics from soccer have been applied to assess team dynamics and their correlation with performance [41]. In traditional sports, this includes "first-principles" approaches, such as using computational geometry (e.g., Voronoi diagrams) to define and quantify dominant regions and teamwork [42]. Other discovery-oriented methods have used unsupervised learning, like minimum entropy partitioning, to identify latent player roles and team formations from large-scale tracking data [43]. A foundational method for valuing a game state, which informs our own baseline heuristics, is the use of stochastic process models. This work pioneered discretizing the game state into "unit-squares" to calculate the expected value of a basketball possession. This provides a theory-driven and non-ML approach to state valuation [44], of which we build our statistical ledger system upon.

3.1.4. Methodologies for Descriptive Analysis

A significant challenge in descriptive analysis is reducing the complexity of continuous gameplay into meaningful units. One key approach is "encounter detection," a method for automatically segmenting a match into discrete and analyzable chunks (e.g., team fights in MOBA games) for post-hoc analysis [45]. Within these segments, other methods have focused on discovering specific repeated action sequences, such as behavioral pattern mining to identify model "skillshots" in Rocket League [46]. However, these automated quantitative methods often lose the "why" behind player actions. This has lad to mixed-method approaches that combine sequence analysis with human-in-the-loop visualization, which argues that preserving strategic context is critical for understanding the goals and motivations behind player behavior [47].

3.1.5. Data Engineering and Preparation

Finally, the analyses in all previously mentioned works depend on a data engineering pipeline. Similar to how video footage in traditional sports are not directly usable for analysis, raw replay files, such as those for CS:GO or Rocket League, also require some form of preprocessing for an analysis-ready dataset. Research in the esports domain has highlighted the process of composing these raw files into datasets as a foundational workflow necessary to enable data exploration [48]. This effort is a necessary precursor to the systems-based contributions of this thesis.

3.2. Machine Learning for Prediction

Machine learning for prediction is a field focused on forecasting outcomes and evaluating performance in sports and esports. We cover foundational models in Rocket League, broader outcome prediction in other domains, and models for high-level metagame strategy. Then, we detail the advanced graph-based, generative, and spatio-temporal methodologies that inform our own framework.

3.2.1. Predictive Modeling in Rocket League

The application of machine learning directly to Rocket League has followed established paths from traditional sports analytics, as it focuses on notational analysis and static shot valuation. Research in this area has successfully used Random Forest models to identify which aggregate metrics (e.g., shots taken, time spent exceeding a speed threshold) best predict match outcome or player rank [8]. Following a parallel path from soccer analytics, other work has applied XGBoost to develop an 'expected goals' (xG) model, used for predicting the probability of a goal from a static snapshot of a shot attempt [12]. These studies are foundational in using ML for static valuation in the domain, as distinct from the dynamic and situational threat assessment our work addresses.

3.2.2. Outcome and Performance Prediction in Other Sports and Esports

3.2.2.1. Goal and Win Probability. To situate our model in the broader landscape, we first review predictive models from traditional sports. Using methods like XGBoost to predict the shot outcomes [10] or Bayesian approaches to model in-game win probability [11]. Similarly, an extensive line of research exists in espots. Standard models have been developed to predict match outcomes from pre-game data in League of Legends [49] and to predict round-by-round winners in CS:GO [7]. These win probability models also serve as analytical instruments for studying player behavior across different skill levels [50]. In MOBAs line Dota 2, this has culminated in real-time prediction models intended for live broadcasts [9, 16]. While this body of work is successful at producing a static predictive metric, with some explicitly note the need for future work to make these predictions actionable for players and coaches [16].

3.2.2.2. Player and Action Valuation. A direct extension of win probability models is to value individual player contributions. The most established framework for this is Win Probability Added (WPA), which defines an action's value as the change in win probability it causes, which as been adapted for esports like CS:GO to value damage

events and player eliminations [13]. A related approach models the expected outcome of a specific event; for instance, in Valorant, predictive models are used to calculate the probability of winning a duel, which is then used to derive a "Wins Above Expected" metric for player evaluation [51]. Other methods beyond WPA, using deep learning sequence models to embed player actions into a quantitative contribution score [52] or applying game-theoretic methods like Shapley values to allocate a team's win to individual players [14]. These models attempt to isolate an individual's impact from their team's to provide a static metric of player performance.

3.2.3. Meta-Game and Strategic Prediction

A separate branch of predictive analytics focuses not on continuous gameplay, but on the high-level "meta-game" decisions that frame a match. This is prominent in tactical shooters like CS:GO, where research has used win probability models to optimize pre-round economic decisions [53] and contextual bandit models to optimize the pre-match map selection (pick/ban) process [54]. This line of inquiry is more extensive in MOBA games, which have a significant body of work dedicated to modeling and predicting the outcome of the pre-match "hero draft" [55]. This entire category of pre-game strategic prediction is largely absent in Rocket League, where the only meaningful pre-match choice is the car selection, which players rarely change.

3.2.4. Advanced Methodologies for Prediction

3.2.4.1. Graph-Based Modeling for Sports. A more advanced methodology, directly related to our own, represents game states as graphs to model complex inter-player relationships. GNNs have been proposed as a sport-agnostic framework to preserve permutation invariance and learn from player interactions. It shows improved predictive performance in both American football and CS:GO [56]. This work also explicitly proposes "what-if" analysis as a potential application. In esports, GCNs have been used in a semi-supervised fashion to model an entire LoL league structure, as the system learns from the graph of team relationships to predict match outcomes [57]. More

traditional graph-based models, using graph theory principles like eigenvector centrality and random walks rather than neural networks, have also been used to model team strength and predict outcomes in international soccer tournaments [58].

3.2.4.2. Reinforcement Learning for Automation (AI Bots). In alignment to the goal of "AI for analysis" is the "AI for automation" track, which uses Deep Reinforcement Learning to create superhuman bots that play the game. In Rocket League, this has been an active area of research. State-of-the-art (SOTA) bots such as Seer have been trained using Proximal Policy Optimization and self-play to achieve high ranks, with some work also exploring learning from human replays as a prior [59]. Other research has focused on sample efficiency, which develops novel reward shaping functions and auxiliary tasks to train competitive bots like Lucy-SKG, which surpassed previous SOTA agents in fewer steps [60]. Another approach involves sim-to-sim transfer, where agents are trained on specific skills, like striking or goalkeeping, in a simplified and fast simulation (e.g., in Unity) before the learned policy is transferred to the full game engine [61]. This entire body of work is distinct from our goal, as our system aims to augment the human coach and player.

3.2.4.3. Generative Modeling for Behavioral Stylometry. A distinct track involves creating generative models to imitate individual human playstyles. This behavioral stylometry aims to create models that do not just play optimally, but play like a specific person. Recent work in this area has been applied directly to Rocket League. By using parameter-efficient fine-tuning methods, a single base model can be conditioned on a learned "style vector" to generate actions in the style of a specific player [62]. This approach is focused on imitation and identification.

3.2.4.4. Foundational Theory for Spatio-Temporal GNNs. Our predictive model's architecture is built upon foundational work in graph representation learning. The core convolutional operations are derived from principles of GCNs, which established a scalable propagation rule for learning on graph-structured data [63]. To capture the

dynamic nature of Rocket League, spatio-temporal reasoning principles can be integrated into the model's design. One example of this is Spatial-Temporal Relation Networks (STRN), which provide a unified framework for encoding cues across both space and time [64]. Furthermore, our model must predict a holistic graph-level property (i.e., goal probability within a time window) rather than node-level properties. This builds on work that solves graph-level representation, for instance, by introducing a "super node" to aggregate features from all nodes [65]. The scalability and relevance of this general approach are demonstrated in parallel domains such as urban traffic, where spatio-temporal foundation models are built to handle complex and large-scale forecasting [66].

3.3. Interactive Systems and Counterfactual Analysis

The preceding review of the literature established that the vast majority of contemporary sports analytics research, spanning from descriptive tools to advanced predictive models, ultimately culminates in the production of a static metric. Such metrics, while informative, are non-interactive and lack prescriptive utility for the end user. This section, therefore, surveys the body of research that directly informs the primary contribution of this thesis: systems designed not for passive summarization, but for active, human-in-the-loop interaction.

This domain of research covers systems that enable users to probe the underlying logic of predictive models, facilitating an understanding of why a prediction was made. More importantly, it includes frameworks for exploring hypothetical, counterfactual "What-If" scenarios. This literature, which is situated at the intersection of Human-Computer Interaction (HCI), Information Visualization, and Explainable AI (XAI), provides the direct academic context for our work and frames this thesis as a novel contribution to an emerging research frontier.

3.3.1. Interactive Systems for "What-If" Analysis

This subsection reviews the most direct peers to our proposed system: interactive platforms that are explicitly designed to enable "what-if" or counterfactual analysis in sports. Research in soccer, for instance, has produced computational and visual systems that can automatically identify suboptimal player movement from tracking data and then compute and suggest an improved alternative trajectory that would have led to a better outcome [17].

Other interactive soccer analytics platforms have been developed featuring an interactive replay and modification feature. This functionality allows an analyst to pause a game replay at a critical moment, manually alter the game state by dragging a player to a new position, and thus create a "what-if" scenario. Upon this user-initiated modification, the system recalculates its underlying predictive models in real-time to provide immediate feedback on the altered state [18]. Another system, Smartboard, employs Large Language Models (LLMs) as the core simulation engine. In this framework, a human coach can sketch a tactical play on an interactive board. A specialized "AI coach agent" then receives this multimodal input, reasons about the specified tactic, and simulates the probable outcomes and opponent responses, thereby providing a sandbox for strategic exploration [67].

3.3.2. The Foundation of Counterfactual Exploration and XAI

Our "what-if" system is also grounded in the established academic fields of XAI and Information Visualization, which seek to solve the "black box" problem. One major line of research, distinct from our interactive approach, is XAI via model distillation. In this paradigm, a complex and opaque model, such as a Deep Reinforcement Learning (DRL) network for valuing player actions in soccer [68], or the NHL [69], is first trained to achieve high accuracy. Then, its "knowledge" is "distilled" into a simpler, transparent model, such as a linear model tree, which can be interpreted by experts [70]. A parallel line of research, originating in the visualization community, pro-

poses using "counterfactual explanations" directly within interactive systems. Works like DECE [71] and CoFact [72] demonstrate that visualizing counterfactual subsets helps users probe a model's decision boundaries and protects them from making mistaken assumptions about the data. Our thesis builds on these foundations by using an interactive counterfactual system as the primary method of explanation.

3.3.3. Interactive Systems for Exploratory Game Analytics

Separate from predictive "what-if" systems, a large body of work exists on interactive systems for purely exploratory visual analysis. Foundational systems like PLATO [73] and Playtracer [74] established the value of visualizing gameplay data using interactive graphs and state projections to identify player strategies. This paradigm has been extended into domain-specific tools, such as Rumble Flow++ for the visual analysis of Dota 2 team fights [75] and systems like NBA GameViz for supporting data-driven basketball journalism [76]. A key aspect of these tools is their novel interaction methods. For example, "Chalkboarding" [77] and ggViz [78] both introduce a paradigm where analysts can retrieve specific plays by sketching player trajectories or positions, much like a coach on a whiteboard. Other systems, like SportSense, focus on creating interactive retrieval systems that seamlessly combine video (qualitative) with tracking data (quantitative) [79]. This area of research also provides the direct justification for our system's 3D interface. Work on "3D Hexglyph Maps" has demonstrated the utility of immersive analytics in virtual reality for analyzing LoL data [80], while the Wichita TacReel framework explicitly uses a web-based 3D player for analyzing combat flight simulator telemetry [81]. Finally, work on "Play and Viz" provides the academic basis for using entertainment game engines, such as the Unity engine used in our own work, as a platform for interactive data visualization [82].

3.3.4. User-Centered Design and Communication

Finally, the development of such interactive systems is grounded in user-centered design. Research on decision support for Rocket League has used interviews with expert

coaches to identify tThe preceding review of the literature established that the vast majority of contemporary sports analytics research, spanning from descriptive tools to advanced predictive models, ultimately culminates in the production of a static metric. Such metrics, while informative, are non-interactive and lack prescriptive utility for the end user. This section, therefore, surveys the body of research that directly informs the primary contribution of this thesis: systems designed not for passive summarization, but for active, human-in-the-loop interaction.

This domain of research covers systems that enable users to probe the underlying logic of predictive models, facilitating an understanding of why a prediction was made. More importantly, it includes frameworks for exploring hypothetical, counterfactual "What-If" scenarios. This literature, which is situated at the intersection of Human-Computer Interaction (HCI), Information Visualization, and Explainable AI (XAI), provides the direct academic context for our work and frames this thesis as a novel contribution to an emerging research frontier.

3.3.5. Interactive Systems for "What-If" Analysis

This subsection reviews the most direct peers to our proposed system: interactive platforms that are explicitly designed to enable "what-if" or counterfactual analysis in sports. Research in soccer, for instance, has produced computational and visual systems that can automatically identify suboptimal player movement from tracking data and then compute and suggest an improved alternative trajectory that would have led to a better outcome [17].

Other interactive soccer analytics platforms have been developed featuring an interactive replay and modification feature. This functionality allows an analyst to pause a game replay at a critical moment, manually alter the game state by dragging a player to a new position, and thus create a "what-if" scenario. Upon this user-initiated modification, the system recalculates its underlying predictive models in real-time to provide immediate feedback on the altered state [18]. Another system, Smartboard,

employs Large Language Models (LLMs) as the core simulation engine. In this framework, a human coach can sketch a tactical play on an interactive board. A specialized "AI coach agent" then receives this multimodal input, reasons about the specified tactic, and simulates the probable outcomes and opponent responses, thereby providing a sandbox for strategic exploration [67].

3.3.6. The Foundation of Counterfactual Exploration and XAI

Our "what-if" system is also grounded in the established academic fields of XAI and Information Visualization, which seek to solve the "black box" problem. One major line of research, distinct from our interactive approach, is XAI via model distillation. In this paradigm, a complex and opaque model, such as a Deep Reinforcement Learning (DRL) network for valuing player actions in soccer [68], or the NHL [69], is first trained to achieve high accuracy. Then, its "knowledge" is "distilled" into a simpler, transparent model, such as a linear model tree, which can be interpreted by experts [70]. A parallel line of research, originating in the visualization community, proposes using "counterfactual explanations" directly within interactive systems. Works like DECE [71] and CoFact [72] demonstrate that visualizing counterfactual subsets helps users probe a model's decision boundaries and protects them from making mistaken assumptions about the data. Our thesis builds on these foundations by using an interactive counterfactual system as the primary method of explanation.

3.3.7. Interactive Systems for Exploratory Game Analytics

Separate from predictive "what-if" systems, a large body of work exists on interactive systems for purely exploratory visual analysis. Foundational systems like PLATO [73] and Playtracer [74] established the value of visualizing gameplay data using interactive graphs and state projections to identify player strategies. This paradigm has been extended into domain-specific tools, such as Rumble Flow++ for the visual analysis of Dota 2 team fights [75] and systems like NBA GameViz for supporting data-driven basketball journalism [76]. A key aspect of these tools is their novel interaction

methods. For example, "Chalkboarding" [77] and ggViz [78] both introduce a paradigm where analysts can retrieve specific plays by sketching player trajectories or positions, much like a coach on a whiteboard. Other systems, like SportSense, focus on creating interactive retrieval systems that seamlessly combine video (qualitative) with tracking data (quantitative) [79]. This area of research also provides the direct justification for our system's 3D interface. Work on "3D Hexglyph Maps" has demonstrated the utility of immersive analytics in virtual reality for analyzing LoL data [80], while the Wichita TacReel framework explicitly uses a web-based 3D player for analyzing combat flight simulator telemetry [81]. Finally, work on "Play and Viz" provides the academic basis for using entertainment game engines, such as the Unity engine used in our own work, as a platform for interactive data visualization [82].

3.3.8. User-Centered Design and Communication

Finally, the development of such interactive systems is grounded in user-centered design. Research on decision support for Rocket League has used interviews with expert coaches to identify their specific informational needs and decision-making processes, providing a direct conceptual framework for building tools that coaches will find useful [15]. This human-in-the-loop approach is critical. Work on co-designing analytics systems for MOBA games, for instance, highlights the necessity of a multi-phased process to bridge the gap between expert domain knowledge and the capabilities of machine learning [83]. Other interactive systems, such as SNIL [84], are designed not for expert analysis but for communication, using LLMs to automatically generate data-driven sports news articles. This body of work underscores the importance of designing systems with a clear understanding of the end-user's goal, whether it be exploration, simulation, or communication.heir specific informational needs and decision-making processes, providing a direct conceptual framework for building tools that coaches will find useful [15]. This human-in-the-loop approach is critical. Work on co-designing analytics systems for MOBA games, for instance, highlights the necessity of a multi-phased process to bridge the gap between expert domain knowledge and the capabilities of machine learning [83]. Other interactive systems, such as SNIL [84], are designed

not for expert analysis but for communication, using LLMs to automatically generate data-driven sports news articles. This body of work underscores the importance of designing systems with a clear understanding of the end-user's goal, whether it be exploration, simulation, or communication.

4. METHODS

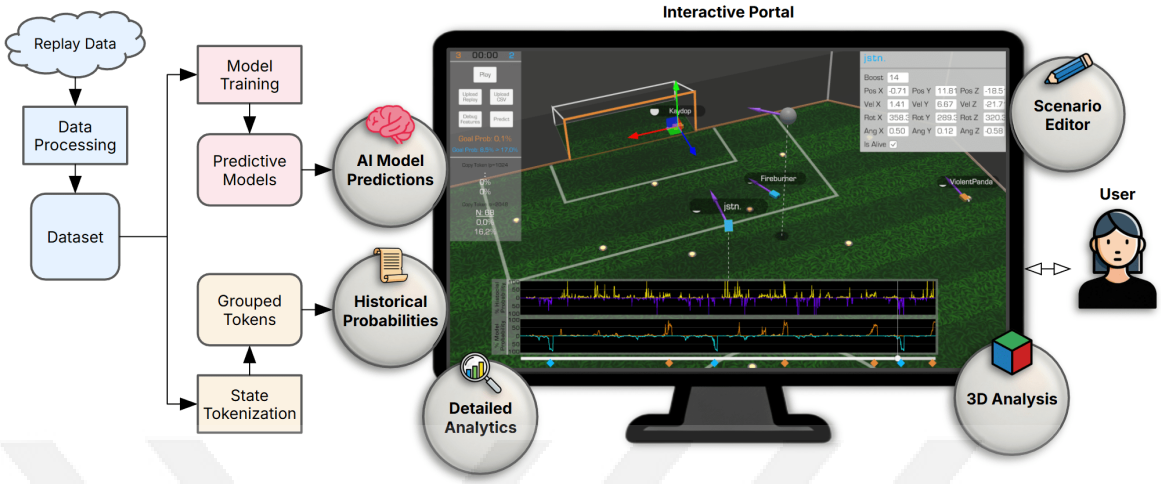


Figure 4.1. High-level system architecture of the peaRL framework.

The proposed framework establishes an end-to-end pipeline for Rocket League analytics that connects raw data and interactive insight. As illustrated in chapter 4.1, the workflow begins with Data Preprocessing, where raw replay files are parsed and transformed into structured datasets. This processed data feeds into two parallel analytical engines: State Tokenization, which retrieves historical probability baselines from a statistical ledger, and Model Architectures, which generate continuous threat predictions using machine learning. Finally, these predictive and statistical insights are synthesized in the User Interface Portal, a Unity-based application that enables interactive 3D visualization and counterfactual scenario analysis.

4.1. Data Preprocessing Pipeline

4.1.1. Data Sourcing and Collection

The data acquisition process for this research centered on ballchasing.com [5], a comprehensive public repository of Rocket League replay files. Replays were collected programmatically using the platform's public API. To ensure organized data management and reproducibility, we utilized the "Replay Group" feature, which allows for the

bulk download of curated tournament archives. We preserved the original hierarchical directory structure of the events with this approach.

Our methodology required the curation of three distinct datasets, each serving a specific analytical purpose:

(i) The Primary Model Dataset: This dataset, totaling 6,605 professional matches, serves as the complete corpus for the machine learning pipeline. It is the exclusive dataset used for training, validating, and testing all ML models. It is sourced from two major Rocket League Championship Series (RLCS) tournaments: the RLCS 2025 Split 1 (Birmingham Major) [85, 86] and the RLCS 2024 Major 1 (Copenhagen Major) [87, 88].

(ii) The Historical Augmentation Dataset: This dataset consists of an additional 3,759 professional matches, collected from the RLCS 2024 Major 2 (London Major) [89, 90] and the RLCS 2024 World Championship (Fort Worth) [91, 92]. To enhance the robustness of our statistical baseline and mitigate data sparsity issues (detailed in section 4.2), this dataset is combined with the Primary Model Dataset to create a more comprehensive historical database. This augmentation dataset is only used for populating the statistical ledger and is never exposed to the machine learning models.

(iii) The Ranked Generalization Dataset: To assess the generalizability of our professional-level models to a broader spectrum of skill levels, we curated a third dataset [93] from elo-based ranked matches (competitive playlist). This set contains 660 randomly selected ranked matches from the years 2024 and 2025, sampled evenly across all 22 in-game ranks (30 replays per rank, from Bronze I to Supersonic Legend, see subsection 2.2). To ensure data quality and exclude matches with anomalous outcomes (e.g., early forfeits), only replays that were played to their full duration were included in this set. This dataset serves as a final out-of-distribution test for the trained models.

4.1.2. Replay Parsing and Player Standardization

The foundation of the preprocessing pipeline involves converting the raw replay files from all three datasets into a structured tabular format. This is accomplished using the carball converter [34], a powerful open-source replay parsing library. The entirety of the data preprocessing pipeline can be seen in subsection 4.2.

For each of the replays, the pipeline programmatically executes the carball parser. This process extracts the high-frequency physics data (e.g., 30Hz-120Hz) for all entities and saves the output into a set of temporary Apache Parquet files:

- game.parquet: Contains global game state (e.g., seconds_remaining, is_overtime).
- ball.parquet: Contains the ball's full 3D physics vector.
- player_*.parquet: A separate file for each player that contains their individual physics data.
- metadata.json & analyzer.json: Contain high-level information, including player metadata, team assignments, goal event timestamps, and precise gameplay period start/end frames.

A critical standardization step is then applied to ensure data consistency across all matches. The pipeline validates that each match is a standard 3v3 game. It then programmatically sorts all six active players, first by team (Blue, then Orange) and subsequently by their unique player identifier. This process creates a consistent and permutation-invariant player ordering (e.g., p0, p1, p2 for Blue; p3, p4, p5 for Orange) that is maintained for the entire dataset.

4.1.3. State Assembly and Feature Engineering

Following the parsing and player standardization steps, the individual player, ball, and game data (from the Parquet and JSON files) are loaded and assembled into a single wide-format pandas DataFrame. Each row in this intermediate DataFrame

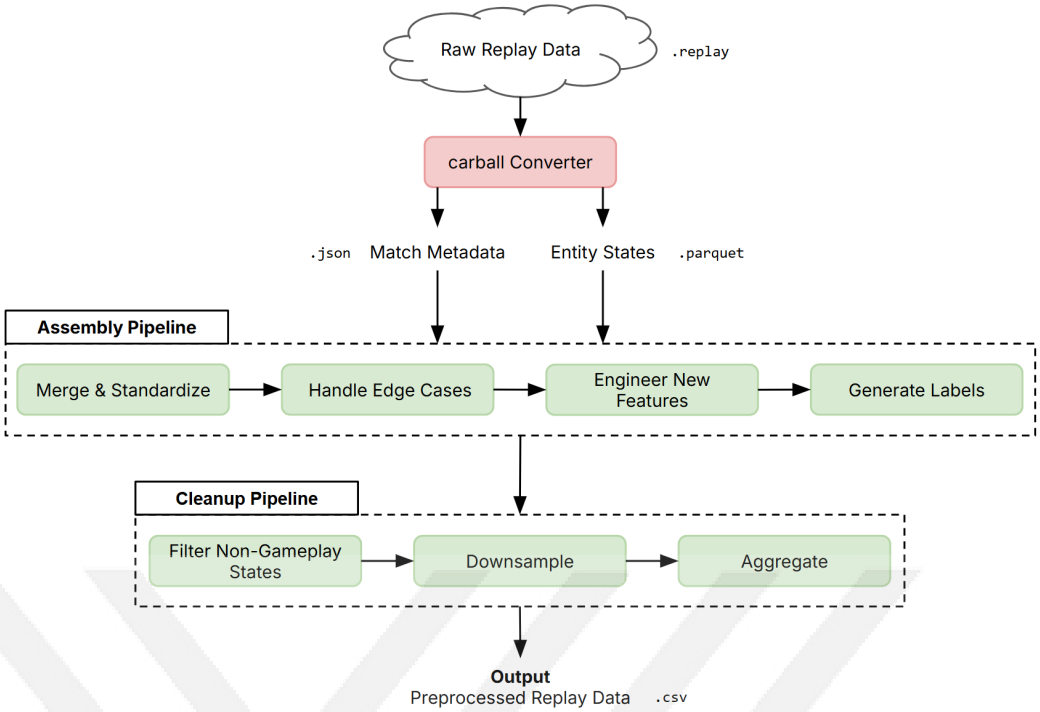


Figure 4.2. Data preprocessing pipeline from raw replay data to tabular output.

represents one high-frequency (e.g., 1/120th of a second) game state, and each column represents a specific feature.

To transform this raw data into a machine-learning-ready format, we implemented a pipeline of null-imputation and feature engineering.

4.1.3.1. Demolition-Aware Null Imputation. A significant challenge in processing Rocket League replay data is the "demolition" mechanic, which temporarily removes a player from the game field. This event results in null (NaN) values for the player's physics state in the raw logs, effectively breaking the fixed-size input vector required by neural network architectures. To address this, we designed a semantic imputation strategy that maintains the integrity of the 6-player state vector.

We first generated a binary alive flag for each player, marking them as dead (0) whenever their position data was null. For these demolished frames, the missing physics values were imputed with "inactive" constants designed to minimize model confusion.

Specifically, the position and velocity vectors were clamped to the origin ($[0, 0, 0]$), the boost amount was forced to zero, and the orientation was reset to the identity quaternion ($[1, 0, 0, 0]$). This approach ensures that a demolished player effectively ceases to exist in the model's spatial reasoning without altering the dimensionality of the input tensor.

4.1.3.2. Feature Engineering. Beyond raw physics, several high-level features were engineered to expose latent game state information. We also established specific conventions to handle edge cases:

- Unit Forward Vectors: The raw quaternion values $q = (w, x, y, z)$ were converted into global 3D unit forward vectors $\vec{f}_{fwd} = (f_x, f_y, f_z)$ to provide a normalized representation of orientation. The conversion is performed by

$$\vec{f}_{fwd} = \begin{bmatrix} 1 - 2(y^2 + z^2) \\ 2(xy + wz) \\ 2(xz - wy) \end{bmatrix}. \tag{4.1}$$

- Team Logic and Possession:
  - Team ID: We adhere to the convention where 0 represents Blue team and 1 represents Orange team.
  - Ball Hit Team Number: The ball_hit_team_num feature tracks the last team to touch the ball. While typically binary (0 or 1), we introduced a neutral value of 0.5 to represent the specific edge case at the start of a kickoff before any player has made contact.
- Stateful Boost Timer Reconstruction: The carball converter do not explicitly log "time until boost respawn." We reconstructed this by writing a stateful processor that iterates through the match chronologically. When a "Boost Pickup" event is detected for a large pad, a timer is initialized to 10.0 seconds and decayed by the frame delta time. This results in 6 continuous features (boost_pad_0_respawn, etc.) representing the exact availability of the field's most critical resources. Small pads were excluded due to their lower strategic impact.

- Score and Time Normalization: Using the goal metadata, we back-filled blue_score, orange_score, and goal_difference (orange_score - blue_score) for every frame between goals. In addition, we created overtime continuity; standard game clocks count down (300s to 0s), but overtime clocks count up from 0s. To create a monotonic time feature, we converted overtime clocks to negative values (e.g., 10 seconds into overtime becomes -10.0).

4.1.3.3. Dataset Schema and Final Feature Vector. The final dataset contains a superset of columns useful for both analysis and modeling.

Metadata and Auxiliary Columns. While not fed into the predictive models, the following columns are preserved for structural integrity and post-processing:

- replay_id: Unique 5-digit string identifier for the match (e.g., replay_00003, replay_09876).
- time: The time passed from the beginning of the replay file. While seconds_remaining value is an integer and only decreases when the gameplay is active, the time value is a float and passes in real-time for the entirety of the replay file.
- is_overtime: Boolean flag indicating if the match has exceeded regulation time.
- blue_score, orange_score, and goal_difference: Raw integer values showing goals scored.

The Model Input Vector. For the machine learning pipeline, we isolate a specific 92-dimensional feature vector. This vector describes the physical and strategic state of the game at any single instant.

To visualize the spatial context of these features, section 4.3 illustrates the field dimensions and coordinate system in Unreal Units, rounded up. The top-down view (left) and the view from the back of the orange net (right) can be seen. The origin

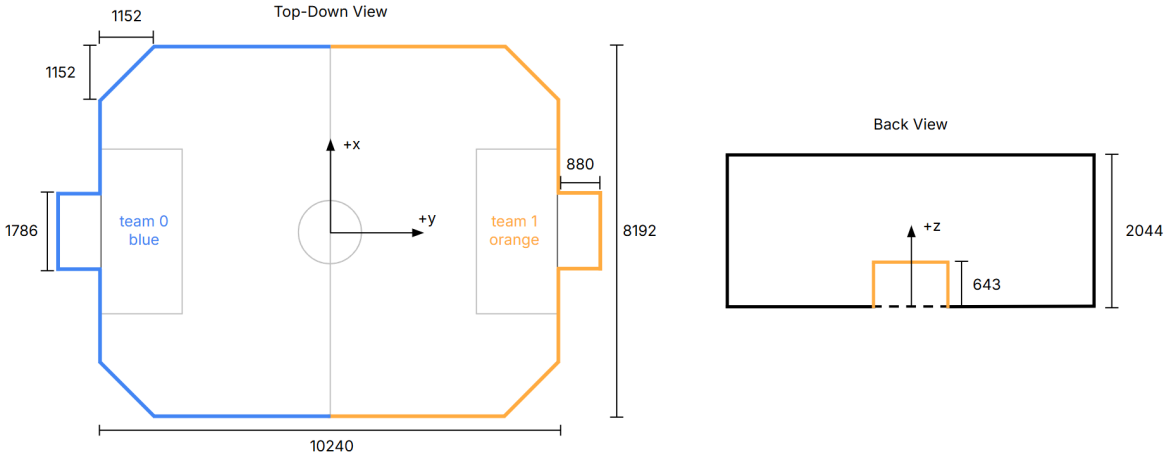


Figure 4.3. The Rocket League field coordinate system and dimensions.

$(0, 0, 0)$ is located at the center of the field, on the ground. All of the round curvatures inside the field are ignored for simplification. The global positions of the cars inside the field are based on their center of mass. All the tested variables of a standard Rocket League game, including dimensions and mechanics, are taken from the RLBot website [94].

section 4.1 provides a comprehensive summary of the input vector composition, detailing feature groups, boundaries, and dimensionality.

4.1.4. Gameplay Filtering and Ground Truth Labeling

A significant challenge in replay analysis is filtering out non-gameplay states. Our pipeline implements a definitive two-stage filtering process to isolate only active and meaningful gameplay. First, all non-gameplay frames, such as pre-match build-ups, pre-kickoff countdowns, and all post-goal replay sequences, are precisely excised using the authoritative gameplay_periods timestamps provided by carball's analyzer.json file. Later, a specialized filter is applied to handle the unique case of overtime kickoffs. The script identifies the start of the overtime countdown and then finds the exact "Go!" frame by detecting the first instance of player movement (a change in any player's x or y position). All static countdown frames before this "Go!" frame are removed.

Table 4.1. Summary of the 92-dimensional input feature vector.

Group	Category	Specific Features	Type	Boundaries	Count	Total Dims
Players	Physics	Position (x, y, z)	Float	$\pm 4096, \pm 5120, [0, 2044]$	6	18
		Velocity (v_x, v_y, v_z)	Float	± 2300 uu/s	6	18
		Unit Forward Vector	Float	$[-1.0, 1.0]$	6	18
	Status	Boost Amount	Float	$[0.0, 100.0]$	6	6
		Alive Status	Binary	$\{0, 1\}$ (1=Alive)	6	6
		Team Identifier	Binary	$\{0, 1\}$ (0=Blue)	6	6
	Relational	Distance to Ball	Float	$[0.0, 13271.9]$	6	6
Ball	Physics	Position (x, y, z)	Float	$\pm 4096, \pm 5120, [0, 2044]$	1	3
		Velocity (v_x, v_y, v_z)	Float	± 6000 uu/s	1	3
Global	Resource	Big Boost Pad Timers	Float	$[0.0, 10.0]$	6	6
	State	Ball Hit Team Number	Float	$\{0, 0.5, 1\}$	1	1
		Seconds Remaining	Integer	$(-\infty, 300]$	1	1
					Total	**92**

Immediately following this cleaning step, ground truth labels are generated. We frame the task as a binary classification problem: predicting a goal within a short time window. Using the metadata.json goal timestamps, all active-play frames within a 5-second anticipation window (i.e., the 5 seconds immediately preceding a goal) are marked as a positive event (team_*_goal_in_event_window = 1). All other active-play frames are considered negative events (label = 0). Note that since each gameplay period is treated separately, there is no way for both teams to have a positive label at the same data point.

4.1.5. Downsampling and Dataset Aggregation

The resulting DataFrame, now cleaned and labeled, still exists at its original high frequency (up to 120Hz). To create a dataset suitable for modeling, a final downsampling step is applied.



Figure 4.4. One second of gameplay with a 5Hz sampling rate, showing 6 consecutive states.

The data is resampled to a consistent 5Hz (one frame every 0.2 seconds). This sampling is applied to both positive (goal-imminent) and negative (non-event) states to preserve the temporal characteristics of the data while significantly reducing redundancy. A visualization of 1 second of gameplay with a 5Hz sampling can be seen in subsection 4.4. Here, as an example, the Orange team is clearing the ball from their half, while the Blue team is rotating back. The player "Atomic" drives ($t = [0.0, 0.4]$), jumps ($t = [0.4, 0.6]$), and flies ($t = [0.6, 1.0]$) towards the ball in the air, moving away from the orange net.

This downsampling process resulted in a final dataset of approximately 10 million labeled game states for the Primary Model Dataset, 5.8 million for the Historical Augmentation Dataset, and 1 million for the Ranked Generalization Dataset (with around 48,000 for each rank). Finally, the pipeline saves a CSV file for each processed replay and then aggregates all individual files.

4.1.6. Dataset Characteristics and Splitting

The final preprocessing pipeline yielded a massive, high-dimensional dataset. We provide a detailed statistical breakdown of the data integrity, volume, and splitting

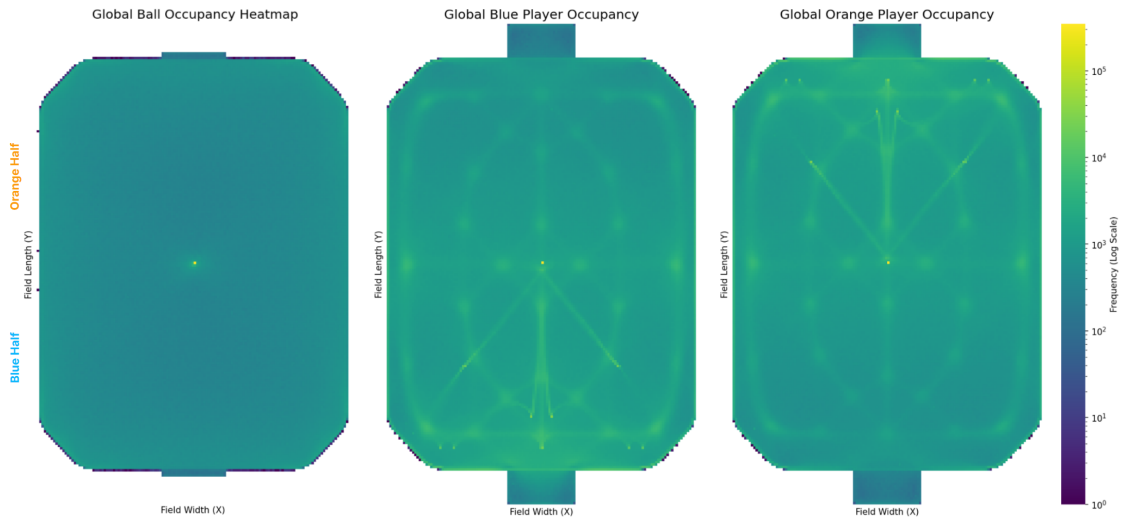


Figure 4.5. Ball, blue team, and orange team heatmaps made out of all data points from the primary model dataset, with a resolution of 200 cells along the longer axis.

strategy to ensure reproducibility.

4.1.6.1. Data Integrity and Volume. The parsing pipeline demonstrated high robustness. For the Primary Model Dataset, 6,299 of 6,489 replays were successfully processed (97.1% success rate), resulting in 9,957,815 labeled states. The Historical Augmentation Dataset saw similar success (3,706 of 3,750 processed), adding 5,888,253 states for statistical ledger calculations.

Calculating based on total frame counts at 5Hz, the average overall professional match length was approximately 5 minutes and 17 seconds, consistent with standard regulation play of 5 minutes plus occasional overtimes. The average goals per match was approximately 4.1. The heatmaps for the ball, blue team players, and orange team players can be seen in subsubsection 4.5, where the repeating kickoff positions and paths across the boost pads can be distinctly observable.

4.1.6.2. Primary Dataset Class Imbalance. Goal events are naturally rare. Across the entire Primary Dataset, the ratio of non-scoring frames to goal-imminent frames is 14.4 to 1. When analyzed by team, a slight asymmetry is observed, where Blue Team

Imbalance is 28.2 to 1, and Orange Team Imbalance is 31.6 to 1. This indicates that, in this specific sample of professional play, the Blue team scored slightly more frequently than the Orange team.

4.1.6.3. Train-Validation-Test Split. To prevent data leakage, the Primary Model Dataset was partitioned by replay file using a random 70/15/15 split. subsubsection 4.2 confirms that the statistical distribution (imbalance and volume) remained consistent across all splits, which validates the randomization strategy.

Table 4.2. Statistical breakdown of the primary model dataset splits.

Metric	**Train**	**Validation**	**Test**	**Total**
Replays	4,409 (70%)	944 (15%)	946 (15%)	**6,299**
Total Rows	6,952,674	1,500,979	1,504,162	**9,957,815**
Blue Imbalance Ratio	28.16 : 1	29.56 : 1	27.48 : 1	28.16 : 1
Orange Imbalance Ratio	31.67 : 1	32.00 : 1	31.10 : 1	31.63 : 1

4.1.6.4. Ranked Generalization Dataset. The Ranked Generalization Dataset processed 30 replays for each of the 22 competitive ranks (Bronze I through SSL) without any failures, which totals 1,011,022 frames.

4.2. State Tokenization and Historical Probabilities

While machine learning models operate on continuous vectors, this research establishes a parallel statistical framework to determine the "ground truth" historical probability of specific game states. This section details the discretization logic (Tokenization) used to convert continuous physics into finite buckets and the construction of the "Statistical Ledger."

4.2.1. State Tokenization Rules

The primary challenge in analyzing Rocket League states statistically is the infinite nature of the continuous feature space. To create a queryable database, we must reduce this dimensionality through discretization and permutation invariance. While many features could have been used for creating a token (e.g., position, velocity, boost amount for players), each one's inclusion exacerbates an already prevalent data sparsity problem, where many unique singled-out tokens are found inside the dataset. These low-instance tokens fumble in practice at the task of historical prediction by aggregation of similar tokens, because there is little to no similar examples to base the prediction on. For this reason, we focus only on tokenization based on positional information.

4.2.1.1. Spatial Discretization (The Grid). While the continuous 3D field coordinates (x, y, z) enable a cubical fragmentation, the Z-axis (height) is omitted to reduce the combinatorial explosion of states, so the positions are projected onto a discrete 2D grid. We defined fixed parcel sizes of 1024 and 2048 unreal units. The discretization floor function $\Phi(x, y, z)$ maps continuous coordinates of the objects to discrete parcel indices (p_x, p_y, p_z) by

$$\Phi(x, y, z) = \begin{cases} p_x = 2 \left\lfloor \frac{x}{\Delta s} \right\rfloor + 1 \\ p_y = 2 \left\lfloor \frac{y}{\Delta s} \right\rfloor + 1 \\ p_z = 2 \left\lfloor \frac{z}{\Delta s} \right\rfloor + 1 \end{cases} \tag{4.2}$$

where x, y, and z defines the position of the objects, and Δs defines the parcel size. We omit the z axis for this formulation. This transforms the standard map (8192×10240 units) into 24 parcels (buckets) for a parcel size of 2048, and 84 parcels (buckets) for a parcel size of 1024.

4.2.1.2. Permutation Invariance. In a standard feature vector, switching Player 1 and Player 2 results in a different input, even if they are in identical locations. For a

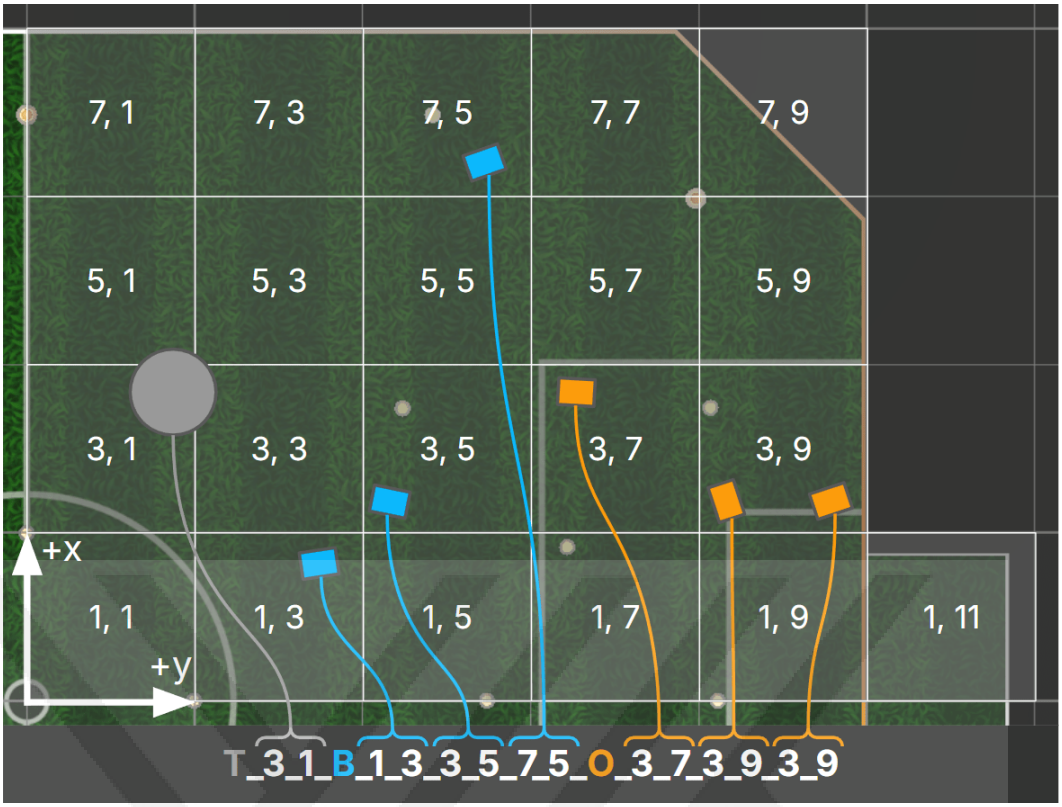


Figure 4.6. Tokenization logic with a parcel size of 1024.

statistical lookup table, this redundancy is fatal. We enforce permutation invariance to ensure that the configuration of the team matters, not the identity of the specific agents. To achieve this, we decouple the agent's identity from their spatial data. Instead of recording the team state as an ordered tuple $\mathcal{C} = (p_1, p_2, p_3)$ where index 1 represents a specific player ID, we treat the team state as an unordered multi-set of occupied positions $\mathcal{C} = \{p_1, p_2, p_3\}$.

In practice, we enforce this property by alphanumerically sorting the discretized parcel coordinates of the players within each team before token generation. This ensures that if two players occupy positions (x_1, y_1) and (x_2, y_2), swapping these positions yields an identical token representation. The tokenization logic can be seen in subsubsection 4.6. Here, the indices of the parcels are related to their center in the coordinate system. The output token has the form: T_[Ball Position]_B_[Sorted Blue Positions]_O_[Sorted Orange Positions].

Despite these reductions, the state space remains vast. The theoretical upper bound for the number of unique states can be calculated by multiplying the ways to place the ball, blue team players, and orange team players. The player positions for each team can be represented as a "Stars and Bars" problem: How many ways are there to put n indistinguishable objects into k distinguishable bins? Which has the solution of

$$N_{team} = \binom{n+k-1}{k-1}. \tag{4.3}$$

So, the theoretical upper bound for the number of unique state tokens can be calculated with the formula of

$$|S| = N_{parcels} \times \left(\binom{3+N_{parcels}-1}{N_{parcels}-1}\right)^2. \tag{4.4}$$

For a parcel size of 1024 ($N_{parcels} = 84$), this yields over 8.7×10^{11} possible states, far exceeding any feasible dataset size. For a parcel size of 2048 ($N_{parcels} = 24$), the space collapses to roughly 5.8×10^9 states, which makes it statistically more approachable while trading off precision.

4.2.2. Canonicalization and Symmetry Validation

To further mitigate data sparsity, we implemented a canonicalization logic that maps symmetrical game states to a single representative token. This assumes that the game physics and player behaviors are symmetrical across the field's axes.

We define four possible transformations for any given state:

(i) Original: The raw state coordinates.

(ii) X-Mirror (Side-to-Side): Inverting the X-axis ($x \to -x$). This represents mirroring the play from left to right.

(iii) XY-Mirror (Attack-to-Defend): Inverting both axes. This effectively swaps the attacking and defending roles, necessitating a swap of the team labels (Blue $\leftrightarrow$

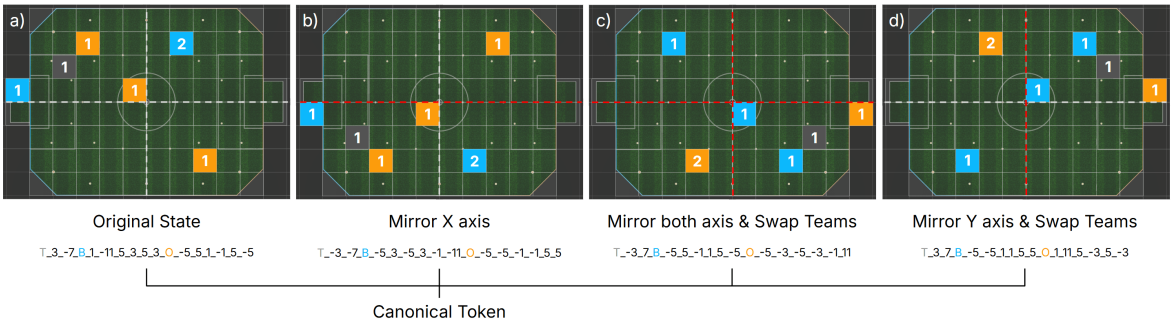


Figure 4.7. Token mirroring and canonicalization logic on parcel size of 1024.

Orange).

(iv) Y-Mirror (Full Rotation): Inverting the Y-axis ($y \to -y$), which means that both modifications are applied. This also necessitates a team swap.

This assumes that the players are not favoring one side (left or right) or one team color (blue or orange). For unifying the four tokens, one canonical token is chosen by lexicographic sorting item 4.7. Here, the gray box represents the ball, while the orange and blue boxes represent the players from the respective teams. The numbers inside the boxes represent the number of objects inside the parcel.

To validate this assumption, we analyzed the distribution of transformations selected across the combination of Primary Model and Historical Augmentation datasets. As shown in item 4.3, the data is distributed nearly evenly ($\approx 25\%$) across all four variations. This empirically confirms that Rocket League gameplay is spatially symmetrical, and our canonicalization logic effectively quadruples the data density without information loss.

Table 4.3. Distribution of canonical transformations for validation of symmetry.

Transformation	**Mirroring Logic**	**Distribution**
Original	No Change	23.95%
X-Mirror	$x \to -x$	24.07%
XY-Mirror	$x, y \to -x, -y$ (Swap Teams)	27.59%
Y-Mirror	$y \to -y$ (Swap Teams)	24.39%

The slightly higher percentage for XY-Mirror is attributed to a high number of kickoff scenarios (visible in subsubsection 4.5), which are perfectly symmetrical and canonically map to this rotation.

4.2.2.1. The Resolution Trade-off with Sparsity. We generated four distinct versions of the Statistical Ledger to analyze the impact of Parcel Size (Grid Resolution) and Data Volume (Original vs. Augmented Dataset). Here, Original Dataset refers to the Primary Model Dataset, whereas Augmented Dataset refers to the combination of Primary Model Dataset and the Historical Augmentation Dataset.

We analyzed the "Goal-Rich" tokens (states where at least one goal has occurred historically) versus "Zero-Goal" tokens. Table 4.4 reveals the insight that even with the addition of 5.8 million extra frames (Augmented Dataset), the state space at high resolution (1024) remains dominated by singled out tokens. Here, "Goal-State Singularity" refers to the percentage of goal-scoring tokens that appear only one time in the entire history. At a parcel size of 1024, over 88% of goal states are effectively unique.

Table 4.4. Sparsity profile of the tokenized state space for primary model dataset (original) and addition of historical augmentation dataset (augmented).

Configuration	**Parcel Size 1024**		**Parcel Size 2048**	
Dataset Scale	**Original**	**Augmented**	**Original**	**Augmented**
Total Unique Tokens	7,383,397	12,329,877	1,432,066	2,154,137
Goal-Rich Tokens	6.9%	7.0%	16.1%	18.7%
Zero-Goal Tokens	93.1%	93.0%	83.9%	81.3%
Avg Instances per Token	1.14	1.16	4.85	6.65
Goal-State Singularity	89.65%	88.03%	25.53%	21.45%

4.2.3. Predictive Performance on Test Set

To evaluate these ledgers as predictive models, we applied them to the held-out Test Set. We devised a "Neighbor Spreading" inference logic to handle unseen states:

(i) Exact Match: If the test token exists in the ledger, return its probability.

(ii) Neighbor Spread: If missing, search the 8 adjacent spatial parcels surrounding the ball. Average the probabilities of any found neighbors.

(iii) Unique Fallback: If no neighbors exist, return 0 (assuming unknown states are safe).

The results in item 4.5 demonstrate the limitations of this approach. Here, "Coverage by Spread" refers to test samples where the exact state was missing, but a prediction was derived by the neighbor spreading logic. While increasing the parcel size to 2048 significantly improved coverage (reducing unique states from $\sim 94\%$ to $\sim 22\%$), the predictive quality (AUPRC) remained low.

Table 4.5. Performance of the historical baseline on the test set with statistical ledgers using original and augmented datasets.

Configuration	**Parcel Size 1024**		**Parcel Size 2048**	
Metric	**Original**	**Augmented**	**Original**	**Augmented**
Exact Token Coverage	3.68%	4.23%	47.08%	**58.17%**
Coverage by Spread	1.50%	1.93%	**20.34%**	19.29%
Unknown Tokens	94.82%	93.84%	32.58%	**22.54%**
Weighted Loss (Blue)	15.15	15.02	7.61	**6.27**
Weighted Loss (Orange)	15.25	15.09	7.85	**6.43**
AUPRC (Blue)	0.0369	0.0375	0.0620	**0.0706**
AUPRC (Orange)	0.0323	0.0331	0.0553	**0.0634**

The experiment of adding more data to overcome sparsity (Augmentation) demonstrates that simply adding data does not solve the sparsity problem; increasing the dataset by around 58% only reduced the unknown state rate by 1% for 1024 parcel size. Although increasing the parcel size to 2048 theoretically decreases precision, its improvement against token sparsity enhances the overall prediction performance. Still, the low AUPRC for the Original Dataset (0.06) compared to machine learning benchmarks suggests that Rocket League states are too complex for frequency-based memorization. Note that further coarsening the grid (e.g., to a parcel size of 4096)

creates a bin that covers almost a quarter of the field. While this might achieve 100% coverage against token sparsity, the loss of spatial resolution would conflate distinct game states (e.g., merging "corner play" with "goal-line play"), which would destroy the predictive signal.

4.3. Model Architectures

4.3.1. Experimental Framework

4.3.1.1. Input Representation and Normalization. Neural networks are highly sensitive to the scale of input features. Raw Rocket League data contains features with vastly different magnitudes: positions can range up to ±6000 unreal units, while team flags are binary $(0, 1)$. To ensure stable gradient descent, we established a universal normalization protocol for all deep learning models (MLP, GNN, LSTM).

We apply Min-Max Scaling to project positional and kinematic features into the range $[0, 1]$, with the equation

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min} + \epsilon} \tag{4.5}$$

where $\epsilon = 1e^{-8}$ ensures numerical stability.

- Position: Normalized using the field bounds ($x \in [-4096, 4096]$, $y \in [-6000, 6000]$, $z \in [0, 2044]$).
- Velocity: Normalized based on the maximum possible hard-coded entity speeds (2300 uu/s for the players and 6000 uu/s for the ball).
- Boost: Normalized from $[0, 100]$ to $[0, 1]$.
- Respawn Time for Boost Pads: Normalized from $[0, 10]$ to $[0, 1]$.
- Players' Euclidean Distance to Ball: Normalized from $[0, 13271.9]$. The upper bound is the distance between the two opposite corners of the field, where two entities can be farthest away from each other.

Directional unit vectors (forward alignment) were preserved in their native range $[-1, 1]$ to maintain geometric linearity. Time is normalized based on the regulation length (300s), which allows overtime values to extend naturally into the negative range to provide a distinct temporal context. This normalized output extends into the negative range. This process yields a standardized feature space of $\mathbb{R}^{92}$, which serves as the foundational input for all subsequent architectures.

4.3.1.2. Handling Imbalance: Weighted Loss. Given the extreme class imbalance ($\approx 1 : 30$), a standard Binary Cross-Entropy (BCE) loss tends to converge toward a trivial solution that predominantly predicts "No Goal." To counteract this, we utilize a Weighted BCE Loss.

We calculate a specific positive weight coefficient w_{pos} for each team's prediction head based on the inverse prevalence of goals in the training set by

$$w_{pos} = \frac{N_{neg}}{N_{pos}}. \tag{4.6}$$

Specifically, we computed $w_{pos} = 28.16$ for the Blue head and $w_{pos} = 31.67$ for the Orange head. The loss value $\mathcal{L}$ for each head is then calculated as

$$\mathcal{L} = -\frac{1}{N_b} \sum_{i=1}^{N_b} [w_{pos} \cdot y_i \log(\sigma(\hat{y}_i)) + (1 - y_i) \log(1 - \sigma(\hat{y}_i))] \tag{4.7}$$

where N_b is the total number of samples in the batch, y_i is the ground truth binary label for sample i (1 for Goal, 0 for No Goal), $\hat{y}_i$ is the raw logit output from the model for sample i, $\sigma(\cdot)$ is the sigmoid function that maps the logit to a probability $[0, 1]$, and w_{pos} is the scalar weight applied exclusively to positive instances (Goal events). This formulation follows PyTorch's convention, where the weighted loss is normalized by the batch size rather than by the sum of sample weights.

This term penalizes false negatives approximately 30 times more than false positives, which ensures that the gradient updates are driven by the rare and high-value events rather than the majority class.

4.3.2. Primitive and Shallow Baselines

Before implementing deep learning architectures, we established a hierarchy of primitive and shallow baselines. These models serve two critical purposes: they verify the integrity of the evaluation pipeline (sanity checks) and quantify the predictive power of raw game physics when decoupled from complex spatial reasoning.

4.3.2.1. Trivial Heuristics Definitions. We evaluated seven trivial strategies that possess no knowledge of the specific game state. Their predictions are based solely on deterministic rules or the class distribution of the training set.

(i) Always No Goal: Predicts $p = 0.0$ for every frame.

(ii) Always Goal: Predicts $p = 1.0$ for every frame.

(iii) Constant Prior: Predicts the natural goal probability observed in the training set ($p = 0.031$) for all frames.

(iv) Balanced Constant: Predicts $p = 0.5$ for all frames. This represents the theoretical optimum for a loss function where the positive class weight perfectly balances the negative class frequency.

(v) Uniform Random: Predicts a random continuous float $p \sim U[0, 1]$.

(vi) Gaussian Random: Predicts a random float sampled from a normal distribution centered at the optimal constant: $p \sim \text{Gaussian}(\mu = 0.5, \sigma = 0.15)$.

(vii) Stratified Random: Predicts a binary outcome $\{0, 1\}$ by flipping a weighted coin (Bernoulli distribution) where $P(1) = 0.031$, where the probability is set to the goal frequency observed in the training set. This represents a model that guesses the correct frequency of goals but has no discriminative ability.

These zero-learning models establish the absolute floor for performance metrics.

4.3.3. Feature-Driven Baseline: Logistic Regression

To establish a baseline for "simple" learning, we trained two Logistic Regression variants. These models serve to quantify the predictive ceiling of high-impact kinematic features, specifically longitudinal (y) position, velocity, and possession, when decoupled from complex graph structures.

Each model maps a feature vector $\mathbf{x}$ to a probability $\hat{y}$ via the sigmoid activation function

$$\hat{y} = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{4.8}$$

is the learnable weight vector, b is the scalar bias term, and T denotes the vector transpose.

Univariate Polynomial Model. We first isolate the ball's longitudinal position (y) to test the hypothesis that simple proximity is the primary driver of threat. To account for non-linear "danger zones" (e.g., the goal line versus the midfield), we apply a 3rd-degree polynomial expansion. The resulting logit z is computed by

$$z = w_1 y_{\text{ball}} + w_2 y_{\text{ball}}^2 + w_3 y_{\text{ball}}^3 + b. \tag{4.9}$$

Here, y_{ball} denotes the ball's position along the y axis, with w_1 to w_3 representing the coefficients, and with b as bias.

Multivariate Linear Model. The second variant expands the feature space to include momentum and possession, though it maintains a strictly linear relationship. The model combines three key kinematic signals: position (y_{ball}), longitudinal velocity ($v_{\text{ball_y}}$), and the last-touch team identifier (h_{team}) by

$$z = w_1 y_{\text{ball}} + w_2 v_{\text{ball_y}} + w_3 h_{\text{team}} + b. \tag{4.10}$$

In this formulation, $w_1, w_2,$ and w_3 correspond to the learned weights, and b is the bias. This baseline effectively tests the predictive power of "who has the ball and how fast it is moving," ignoring all other player-specific spatio-temporal context.

4.3.4. Gradient Boosted Decision Trees: XGBoost

Unlike deep learning models, which optimize a loss surface via gradient descent, XGBoost constructs an ensemble of weak learners (decision trees) sequentially. Each new tree is trained to predict the residual errors of the prior ensemble.

4.3.4.1. Data Processing and Architecture. In contrast to the MLP and GNN architectures, the XGBoost models were trained on unscaled features. Decision trees rely on splitting criteria (e.g., $x > 3000$) which are invariant to monotonic transformations; thus, Min-Max normalization provides no theoretical benefit while adding computational overhead.

For the XGBoost model, we adopted a Dual-Model Architecture rather than a Multi-Task Learning approach. Two independent classifiers were trained, where each team's classifier was trained exclusively to minimize the log-loss of their respective scoring labels. This decoupling allows each model to specialize in the specific offensive patterns of a single team without interference from the opposing team's loss gradient.

4.3.4.2. Training Configuration. The models were trained using the hist tree construction method, which bins continuous features into discrete histograms to accelerate training on large-scale datasets. To prevent overfitting and ensure robustness, we enforced a maximum depth constraint on individual trees, balancing model capacity with generalization to avoid memorizing specific replay noise. We further introduced variance and reduced correlation between ensemble members via stochastic subsampling to utilize both random row-wise data selection and column-wise feature subsets. These regularization techniques were coupled with a conservative learning rate and a high

maximum estimator budget to ensure stable convergence. Consistent with our general methodology, we addressed class imbalance by dynamically scaling the positive class weight to the inverse class frequency, while the final inference threshold was optimized on the Validation Set to maximize the F1-Score.

4.3.5. Multi-Layer Perceptron (MLP)

The Multi-Layer Perceptron (MLP) serves as our primary deep learning baseline. Unlike the primitive heuristics, the MLP has the capacity to learn non-linear interactions between features (e.g., the relationship between a player's boost amount and their velocity). However, unlike GNNs, the MLP treats the game state as a flat, unstructured vector, devoid of explicit spatial or relational information.

4.3.5.1. Data Processing and Architecture. In contrast to the tree-based models, neural networks are highly sensitive to the scale of input features. Consequently, all continuous inputs (position, velocity, timers) were normalized using Min-Max Scaling before being fed into the network.

Structurally, we adopted a Multi-Task Learning (MTL) architecture. Unlike the XGBoost approach, which required training two separate models, the MLP utilizes a shared encoder body to extract a high-level representation of the game physics. This latent representation then branches into two team-specific heads (Blue and Orange), which allows the model to learn a unified understanding of the match dynamics while optimizing for team-specific scoring probabilities simultaneously.

The network architecture is composed of a shared encoder body that bifurcates into two independent prediction heads. The input vector (92 features) is first projected into a high-dimensional space ($92 \rightarrow 256$) before being compressed into a compact latent bottleneck ($256 \rightarrow 128$). Each linear transformation within this shared body is followed by Batch Normalization, a ReLU activation, and a Dropout layer (rate 0.5) to enforce regularization. Finally, the resulting 128-dimensional latent vector

is fed into two parallel linear layers ($128 \rightarrow 1$), which generate the raw logits for the blue and orange teams, respectively. This architecture forces the shared body to learn generalized representations of scoring probability, such as the risk inherent in an unprotected goal, rather than overfitting to specific team identities.

4.3.6. Spatially-Aware Graph Neural Networks (GNNs)

The fundamental limitation of the MLP and XGBoost baselines is their treatment of the game state as a flattened vector. In reality, Rocket League is a geometric system defined by the relationships between entities. A player's coordinates are meaningless in isolation; their strategic value is derived entirely from their distance to the ball, their proximity to teammates, and their alignment with the goal.

To explicitly encode this geometric prior, we model the game state as a graph $G = (\mathcal{V}, \mathcal{E})$. We iteratively developed two distinct GNN architectures to test the importance of graph topology and message-passing mechanisms: the Graph Convolutional Network (GCN) and the Graph Attention Network (GAT).

4.3.6.1. Graph Construction and Topology. We formulate the graph topology in two ways. One where we treat only the 6 players as nodes, and one where we add the ball as a 7th node within the graph.

Node Features ($\mathcal{V}$). The set of nodes $\mathcal{V}$ consists of 6 Player Nodes (P_{0-5}) and, in the unified configuration, 1 Ball Node (B).

- Player Nodes: Each player node contains the 12-dimensional player-specific feature vector (Position, Velocity, Forward Vector, Boost, Alive Status, Team ID), seen in section 4.1.
- Ball Node: The ball node contains its 6-dimensional physics vector. To align feature dimensions with player nodes (allowing for shared weight matrices), we

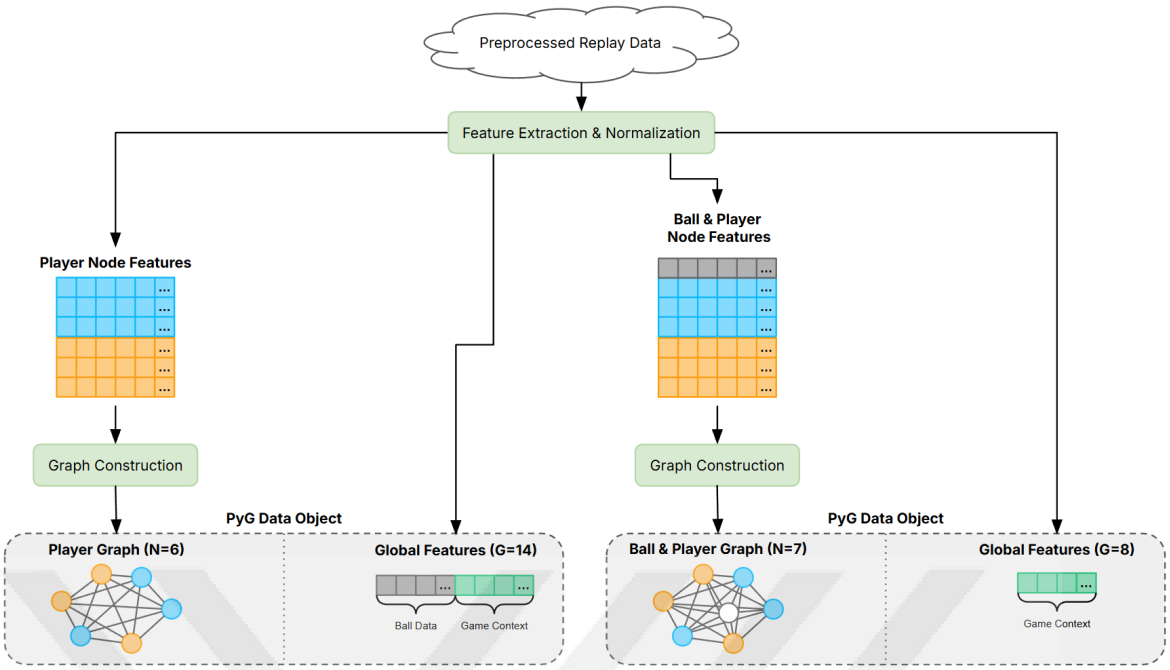


Figure 4.8. The structure of the 6-node (left) and the 7-node (right) graph structure and the respective PyG data objects to be used in GNN model trainings.

zero-pad the ball's feature vector to 12 dimensions. Specifically, the ball is assigned a neutral team ID (2) and a fixed "alive" status.

Edge Connectivity ($\mathcal{E}$). We utilize a fully connected directed graph where every entity is connected to every other entity (explicit self-loops are omitted in the dataset, as GNN layers typically inject them automatically). This ensures that information can propagate from any agent to any other agent (e.g., a defender to a striker) in a single message-passing layer. With this, there are 30 edges for the 6-node graph and 42 edges for the 7-node graph. The graph structures can be seen in section 4.8.

Unified Graph Architecture. The unified graph architecture for training the models can be seen in subsubsection 4.9. The framework is configurable for both 6-node (Player-Centric) and 7-node (Player+Ball) topologies. The Graph Encoder toggles between isotropic Graph Convolutional Network (GCN) layers and anisotropic Graph Attention Network (GAT) layers, with the latter utilizing explicit edge attributes

($d_E \in [1, 4]$) for attention computation. Note the conservation of information: in 6-node variants, ball physics are preserved by shifting them into the Global Feature vector ($G = 14$) for Late Fusion, whereas in 7-node variants, they are processed geometrically ($G = 8$).

4.3.6.2. Graph Convolutional Networks (GCN). The GCN served as our initial attempt to introduce relational inductive biases. It updates node representations by aggregating features from neighbors using a fixed, isotropic averaging operation of

$$\mathbf{h}'_i = \mathbf{W} \sum_{j \in \mathcal{N}(i)} \frac{1}{\sqrt{\deg(i)\deg(j)}} \mathbf{h}_j \tag{4.11}$$

where $\mathcal{N}(i)$ represents the neighbors of node i, (including a self-loop added by the GCN layer), $deg(i)$ represents the number of neighbors for the node i, $\mathbf{h}_j$ represents the input feature vector of a neighbor node j that sends a message to the node i during the GCN update ($\mathbf{h}_j \in \mathbb{R}^{12}$), $\mathbf{h}'_i$ represents the updated latent representation of node i after the GCN layer operation ($\mathbf{h}'_i \in \mathbb{R}^{hidden_dim}$), and $\mathbf{W}$ represents a learnable weight matrix applied to the aggregated neighbor features, which is the core trainable parameter of the GCN layer ($\mathbf{W} \in \mathbb{R}^{hidden_dim \times 12}$).

We evaluated GCNs on both the 6-Node (Player-Centric) and 7-Node (Unified) topologies defined in section 4.8. In the 6-node variant, the ball's physics (position and velocity) were treated as a global context vector concatenated after graph pooling, which tests whether explicit message passing between the ball and players is necessary for accurate prediction.

4.3.6.3. Graph Attention Networks (GAT) with Relational Edge Features. In order to overcome the isotropic limitation of GCNs, we implemented Graph Attention Networks. This architecture introduces a learnable attention mechanism, which allows specific nodes (e.g., the player closest to the ball) to dominate the message-passing signal while suppressing irrelevant nodes.

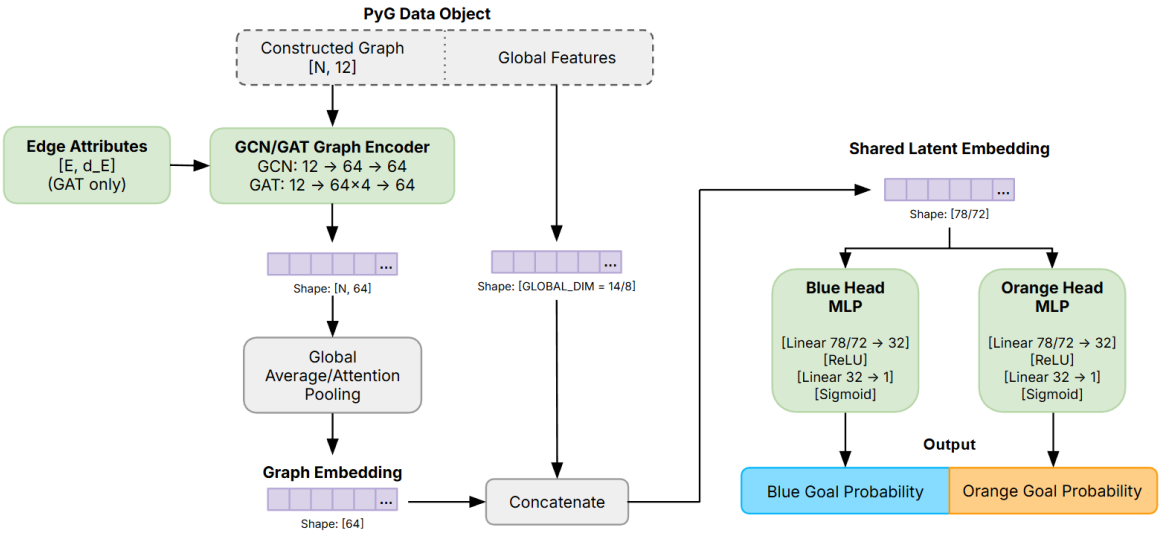


Figure 4.9. The unified GNN architecture used for ablation studies.

Edge Feature Engineering. Our implementation explicitly injects rich geometric edge features into the attention mechanism. For every edge e_{ij} connecting source node j to target node i, we construct a vector $\mathbf{e}_{ij} \in \mathbb{R}^4$ containing the following attributes to be used in an ablation study:

(i) Inverse Distance: A soft proximity metric that decays as entities move apart

$$e_{dist} = \frac{1}{1 + (d_{ij}/1500)^2} \tag{4.12}$$

where d_{ij} represents the distance between the two entities.

(ii) Team Relationship: A binary indicator encoding the adversarial nature of the game

$$e_{\text{team}} = \mathbb{I}(\text{team}_i = \text{team}_j). \tag{4.13}$$

(iii) Velocity Alignment (Weighted): Measures if node i is moving towards node j, weighted by the speed of node i. This ensures that fast-moving entities exert stronger attentional influence than stationary ones. The velocity alignment is

calculated by

$$e_{\text{vel}} = \left(\frac{\mathbf{v}_i \cdot \mathbf{p}_{ij}}{\|\mathbf{v}_i\| \|\mathbf{p}_{ij}\|} \right) \cdot \frac{\|\mathbf{v}_i\|}{v_{\max}} \tag{4.14}$$

where $\mathbf{v}_i$ is the velocity vector of node i, $\mathbf{p}_{ij}$ is the relative position vector $(\mathbf{x}_j - \mathbf{x}_i)$, $\| \cdot \|$ denotes the Euclidean norm, and $v_{\max}$ is the maximum player speed (2300 uu/s) used for normalization.

(iv) Forward Vector Alignment: Measures if node i is physically facing node j. Computed as the cosine similarity between the forward vector and the relative position vector, with the equation of

$$e_{\text{fwd}} = \frac{\mathbf{f}_i \cdot \mathbf{p}_{ij}}{\|\mathbf{f}_i\| \|\mathbf{p}_{ij}\|} \tag{4.15}$$

where $\mathbf{f}_i$ is the unit forward direction vector of node i and $\mathbf{p}_{ij}$ is the relative position vector to the target node j.

Architecture Evolution and Pooling Strategies. We isolated the impact of pooling mechanisms by comparing three GAT variants:

- Model A (6-Node GAT + Attention Pooling): Utilized the player-centric graph with a gating network to select the most relevant player.
- Model B (7-Node GAT + Mean Pooling): Utilized the unified topology but averaged the final embeddings.
- Model C (7-Node GAT + Attention Pooling): This model combines the 7-node topology with Global Attention Pooling with

$$\mathbf{h}_G = \sum_{i \in \mathcal{V}} \text{softmax}(f_{gate}(\mathbf{h}_i))(\mathbf{h}_i) \tag{4.16}$$

where $\mathbf{h}_G$ represents the final graph-level representation vector aggregated from the entire set of nodes ($\mathbf{h}_G \in \mathbb{R}^{64}$), $\mathcal{V}$ denotes the set of all nodes in the current graph instance, and $\mathbf{h}_i$ represents the latent feature vector of node i. The function f_{gate} is a learnable linear transformation ($f_{gate} : \mathbb{R}^{64} \to \mathbb{R}^1$) that projects each

node embedding to a scalar importance score. The softmax operation normalizes these scores across all nodes to produce a scalar attention coefficient α_i, which scales the contribution of each node vector $\mathbf{h}_i$ before the summation.

The final combination allows the network to perform relational reasoning between the ball and players (via the 7-node edges) and then dynamically select the most critical entity to drive the final prediction (via Attention Pooling).

Global Context Integration. In all GNN variants, the final pooled graph embedding $\mathbf{h}_G$ is concatenated with the vector of global features (Boost Timers, Game Clock, Ball Hit Team) before passing through the linear prediction heads.

Prediction Heads. After pooling, the graph embedding is concatenated with the global context vector and passed through two parallel MLP heads, one for each team. Each head consists of two linear layers with ReLU activation and outputs a single probability via a sigmoid function

$$p_{blue} = \sigma(f_{blue}([\mathbf{h}_G; \mathbf{g}])), p_{orange} = \sigma(f_{orange}([\mathbf{h}_G; \mathbf{g}])) \tag{4.17}$$

where $\mathbf{h}_G$ represents the pooled graph embedding, $\mathbf{g}$ represents the state vector, f_{blue} and f_{orange} represent the final linear layers, and σ represents the sigmoid activation function.

This design enforces architectural symmetry between the teams while allowing the model to learn team-specific scoring dynamics.

4.3.7. Temporal Sequence Models (LSTM)

While GNNs capture the spatial relationships of a single frame, they lack an inherent memory of the past. Rocket League is a dynamic system where momentum, acceleration, and intent are revealed over time. To capture these temporal dependen-

cies, we implemented Long Short-Term Memory (LSTM) networks.

4.3.7.1. Sequential Data Engineering: The Boundary Problem. Training sequence models on discontinuous replay data presents a unique engineering challenge distinct from continuous time series analysis. Unlike a contiguous stream, our dataset is composed of thousands of discrete matches, meaning that a naive sliding window approach ($L = 6$) would inevitably corrupt the input data at critical boundaries. Specifically, windows must be prevented from crossing match transitions (spanning disparate replays) and after a goal and beginning of overtime resets (where positions teleport to kickoff). Ignoring these discontinuities would force the model to learn physically impossible state jumps, which would effectively introduce high-magnitude noise into the training signal.

To solve this, we developed a Sequential Lazy Dataset with dynamic outlier handling. For every query index t, the loader attempts to construct a sequence $X_{seq} = [x_{t-L+1}, \ldots, x_t]$. It recursively validates that every frame x_{t-k} belongs to the same replay ID and score difference context as the anchor frame x_t. If a boundary violation is detected, the sequence is padded by duplicating the oldest valid frame, effectively "freezing" the state at the start of the valid window. This ensures that the LSTM never observes completely unrelated transitions.

4.3.7.2. Baseline LSTM Architecture. We first established a pure LSTM baseline that operates directly on the flattened feature sequence, as illustrated in subsubsection 4.10, which processes a flattened sequence of 6 frames ($D_{in} = 92$) through a shared LSTM layer ($H = 64$). Note the bottleneck operation at the final timestep (t_6), where dropout ($p = 0.5$) is applied before the network branches into two independent task-specific MLPs. The model ingests a temporal window of $L = 6$ frames, corresponding to one second of gameplay at 5Hz, where each timestep is represented by the standard 92-dimensional feature vector. This sequence is processed by a single LSTM layer with a hidden dimension of 64 units. To mitigate overfitting to specific trajectory patterns, we apply a dropout layer (rate 0.5) to the final hidden state $\mathbf{h}_T$ before passing the

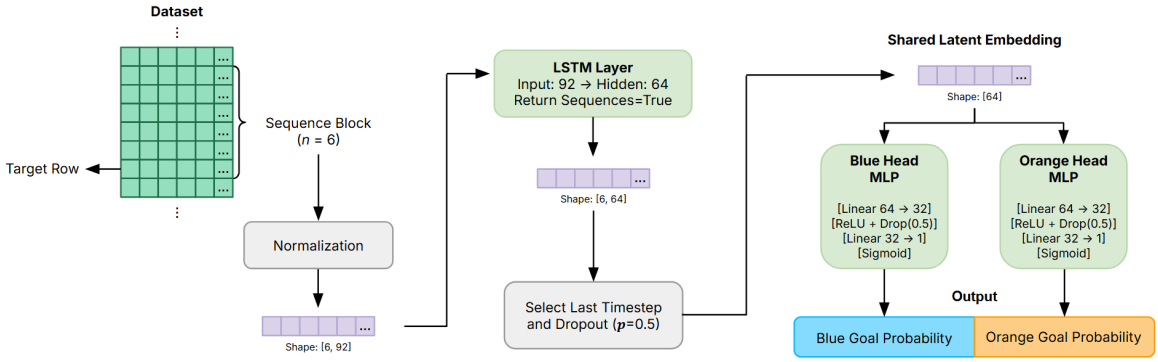


Figure 4.10. Architecture of the baseline LSTM model.

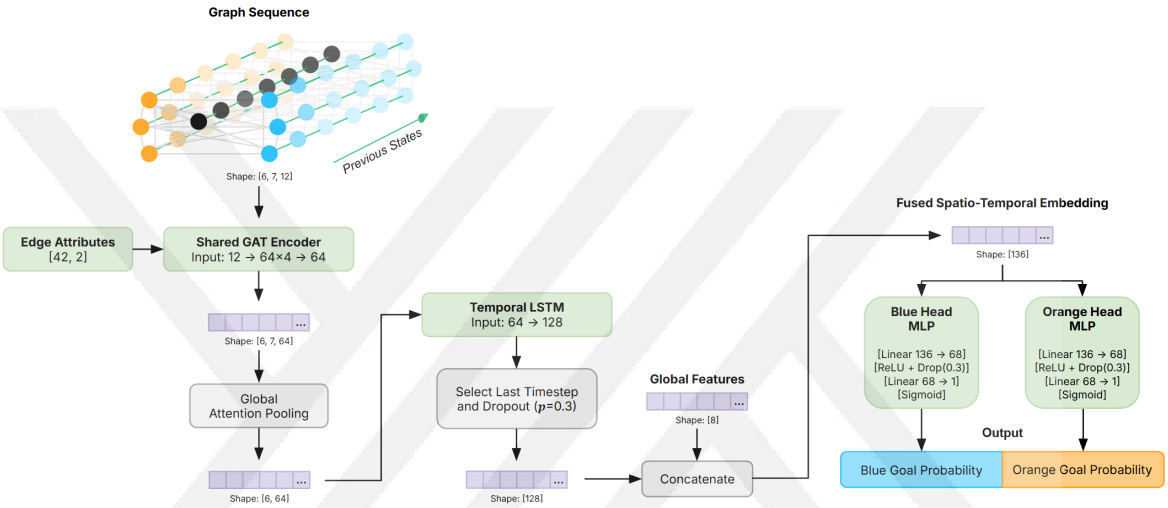


Figure 4.11. Schematic of the spatio-temporal GNN-LSTM architecture.

resulting latent vector to the independent prediction heads.

4.3.7.3. The Spatio-Temporal GNN-LSTM. To combine the spatial reasoning of GNNs with the temporal memory of LSTMs, we designed a hybrid GNN-LSTM architecture with a Late Fusion strategy. The LSTM operates exclusively on the sequence of learned graph embeddings to extract temporal dynamics from the spatial abstractions. The diagram seen in subsubsection 4.11 illustrates the Late Fusion data flow: the LSTM processes a sequence of spatial embeddings generated by the GAT (left), while the global context vector bypasses the recurrent layers and is injected via concatenation (right) immediately before the prediction heads. The global context vector is injected only at the final decision layer (subsubsection 4.11). The related vectors and states

can be calculated with the equations

$$\mathbf{h}_{graph,t} = \text{GAT}(\text{Graph}_t), \tag{4.18}$$

$$\mathbf{h}_{temporal} = \text{LSTM}([\mathbf{h}_{graph,t-5}, \ldots, \mathbf{h}_{graph,t}]), \tag{4.19}$$

$$\mathbf{z}_{final} = \mathbf{h}_{temporal} \oplus \mathbf{x}_{global,t}, \tag{4.20}$$

where Graph_t represents the constructed 7-node graph at time step t, $\mathbf{h}_{graph,t}$ represents the spatially pooled latent vector output by the GAT encoder for the frame at time t ($\mathbf{h}_{graph,t} \in \mathbb{R}^{64}$), and $[\mathbf{h}_{graph,t-5}, \ldots, \mathbf{h}_{graph,t}]$ denotes the sequential concatenation of these spatial embeddings over the window length $L = 6$. $\mathbf{h}_{temporal}$ represents the final hidden state of the LSTM after processing the sequence ($\mathbf{h}_{temporal} \in \mathbb{R}^{128}$), which captures the evolution of the game state. Finally, $\mathbf{x}_{global,t}$ is the global feature vector for the last frame ($\mathbf{x}_{global,t} \in \mathbb{R}^{8}$), $\oplus$ denotes the vector concatenation operation, and $\mathbf{z}_{final}$ is the combined feature vector ($\mathbf{z}_{final} \in \mathbb{R}^{136}$) passed to the prediction heads.

By feeding the LSTM a sequence of high-level spatial abstractions rather than raw coordinates, this architecture aims to learn the "evolution of threat" rather than just the evolution of physics.

4.4. User Interface Portal

To operationalize the predictive models and statistical frameworks described in section 4.2 and section 4.3, we developed a bespoke 3D visualization environment using the Unity Engine. While Rocket League provides a native replay viewer, it is a closed system that offers only passive viewing; it does not allow for state modification, external model integration, or quantitative analysis.

Our portal addresses these limitations by integrating the full data pipeline, from parsing to prediction, into a real-time interactive sandbox. This tool transforms the research from a descriptive exercise into a prescriptive instrument for counterfactual analysis ("What-If" scenarios), as shown in the overview in section 4.12. Here, the

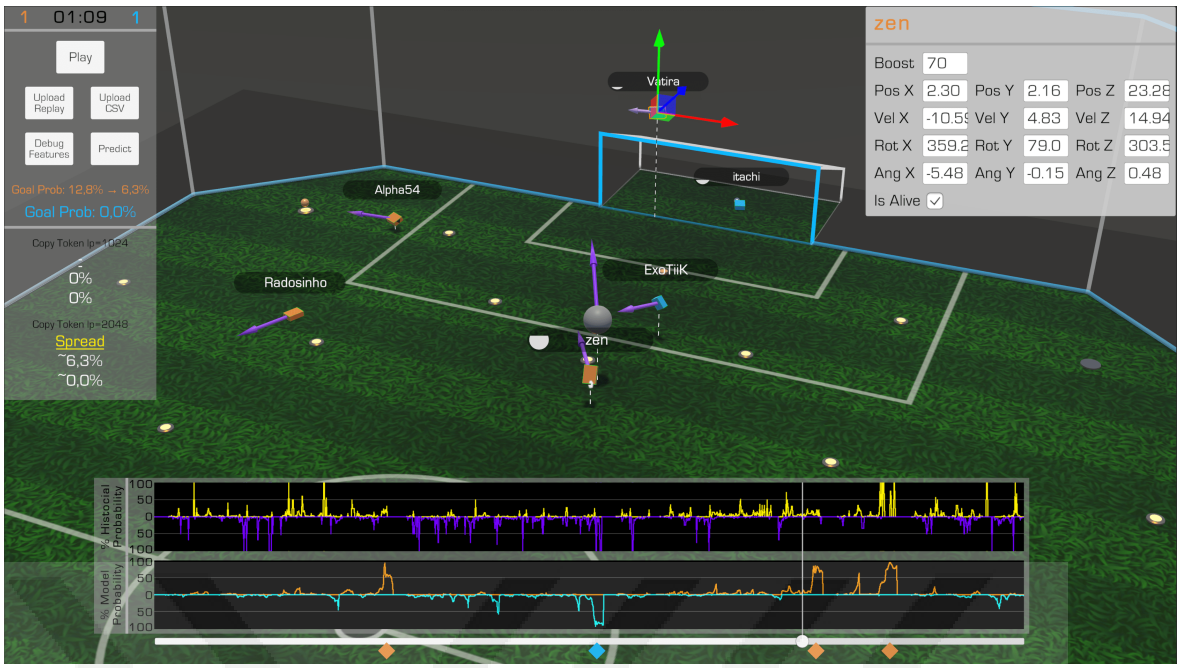


Figure 4.12. The peaRL user inteface portal.

environment renders the 3D game state with auxiliary analytical overlays, including velocity vectors (purple/white arrows), player identification plates and boost fillings, and a synchronized goal scoring probability timeline at the bottom.

4.4.1. System Architecture and Data Ingestion

The portal is designed as a standalone application that connects raw game logs with the machine learning backend.

4.4.1.1. Coordinate System Transformation. A critical engineering challenge was mapping the native Rocket League coordinate system (Unreal Engine: Z-up, Left-Handed, Centimeters) to the Unity environment (Y-up, Left-Handed, Meters). To ensure visual fidelity and correct model inference, we implemented a transformation pipeline within the Replay Manager. Raw positions $\mathbf{p}_{raw} = (x, y, z)$ are mapped to Unity vectors $\mathbf{p}_{unity}$ via

$$\mathbf{p}_{unity} = \left(\frac{x}{s}, \frac{z}{s}, \frac{-y}{s} \right) \tag{4.21}$$

where $s = 100.0$ converts Unreal Units to Unity meters. Rotational quaternions are similarly remapped from the Unreal format (q_x, q_y, q_z, q_w) to Unity's (x, y, z, w) transform space as $(q_y, q_w, q_x, -q_z)$ to align agent orientation within the field.

4.4.1.2. Replay State Management. The application maintains the entire match state in memory as a list of FrameData objects. Each frame encapsulates the discrete state of players, ball, boost pad timers, and game metadata. The visualization is not a read-only stream: the Replay Manager treats the loaded frame list as a mutable state buffer. User interactions write changes directly back into the FrameData object. This allows the predictive models to query the altered data rather than the rendered scene.

4.4.2. The Analytic Replay Viewer

The core scene functions as an enhanced replay viewer. It augments standard timeline controls with analytical overlays and synchronized threat visualizations.

4.4.2.1. Temporal Visualization. The playback system utilizes a coroutine-based clock that decouples rendering framerate from replay frequency. A replayClock variable advances based on real-time delta, and the system performs a binary search to retrieve the nearest frame from the data buffer. At each frame the transform values of the entities are changed accordingly. Additional updated visual overlays include the scoreline, seconds remaining, 3D velocity vector arrows that change color and size based on the velocity of the object, and color-coded goal markers along the scrubbing slider indicating scoring events for rapid navigation.

4.4.2.2. Temporal Threat Visualization. A central feature of the analytics interface is the temporal alignment of predictive metrics with the replay timeline. To facilitate this, two synchronized trend lines are rendered directly above the scrub bar: the Model Prediction Graph, which traces the continuous, frame-by-frame threat estimates $(\hat{y})$ produced by the neural network, and the Historical Probability Graph, which pro-

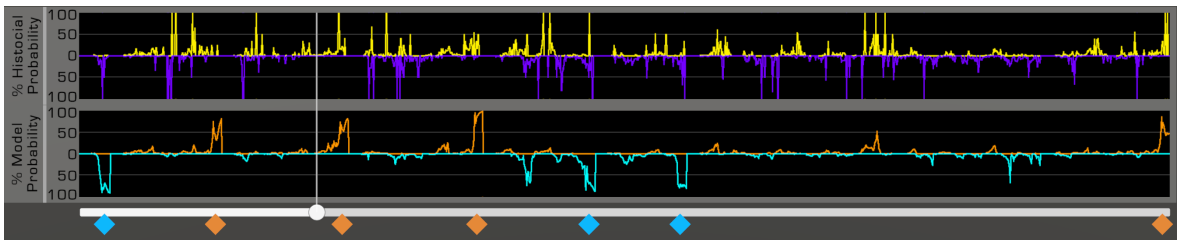


Figure 4.13. Detailed view of the synchronized goal probability graphs.

vides a comparative baseline by plotting the discrete goal probabilities derived from the Statistical Ledger (using a spatial parcel size of 2048). This pairing enables the identification of divergences, such as a high threat detected by the model but absent in historical precedent. subsubsection 4.13 illustrates the visual divergence between these two signals during high-risk moments. Here, the continuous lines on the lower graph track the ML model's prediction (Orange/Blue), whereas the lines on the upper graph track the Statistical Ledger's predictions (Yellow/Purple) with a 2048 parcel size. The markers under the interactive slider represent the goal occurrences at that moment for each team.

4.4.3. Real-Time Inference and Counterfactuals

To enable live predictions, we replicated the Python feature engineering pipeline within C#. The Feature Extractor module converts the Unity scene state into the 92-dimensional vector required by the models.

4.4.3.1. Counterfactual Interaction. To facilitate prescriptive analysis, the portal provides two distinct modalities for state modification, as visualized in subsubsection 4.14. Here, on the left, the Original state from a replay can be seen. On the right, direct manipulation of a player's (Scrub Killa) position using the 3D Transform Gizmo can be observed. The Precision Injection panel on upper right corner allows numerical adjustment of physics state variables like velocity and boost. The modified parameters are highlighted. After modification, the goal prediction value output by the ML model is compared against the original state. While a shot is incoming, moving the defending player to a safer spot decreases the attacking team's goal-scoring probability.

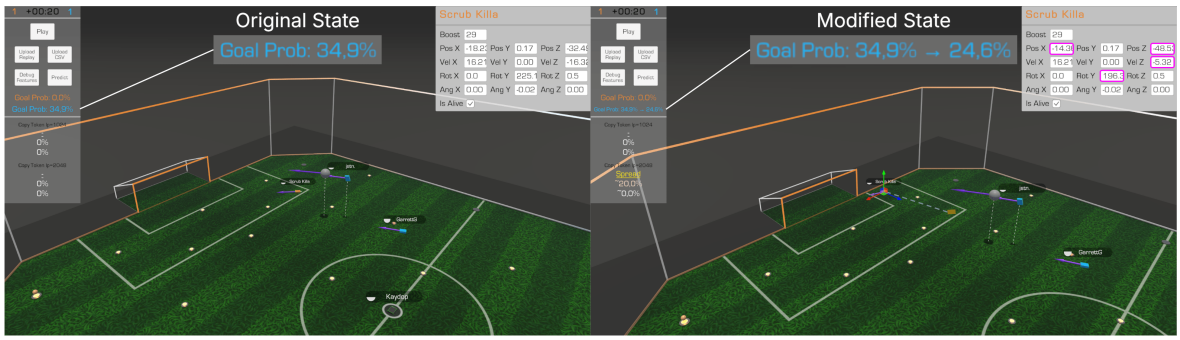


Figure 4.14. Counterfactual modification tools for the user interface portal.

For intuitive, spatial adjustments, users can select any entity to activate a 3D transform gizmo, which lets direct manipulation of position and rotation within the scene view. Alternatively, for scenarios requiring exact configurations, a dedicated Precision Injection panel offers numerical input fields and check boxes to allow users to fine-tune specific parameters such as velocity vectors and player boost amounts that are difficult to adjust visually.

4.4.3.2. The Inference Engine (Unity Sentis). For on-device execution, we integrate Unity Sentis (v2.1.3), which enables native GPU inference of ONNX models. The Model Inference script maintains a persistent worker on BackendType GPUCompute. When the user triggers a prediction, the 92-feature vector is wrapped in a float tensor, evaluated asynchronously, and the resulting logits are pushed back to the UI. With the "Debug Features" button, users can view the 92-dimensional vector input for prediction.

4.4.4. Historical Data Integration

To display statistical baselines alongside ML predictions, the full tokenization logic from section 4.2 was ported to C#. A Historical Data Manager loads the large CSV ledgers (1024 and 2048 parcel sizes) at runtime and applies canonicalization (player sorting, spatial symmetry checks) to ensure correct ledger indexing regardless of encounter orientation. For debugging and reproducibility, users can copy the original and the canonical token strings.

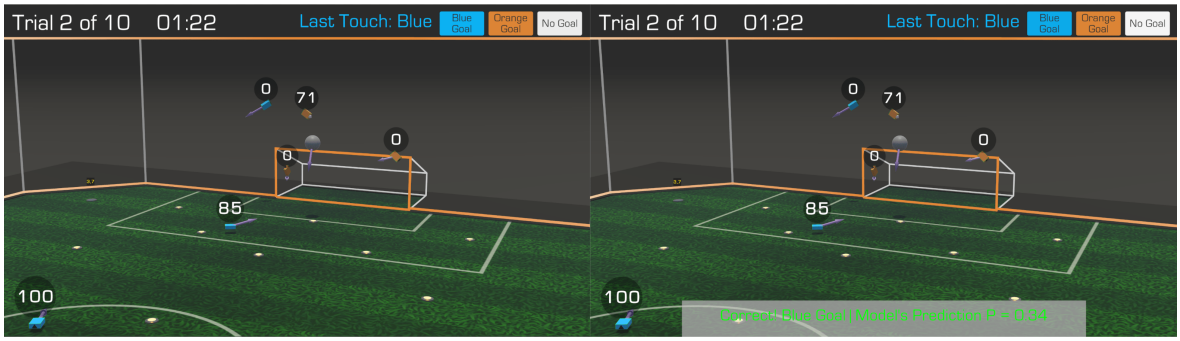


Figure 4.15. The human benchmarking module for the user interface portal.

4.4.5. Human Benchmarking Module

To contextualize model performance, a gamified Human Benchmarking Scene is created to evaluate user predictions. As seen in subsection 4.15, users are presented with a static game state and must classify the outcome. Visual aids include velocity vectors (arrows) and boost indicators to ensure fair information parity with the ML models. After the prediction is made, users immediately get feedback and see the ML model's prediction. The user classifies each state as Blue Goal, Orange Goal, or No Goal. In order to make sure everything the single-state ML model sees is also visible for the user, we include the last team to hit the ball, seconds remaining, the boost amount of each player, and the remaining seconds for each big boost pad to respawn directly on top. After the trial, the system reports the performance metrics such as Precision, Recall, and F1-Score, to provide a direct comparison between human and AI capabilities.

5. EVALUATION

5.1. Experimental Setup

5.1.1. Implementation and Training Details

All deep learning models were implemented using PyTorch (2.7.1), with Graph Neural Networks utilizing the PyTorch Geometric (PyG) library for sparse adjacency operations. Training was performed on a heterogeneous compute cluster ranging from single-GPU instances to multi-GPU nodes, depending on the memory requirements of the temporal unrolling.

5.1.1.1. Optimization Strategy. While all differentiable architectures utilized the Adam optimizer and the standardized Weighted Binary Cross-Entropy loss, specific hyperparameters were tuned per architecture family to ensure stable convergence.

- MLP Baseline: Trained with a higher learning rate ($\eta = 0.001$) and standard batch sizes, converging rapidly (≈ 8 hours).
- GNN Variants (GCN/GAT): Required a reduced learning rate ($\eta = 0.0005$) to stabilize the message-passing weight updates. Training times increased significantly ($\approx 60 - 70$ hours) due to the computational overhead of sparse matrix multiplications and the dynamic 7-node graph construction.
- Sequence Models (LSTM/Hybrid): Due to the high memory cost of backpropagation through time (BPTT) over the sequence window, the batch size was reduced from 1024 to 128. The learning rate was further annealed to $\eta = 0.0001$ to prevent exploding gradients in the recurrent layers.

The specific configuration for each architecture family is detailed in subsubsection 5.1.

Table 5.1. Hyperparameter configuration and computational cost by model family.

Model Family	**Batch Size**	**LR (η)**	**Hidden Dim**	**Hardware**	**Training Time**
XGBoost	N/A	$5e-3$	Max Depth 6	16 CPUs	$\approx$ 28 h
MLP	1024	$1e-3$	$256 \rightarrow 128$	1 GPU	$\approx$ 8 h
GNN	1024	$5e-4$	64	1 GPU	$\approx$ 60 h
LSTM	128	$5e-4$	64	1 GPU	$\approx$ 59 h
GNN-LSTM	128	$1e-4$	64 (G) / 128 (L)	2 GPUs	$\approx$ 299 h

5.1.1.2. Model Checkpointing. To prevent overfitting, we did not rely on a fixed epoch count. Instead, we employed a model checkpointing strategy by monitoring the validation loss. The model state was saved whenever the validation metric improved. For GNNs, the best performance typically emerged early (Epochs 5-10) before overfitting to the noise in the trajectory data, whereas the hybrid GNN-LSTM required significantly longer wall-clock time but converged in fewer epochs (Best: Epoch 12) due to the reduced batch size, which results in more updates per epoch.

5.1.2. Standardization of Evaluation Metrics

A critical challenge in comparing diverse models (e.g., XGBoost vs. GNNs) is ensuring that the reported loss values are mathematically comparable. While both architectures optimize for Logarithmic Loss, standard library implementations differ in their normalization strategies. Scikit-Learn (XGBoost) normalizes the loss by the sum of sample weights ($\sum w_i$), whereas PyTorch (GNN/MLP) normalizes by the total number of samples (N), regardless of the weights applied to individual instances. This discrepancy initially caused the XGBoost model to report substantially lower loss values on an incompatible scale. It ended up obscuring their true calibration behavior relative to neural models.

To ensure strict comparability, we standardized the metric across all experiments. We applied the Weighted Binary Cross-Entropy formulation post-hoc to the XGBoost predictions. This re-normalization aligns the tree-based outputs with the neural net-

work loss scale and ensures that the reported validation loss reflects the true weighted penalty ($w_{pos} \approx 30$) for missing rare goal events, consistent with the Deep Learning objectives. While XGBoost is not trained under this exact normalization, applying the same weighted BCE post-hoc ensures that evaluation reflects a common risk functional.

5.2. Baseline Models

To rigorously assess the performance of our deep learning architectures, we first established a hierarchy of baselines. These range from "zero-intelligence" heuristics to simple physics-based regressions. The primary goal of this section is to quantify the predictive floor, the performance achievable without complex spatial reasoning, and to validate the integrity of our evaluation metrics on a highly imbalanced dataset.

Analyzing these baselines allows us to calibrate our expectations for the performance metrics. Specifically, they demonstrate the failure modes of standard accuracy in imbalanced datasets and reveal the mathematical properties of the Weighted Log Loss function used in this study.

5.2.1. Baseline Results and Analysis

The performance of these baselines on the Primary Model Dataset's Test Set is summarized in Table subsection 5.2.

5.2.1.1. The Accuracy Paradox. The "Always No Goal" model achieves an Accuracy of 96.8%. This empirically confirms that accuracy is a deceptive metric for this dataset. A model that learns absolutely nothing can appear highly successful simply by predicting the majority class. Consequently, we rely on F1-Score, AUPRC, and loss metrics for model selection.

Table 5.2. Performance metrics for the trivial baselines on the test set for the primary model dataset.

Model	Team	Precision	Recall	F1	Accuracy	AUPRC	Loss
Always No Goal	Orange	0.000	0.000	0.000	**0.968**	0.031	27.312
	Blue	0.000	0.000	0.000	**0.964**	0.035	27.554
Always Goal	Orange	**0.031**	**1.000**	**0.060**	0.031	0.031	26.770
	Blue	**0.035**	**1.000**	**0.067**	0.035	0.035	26.660
Constant Prior	Orange	0.000	0.000	0.000	**0.968**	0.031	3.478
	Blue	0.000	0.000	0.000	**0.964**	0.035	3.405
Balanced Constant	Orange	0.000	0.000	0.000	**0.968**	0.031	**1.356**
	Blue	0.000	0.000	0.000	**0.964**	0.035	**1.360**
Uniform Random	Orange	**0.031**	0.502	0.059	0.500	0.031	1.953
	Blue	**0.035**	0.500	0.065	0.500	0.035	1.960
Gaussian Random	Orange	**0.031**	0.498	0.058	0.501	0.031	1.484
	Blue	**0.035**	0.498	0.065	0.501	0.035	1.482
Stratified Random	Orange	0.030	0.030	0.030	0.940	0.031	27.307
	Blue	0.034	0.033	0.034	0.932	0.035	27.534

5.2.1.2. The Sensitivity of Weighted Loss. A critical finding is the divergence in Weighted Log Loss between the Constant Prior (≈ 3.45) and Balanced Constant (≈ 1.36) models. Although the natural probability of a goal is around 0.03, predicting this value results in a higher loss because the weighted loss function heavily penalizes false negatives ($w_{pos} \approx 30$).

Mathematically, the weighting scheme shifts the "optimal zero-knowledge guess" from the natural class probability (0.03) toward the balanced center (0.5). The Balanced Constant model predicts 0.5 for everything. It minimizes risk by acknowledging the weighted importance of the positive class. On the other hand, the Stratified Random model predicts 0 or 1 based on a coin flip. Because Log Loss penalizes confident errors infinitely ($-\ln(0) \to \infty$), this model achieves a catastrophic loss of ≈ 27.5.

This establishes a clear numerical baseline for calibration: a Weighted Log Loss greater than 1.36 implies that a model's probability estimates are, on average, less reliable than simply predicting a constant 0.5 for every state. A model can still be a

strong ranker (high AUPRC) with a higher loss, but it indicates a failure in probabilistic calibration, a phenomenon we observe specifically in the XGBoost model.

5.2.1.3. The AUPRC Baseline. While Weighted Log Loss measures calibration quality, AUPRC serves as our primary metric for classification capability in this imbalanced domain. As shown in subsection 5.2, all trivial baselines yielded an AUPRC approximately equal to the positive class prevalence (≈ 0.035). This represents the performance of a "random guesser." Since AUPRC measures the ability to rank positive samples higher than negative ones, constant or random predictions provide no ranking value. Thus, true predictive success is defined not just by beating the loss floor of 1.36, but by achieving an AUPRC significantly higher than 0.035.

5.2.2. Logistic Regression Performance Analysis

subsection 5.3 summarizes the performance of the Univariate and Multivariate logistic regression models. The Univariate model, using only the ball's y position, achieves an AUPRC of ≈ 0.12. This confirms that the ball's longitudinal position is a significant predictive signal in the dataset. By capturing non-linear "danger zones" via polynomial expansion, this single feature alone provides a $4\times$ improvement over the random baseline (0.03). Adding two more important features (Longitudinal Ball Velocity and Possession) in the Multivariate model yields a significant performance jump (F1 ≈ 0.21, Loss ≈ 1.03). This establishes that prediction cannot rely on static location alone; the direction of the play and the team controlling it are essential context.

Table 5.3. Performance metrics for the Logistic Regression baselines (univariate and multivariate models) on the test set for the primary model dataset.

Model	Features	Team	Threshold	Precision	Recall	F1	Accuracy	AUPRC	Loss
Univariate	y, y^2, y^3	Orange	0.720	0.095	0.271	0.141	0.897	0.115	1.146
		Blue	0.895	0.096	**0.298**	0.145	0.877	0.123	1.265
Multivariate	y, v_y, Hit	Orange	0.786	**0.148**	**0.304**	**0.199**	**0.923**	**0.134**	**1.033**
		Blue	0.792	**0.168**	0.294	**0.214**	**0.924**	**0.145**	**1.044**

The inspection of the Logistic Regression coefficients provides a validation of the

dataset's coordinate normalization and the fundamental physics of the domain.

In the multi-variate model, the learned weights exhibit a near-perfect inverse symmetry between the two team heads, which reflects the mirrored nature of the field. For the ball's y position, the blue head learned a positive coefficient ($+1.049$), rewarding ball presence in the positive-Y half (Orange's defensive zone). Conversely, the Orange head learned a negative coefficient (-1.016), rewarding ball presence in the negative-Y half. On the other hand, for the ball's y velocity, a similar inversion is observed in velocity ($+0.608$ vs. -0.634). This confirms that the model correctly identifies that a ball moving towards the opponent's net is as critical as the ball being near it. Whereas the possession indicator (ball hit team) yielded coefficients of ± 0.28.

The univariate model, restricted to only the ball's y position, attempted to compensate for the lack of velocity data by fitting a high-degree polynomial (y^3). The drastic weight fluctuation (e.g., Orange coefficients: $y = -2.23$, $y^2 = 3.91$, $y^3 = -2.90$) indicates the model struggling to map a linear feature to a non-linear log-odds target.

5.3. Evaluation of Full State-Space Machine Learning Architectures

We evaluated the performance of our proposed architectures on the Primary Model Dataset's test set. section 5.4 presents the comprehensive results across all model configurations.

5.3.1. Performance Analysis: Generalization vs. Calibration

The transition from primitive heuristics to learned representations yielded the first major performance jump. While the trivial Constant Prediction baseline achieved 97% accuracy (due to class imbalance), it failed to retrieve goals (F1-score of 0.0). The Multi-Layer Perceptron (MLP) established a strong learning baseline with an F1-score of 0.343 (Orange) and 0.362 (Blue).

Table 5.4. Primary model dataset test results for the full state-space ML models.

Model	Team	Threshold	Precision	Recall	F1	Accuracy	AUPRC	Loss
XGB	Orange	0.593	0.372	0.319	0.344	**0.962**	0.325	1.822
	Blue	0.578	0.370	0.360	0.365	0.956	0.352	1.663
MLP	Orange	0.860	0.341	0.344	0.343	0.958	0.312	0.848
	Blue	0.855	0.366	0.359	0.362	0.955	0.331	0.850
GCN-6n	Orange	0.856	0.289	0.293	0.291	0.955	0.232	0.941
	Blue	0.837	0.284	0.340	0.310	0.946	0.255	0.948
GCN-7n	Orange	0.835	0.233	0.288	0.257	0.948	0.191	0.982
	Blue	0.816	0.238	0.326	0.275	0.939	0.212	0.983
GAT-6n-att-e1	Orange	0.875	0.337	0.350	0.343	0.958	0.306	0.853
	Blue	0.866	0.345	0.376	0.360	0.953	0.329	0.850
GAT-6n-att-e2	Orange	0.876	0.330	0.354	0.341	0.957	0.307	0.849
	Blue	0.863	0.358	0.370	0.364	0.954	0.336	0.847
GAT-6n-att-e3	Orange	0.875	0.334	0.355	0.344	0.957	0.310	0.857
	Blue	0.862	0.359	0.360	0.359	0.955	0.326	0.851
GAT-6n-att-e4	Orange	0.867	0.342	0.342	0.342	0.959	0.308	0.857
	Blue	0.864	0.354	0.373	0.363	0.954	0.333	0.845
GAT-7n-att-e1	Orange	0.880	0.349	**0.356**	0.353	0.959	0.324	0.839
	Blue	0.875	0.358	0.380	0.369	0.954	0.343	0.832
GAT-7n-att-e2	Orange	0.880	0.357	0.354	0.355	0.960	0.334	0.832
	Blue	0.876	0.367	0.387	0.377	0.955	0.352	0.832
GAT-7n-att-e3	Orange	0.895	**0.378**	0.335	0.355	**0.962**	0.333	**0.826**
	Blue	0.858	0.385	0.368	0.377	0.957	0.361	0.829
GAT-7n-att-e4	Orange	0.875	0.354	0.355	0.354	0.959	0.331	0.830
	Blue	0.876	0.360	0.386	0.372	0.954	0.348	0.833
GAT-7n-avg-e1	Orange	0.877	0.368	0.333	0.349	0.961	0.325	0.838
	Blue	0.880	**0.396**	0.346	0.369	**0.958**	0.352	0.836
GAT-7n-avg-e2	Orange	0.889	0.369	0.347	**0.358**	0.961	**0.336**	0.831
	Blue	0.854	0.370	**0.393**	**0.381**	0.955	**0.367**	**0.825**
GAT-7n-avg-e3	Orange	0.895	0.367	0.344	0.355	0.961	0.332	0.829
	Blue	0.864	0.363	0.392	0.377	0.954	0.353	0.827
GAT-7n-avg-e4	Orange	0.877	0.367	0.341	0.353	0.961	0.331	0.834
	Blue	0.869	0.355	0.387	0.370	0.953	0.348	0.828
LSTM	Orange	0.854	0.312	0.343	0.327	0.956	0.286	0.867
	Blue	0.859	0.325	0.363	0.343	0.951	0.301	0.874
GAT-7n-att-e2 + LSTM	Orange	0.872	0.371	0.339	0.354	0.961	0.330	0.848
	Blue	0.873	0.390	0.366	0.378	0.957	0.355	0.834

5.3.1.1. The Calibration Divergence (XGBoost vs. Neural Networks). Notably, the XGB model performed comparably to the MLP in terms of classification metrics (F1 ≈ 0.35, AUPRC ≈ 0.34), indicating that gradient-boosted trees and deep dense networks hit a similar performance ceiling on the flattened feature set. By the standard of ranking (AUPRC), the XGB model beats MLP by a margin of around 0.02.

However, a significant divergence was observed in the Weighted Log Loss. Despite similar ranking performance (AUPRC), the XGBoost model yielded a test loss nearly double that of the neural networks (≈ 1.74 vs ≈ 0.84). This is not a calculation

error, but a fundamental difference in calibration. Neural networks tend to produce smoother probability distributions and rarely collapse to extreme values, outputting soft probabilities (e.g., 0.1 or 0.2) even for safe states, whereas XGBoost tends to be "sharp," frequently pushing probabilities to extreme values (e.g., 10^{-6}).

Because Weighted Log Loss penalizes confident errors heavily, XGBoost incurs massive penalties for the rare instances where it confidently predicts "No Goal" on a state that results in a goal. This suggests that while XGBoost is an effective ranker of danger, Neural Networks provide safer, more calibrated probability estimates for a user-facing analytics tool.

5.3.1.2. Head-Wise Performance Asymmetry. A consistent trend observed across every evaluated architecture (from Logistic Regression to the Hybrid GAT-LSTM) is a performance divergence between the team-specific prediction heads. The Blue Team models consistently outperform their Orange counterparts by a margin of approximately ΔAUPRC ≈ 0.02 and ΔF1 ≈ 0.02.

This discrepancy is attributable to the marginal class imbalance within the Primary Model Dataset. The calculated positive class weights for the loss function were $w_{pos} = 28.16$ for the Blue team and $w_{pos} = 31.67$ for the Orange team. Since these weights are inversely proportional to the frequency of positive labels (goals), this indicates that the training set contains a slightly higher volume of Blue goals than Orange goals.

In imbalanced learning, even a marginal increase in the minority class volume can stabilize gradient descent and improve the definition of the decision boundary. Consequently, the Blue head benefits from a richer signal density, which creates a higher precision-recall stability. The uniformity of this gap across all model types confirms that the asymmetry is intrinsic to the dataset distribution rather than an architectural bias.

5.3.2. The Failure of Isotropic Graph Convolutions (GCN)

To test the hypothesis that graph topology improves prediction, we introduced the Graph Convolutional Network (GCN-6n). Contrary to expectation, this model underperformed the flat MLP baseline, dropping to an F1-score of roughly 0.30.

This negative result highlights a critical limitation of the GCN architecture in this domain, namely, Isotropic Averaging. By treating all neighbors equally (normalized only by degree), the GCN effectively "blurs" the distinct signals of the players. In a high-speed scenario, a defender 5 meters away is critically different from a striker 5 meters away; the GCN, unable to distinguish these via edge attributes, dilutes the signal.

5.3.3. Topology Impact: The "Ball-as-Node" Dilemma

We further investigated the graph topology by explicitly modeling the ball as a 7th node. In GCNs, performance degraded further (F1 $\approx$ 0.25). Because the GCN forces the ball node to aggregate information from players using the same shared weights as player-to-player messages, the unique physics of the ball were likely washed out by the noise of player features. When switching to GATs, this trend reversed. The GAT-7n variants consistently outperformed their 6-node counterparts. The attention mechanism allowed the network to learn asymmetric weights. This means that they are effectively learning to "listen" to the ball node differently than it listens to player nodes.

5.3.4. Ablation of Edge Features

To determine the optimal relational bias, we performed a systematic ablation study across all three GAT variants (GAT-6n-att, GAT-7n-att, and GAT-7n-avg). We incrementally injected explicit edge attributes (e_1 to e_4) into the attention mechanism to isolate their impact on the model's ability on predicting.

- Inverse Distance (e_1): Serving as the foundational baseline, proximity alone provided a strong discriminative signal. The models achieved an initial AUPRC ranging from 0.306 (6n-att) to 0.352 (7n-avg) on the test set. This confirms that Euclidean distance is a driver of attention in the Rocket League state space.
- Team Identity (e_2): The addition of the binary "Teammate/Opponent" flag yielded the most significant and consistent performance lift. In the 7n-avg configuration, adding this feature pushed the Blue team AUPRC to its study-wide peak of 0.367, with similar gains observed in the Orange head (0.336). This confirms that explicitly distinguishing "Friend vs. Foe" is critical for correctly ranking threat probabilities, as the strategic implication of proximity is inverted depending on team alignment.
- Diminishing Returns on Kinematics (e_3, e_4): The inclusion of Velocity Alignment (e_3) and Forward Orientation (e_4) resulted in negligible gains or even slight regressions. For instance, in the best-performing 7n-avg model, adding velocity alignment caused the Blue AUPRC to drop from 0.367 (e_2) to 0.353 (e_3). This suggests that the Graph Encoder is sufficiently capable of inferring relative motion dynamics directly from the node-level features, which makes the explicit edge-encoding of these properties redundant and potentially noisy.

The consistency of these AUPRC trends across all three architectures indicates that while simple priors (Distance, Team) are essential for guiding the attention head, complex kinematic relationships are effectively learned implicitly by the network.

5.3.5. The Hybrid GAT-LSTM and the Interpretability Trade-off

Finally, we evaluate the graph-based LSTM. For the spatial encoder of this hybrid architecture, we selected the GAT-7n-att-e2 configuration. Our primary motivation was the preservation of signal complexity; while Mean Pooling tends to "smooth" the graph into a uniform average, we hypothesized that Attention Pooling would generate distinct and high-variance feature vectors that allow the LSTM to track specific key actors (e.g., the ball carrier) across the temporal window. Additionally, this choice

maintains the architectural potential for explainability, preserving the latent importance scores (α_i) for future granular analysis. Ultimately, the Hybrid model achieved a Blue AUPRC of 0.355, which is statistically highly comparable to the static attention baseline (0.352) and only marginally trails the highest performer of mean pooling (0.367). This performance plateau suggests that the spatial configuration of a single frame is so highly predictive of the immediate future that it effectively saturates the predictive signal for short horizons, which leaves little marginal gain for complex temporal sequence modeling.

5.4. Stratified Performance and Interpretability Analysis

While aggregate metrics like AUPRC provide a high-level summary of model performance, they obscure the internal logic driving predictions and the model's robustness across specific gameplay scenarios. In this final analysis, we deconstruct the model's decision-making process through feature attribution and assess its generalization capability across distinct threat levels and competitive ranks.

5.4.1. Interpretability: The Physics of Prediction

To understand the drivers of the predictive signal, we analyzed the Feature Gain scores from the XGBoost model. As a tree-based model, XGBoost provides a transparent measure of "Gain" that represents the average reduction in training loss contributed by a feature each time it is used for a split. subsection 5.1 visualizes the top 30 features for both the Orange and Blue team models.

5.4.1.1. Possession as the State Switch. The single most informative feature across both models is the ball hit team number, which implies the team of the last player to hit the ball, and can be generalized as possession. This categorical variable encodes the "State of Play": Blue Possession (0), Orange Possession (1), or Neutral/Kickoff (0.5). The high gain attributed to this feature partially stems from the "Kickoff State" (0.5). In Rocket League, a goal immediately following a kickoff is statistically rare; for the

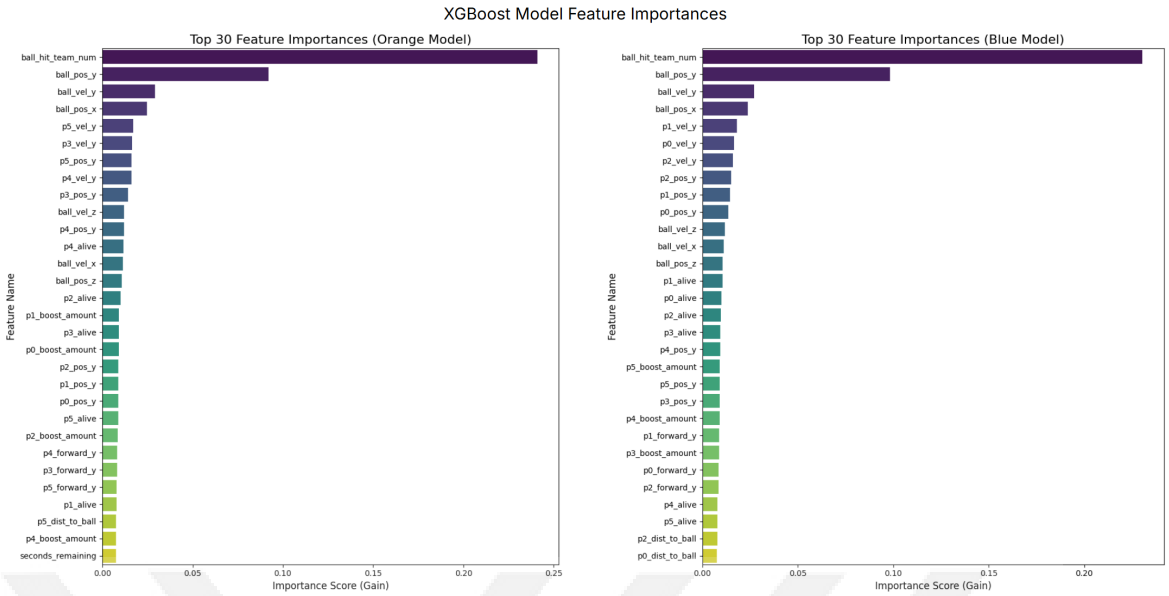


Figure 5.1. Top 30 feature importances (gains) for the XGBoost dual-models.

Primary Model Dataset, the kickoff goals are only 2.1% for both teams combined. By identifying this neutral state, the model effectively learns a "safe mode" and suppresses goal probabilities until active possession is established. Beyond kickoffs, this feature instantly orients the model's physics engine. If the ball hit team number is equal to 0 (Blue), the Blue model treats the state as an offensive opportunity, while the Orange model treats it as a defensive situation. From the Primary Model Dataset, it is statistically observed that the team with the last touch is approximately 3.8 times more likely to score than to be scored upon in the subsequent window.

5.4.1.2. Kinematic and Role-Based Prioritization. Following possession, the hierarchy of importance strictly follows physical intuition. The ball's longitudinal position (ball_pos_y) and velocity (ball_vel_y) are the dominant continuous features. This confirms that, at its core, goal prediction is highly influenced by the ball's distance to the net and speed of approach towards it. The dataset structure fixed Players $\{0, 1, 2\}$ as Blue and Players $\{3, 4, 5\}$ as Orange. The feature plots reflect this asymmetry. The Blue Model places higher importance on its own attackers to calculate offensive viability, while the Orange Model prioritizes its own corresponding players for the same reason. This structural consistency validates that the models are not overfitting to

noise but are instead learning a generalized representation of "Attacker vs. Defender" dynamics. Additionally, the models assign higher importance to the defending team's boost reserves than to the attackers'. This aligns with the tactical concept of "boost starvation"; a defender with depleted resources is a much stronger predictor of a conceded goal than a fully-boosted attacker is of a scored goal. This suggests that the model has correctly identified defensive resource depletion as a primary failure state.

5.4.1.3. The Structural Engine: Attention Distribution. To understand how the Graph Neural Network processes the spatial configuration of the team, we extracted the learned attention weights (α_i) from the GAT-7n-att-e2 model. Unlike other architectures, the GAT assigns importance scores to nodes relative to the graph's topology.

We aggregate these weights based on canonical tactical roles used in Rocket League gameplay analysis relative to the team perspective of the prediction head (e.g., for the Orange Head, Orange players are "Allies" and Blue players are "Opponents"). This allows us to average the attention distributions across both heads into a unified view using the Rocket League community jargon:

- 1st Man: The player closest to the ball (The Challenger).
- 2nd Man: The player with the second closest proximity, supporting the play.
- 3rd Man: The player furthest from the ball, typically covering the defensive backfield.

As illustrated in subsubsection 5.2, the GAT did not converge to a sparse selection strategy (i.e., focusing solely on the "1st Man" or "Ball"). Instead, it converged to a "Soft Mean Pooling" strategy. The Ball node received the highest average attention (0.160), slightly above the uniform baseline ($1/7 \approx 0.143$), while all player roles, regardless of being the 1st, 2nd, or 3rd man, hovered between 0.138 and 0.142.

Crucially, the model assigns statistically equivalent importance to both friends and foes; the weights for Ally and Opponent roles hovered tightly together between

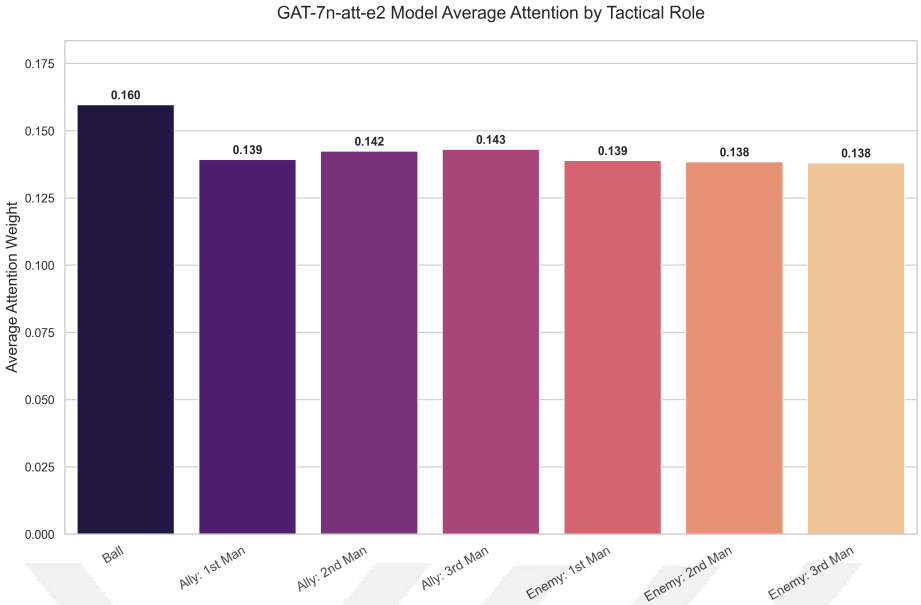


Figure 5.2. Average attention weight distribution by tactical role for the GAT-7n-att-e2 model.

0.138 and 0.142. This indicates that the network learned that "ignoring" the enemy is just as dangerous as ignoring a teammate. In a high-velocity domain like Rocket League, the positioning of the "Opponent 3rd Man" is often just as predictive of a future outcome as the mechanics of the "Ally 1st Man," validating the competitive performance of the isotropic Mean Pooling variants observed.

This finding offers an architectural insight that the model effectively learned that "ignoring" any player is statistically dangerous. In a high-velocity domain like Rocket League, the off-ball positioning of the "3rd Man" is often just as predictive of a future goal (or save) as the on-ball mechanics of the "1st Man." Additionally, the model assigns statistically equivalent importance to both friends and foes; the weights for Ally and Enemy roles hovered tightly together. This validates the competitive performance of the isotropic Mean Pooling variants observed; the optimal strategy for this feature space appears to be a holistic aggregation of the entire field rather than a selective attention mechanism.

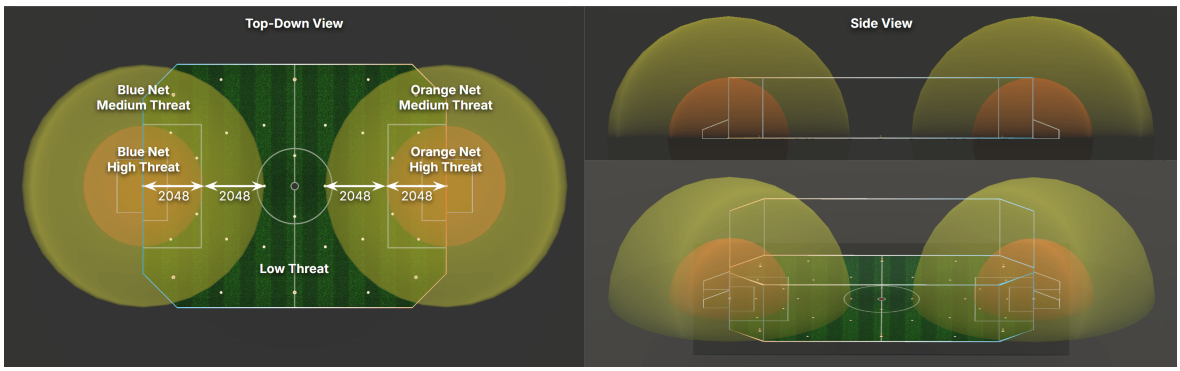


Figure 5.3. Threat characterization logic visualization.

5.4.2. Stratified Performance: Threat Zone Analysis

To assess model robustness across varying degrees of gameplay intensity, we partitioned the test set into five distinct "Threat Zones" based on the ball's Euclidean distance to each net. This stratification allows us to isolate performance in high-variance situations versus low-entropy midfield play.

5.4.2.1. Zone Definition and Distribution. We characterized the game state into five mutually exclusive categories using distance thresholds (High: < 2048 uu, Medium: 2048 to 4096 uu). subsubsection 5.5 details the distribution of states and goal events across these zones for the test set.

This distribution reveals a clear "Risk/Reward" gradient. The Low Threat zone accounts for the majority of gameplay (around 43%) but contains a diffused goal signal. In contrast, the High Threat zones are sparse (around 6% of the entirety of data) but highly concentrated with positive labels for the attacking team.

5.4.2.2. Comparative Performance Analysis. subsubsection 5.4 presents the AUPRC scores for the XGBoost and the three primary GNN variants across these zones. The XGBoost model was elected as the baseline for tabular learning. It serves as the control group to measure the specific value of deep geometric learning against a gradient boosting standard. The GAT-7n-att-e2 model was chosen as the representative for the

Table 5.5. Distribution of test set states and positive labels across threat zones for primary model dataset.

Model Perspective	Threat Category	Positive States	Total States
Orange Head	Orange Net High Threat	262	103,007
	Orange Net Medium Threat	2,401	329,478
	Low Threat	13,873	654,669
	Blue Net Medium Threat	15,520	319,812
	Blue Net High Threat	14,808	97,191
Blue Head	Orange Net High Threat	16,443	103,007
	Orange Net Medium Threat	17,743	329,478
	Low Threat	15,730	654,669
	Blue Net Medium Threat	2,506	319,812
	Blue Net High Threat	395	97,191

explicit attention mechanism. This model tests the hypothesis that learning distinct importance weights for each player yields better generalization than uniform aggregation. The GAT-7n-avg-e2 model was selected as the top-performing isotropic model. Its inclusion allows us to isolate whether the "complexity" of the attention head is necessary, or if a simple holistic average of the field is sufficient for high-level prediction. Finally, the hybrid GAT-LSTM model was included to represent the temporal domain. This model serves to verify whether adding historical context provides stability. The analysis yields three critical insights regarding the "Predictive Horizon" of the models:

The Saturation of High-Threat States. In the High Threat zones (e.g., "Blue Net High Threat"), all models achieve their peak performance (AUPRC ≈ 0.62). As shown in subsubsection 5.5, these zones are physically dense with goal events ($N_{goals} \approx 15,000$). In these scenarios, the physical signal is overwhelming: the ball is close to the goal and moving fast. Consequently, the performance gap between the simple XGBoost (Red) and the complex GAT-LSTM (Yellow) is minimal. This can bring the idea of when a goal is imminent, physics (kinematics) dominates strategy.

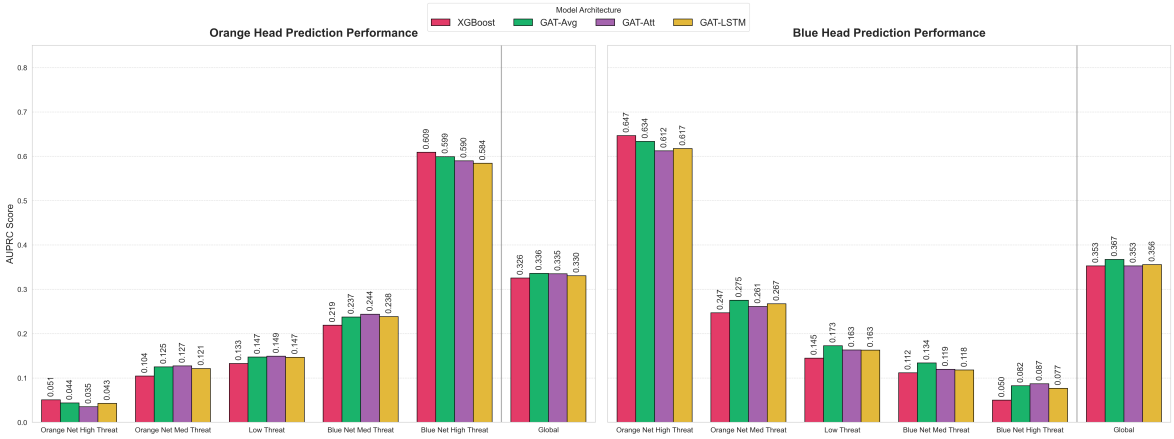


Figure 5.4. Stratified AUPRC performance across threat zones for XGBoost, GAT-7n-avg-e2 (GAT-Avg), GAT-7n-att-e2 (GAT-Att), and GAT-LSTM hybrid models for the test set of the primary model dataset.

The Strategic Advantage in Medium Threat. The most significant divergence occurs in the Medium Threat zones. Here, the GAT-7n-avg (Green) consistently outperforms the XGBoost baseline. For example, in the "Orange Net Medium Threat" zone, the GAT-Avg achieves an AUPRC of 0.275 compared to XGBoost's 0.247. This indicates that graph-based architectures are slightly better at recognizing developing plays, situations where the ball is not yet critically dangerous, but the spatial arrangement of the attacking team implies a high probability of a scoring chance.

The Difficulty of the Neutral and Low Risk Zones. In the Low Threat (Midfield) zone, performance universally drops (AUPRC ≈ 0.15). Despite containing the highest raw count of goals ($29,603$ combined), these positives are diluted by a massive volume of non-goal states ($654,669$), which creates a high-noise environment. However, the GNN variants maintain a slight edge over XGBoost, validating that structural awareness helps filter noise even when the goal probability is statistically low. This signal degradation is most pronounced in the "Deep Defense" zone; naturally, when the ball is threatening one net, the immediate threat to the opposite net is negligible. The extreme scarcity of such long-range counter-attack goals results in the lowest observed performance across all spatial partitions.

5.4.2.3. Ranked Generalization Dataset. The Primary Model Dataset on which the ML models were trained on are exclusively based on professional tournament matches. This means that the models have learned the highest skill gamelay, whereas the majority of Rocket League players lack the skill set seen by the professional players. To evaluate the model's adaptability across the full spectrum of competitive play, we curated a specialized Ranked Generalization Dataset. This dataset consists of 30 randomly selected full-length replays for each of the 22 competitive ranks, ranging from Bronze I to Supersonic Legend (SSL).

The processing pipeline successfully extracted features from all target replays without failure, yielding a total of $1,011,022$ frames. A distinct variance in match duration was observed across the hierarchy: while the lower and intermediate ranks (Bronze through Champion) averaged approximately 46,000 frames per rank group ($\approx$ 5.1 minutes of gameplay per match), the SSL rank comprised 62,608 frames ($\approx$ 7.0 minutes per match). This increase in frame count at the highest level is indicative of extended match durations, likely resulting from tighter defensive play leading to frequent overtime scenarios.

We evaluated the four candidate models (GAT-Att, GAT-Avg, GAT-LSTM, and XGBoost) on these rank-specific subsets to assess performance consistency. subsubsection 5.5 illustrates the global AUPRC evolution across all ranks for both teams. For a comprehensive decomposition of these trends stratified by the Threat Characterization System (Low, Medium, and High risk), please refer to Appendix A.1 in the Appendix.

The results highlight several key behavioral trends. The GAT variants (GAT-avg and GAT-att) usually maintain the upper bound of performance across most ranks. In contrast, the XGBoost model (red line) exhibits lower AUPRC peaks and more severe performance dips, particularly on the ranks before Diamond I.

There are notable dips and highs in predictive performance observed throughout the ranks. It can be observed that the model performance results for the blue and

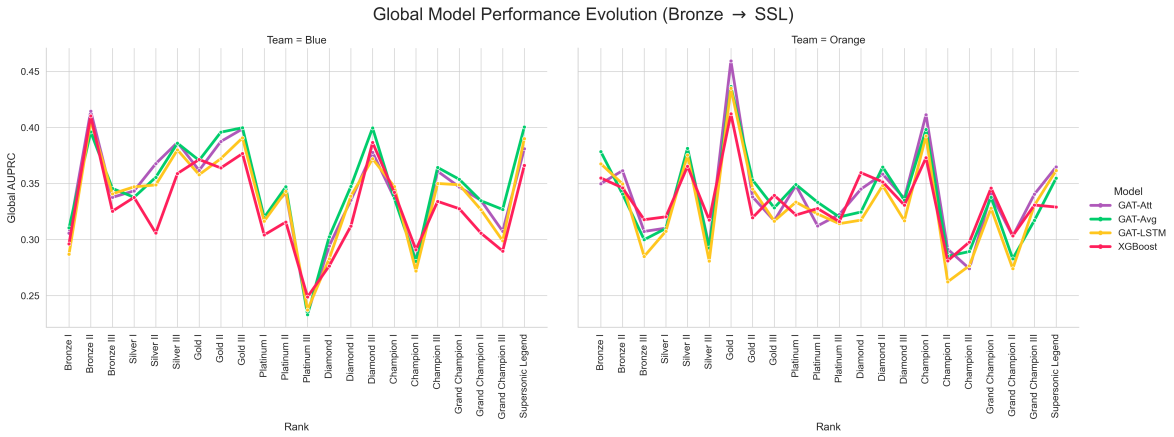


Figure 5.5. Model performance (AUPRC) across competitive ranks from the ranked generalization dataset for XGBoost, GAT-7n-avg-e2 (GAT-Avg), GAT-7n-att-e2 (GAT-Att), and GAT-LSTM hybrid models.

orange teams have no specific correlation except for the ranks beyond Champion I. This may be due to the understanding of the lower ranks as transitional zones where player consistency is highly variable, which makes goal outcomes inherently more stochastic and harder to model compared to the structured play of lower or higher ranks.

Despite the increased complexity of play at Supersonic Legend (SSL), the models demonstrate a sharp increase in performance (particularly visible in the Blue team evaluation). Combined with the longer average match duration, this suggests that while SSL gameplay is faster, it is also more systematic and "rational," which allows the graph-based models to capture the underlying spatial logic more effectively than in the chaotic intermediate ranks.

To contextualize these trends, we compared the global performance baselines established in section 5.4 against the aggregate statistics of the Ranked Generalization Dataset. On the Primary Model Dataset, the selected architectures achieved AUPRC scores of approximately 0.36 (Blue) and 0.33 (Orange). These figures align closely with the global averages observed in the ranked dataset, which yielded means of 0.34 for Blue and 0.33 for Orange. However, this stability in the mean masks significant volatility: the Blue team performance fluctuates drastically between a minimum of 0.24 (Platinum

III) and a maximum of 0.41 (Bronze II), with the Orange team exhibiting a similar spread with a minimum of 0.28 (Champion II) and a maximum of 0.44 (Gold I). This confirms that while the models generalize robustly to unseen replays on average, their predictive reliability is heavily dependent on the specific entropy of the competitive rank.

6. DISCUSSION AND FUTURE WORK

This chapter provides a critical examination of the proposed predictive framework and outlines potential avenues for future research. The discussion is divided into two primary sections: first, a retrospective analysis of the model performance, feature engineering choices, and technical limitations; second, an exploration of the broader implications, including advanced architectural possibilities, interactive applications, and the evolving nature of esports analytics.

6.1. Critique of Model Performance and Methodological Choices

The experimental results presented in the evaluation chapter reveal a nuanced relationship between architectural complexity and predictive performance. Our comprehensive benchmark, spanning from primitive heuristics and linear baselines to advanced Temporal GNNs, demonstrates that while learned representations significantly outperform statistical baselines, the marginal gains in predictive ranking (AUPRC) diminish as model complexity increases. Notably, we observed a performance convergence among the top-tier architectures, suggesting that the kinematic signal of Rocket League with the global context is robust enough to be captured by diverse modeling paradigms. Consequently, the primary advantage of the proposed Graph Attention networks lies not in raw predictive dominance but in their superior probabilistic calibration and their unique capacity to provide structural insights.

6.1.1. Feature Engineering and State Representation

The current feature set relies heavily on positional and kinematic data (position, velocity) but abstracts away microscopic mechanical details that are critical at the highest levels of play. Specifically, the model does not explicitly track hidden internal states such as car acceleration (throttle), jump availability, the flip timer after a jump, or power slide status. In Rocket League, the possession of a secondary jump (flip)

sometimes dictates whether an important action is physically possible, regardless of the car's current velocity or position. Integrating these states would allow the model to distinguish between a truly helpless player and one who retains the mechanical capacity for a recovery or an extra play potential, thereby reducing false predictions in a variety of scenarios.

Additionally, the models currently treat players as point masses or centroids. However, the game physics rely on precise hitboxes (oriented bounding boxes). Incorporating features such as the distance between the ball and the car's closest vertex, rather than its center could improve the model's sensitivity to hit challenges on entities and precise dribbling mechanics. Similarly, the model currently generalizes all vehicles. However, the choice of "Battle Car" (e.g., Octane vs. Dominus) dictates the hitbox dimensions and turning radius. Including the car type as a categorical feature could help the model account for the slight mechanical advantages specific hitboxes offer in tight interactions.

The system currently treats all agents anonymously. In reality, each player controlling the car has a different mechanical skill set, which would affect the crucial predictions. Incorporating player-specific embeddings (e.g., historical average goals, saves, shots, points per match, playstyle vectors) could allow the model to adjust its predictions based on the known lethality or defensiveness of specific players. Analogous to how a basketball player's unique 3-point efficiency alters the probability of a scoring event, these priors would allow the model to weight identical spatial opportunities differently based on the agent's identity.

In addition, the current model focuses on individual agent kinematics but lacks explicit "gestalt" features that quantify collective team shape. Hand-crafted geometric features, such as the Team Triangle Area (measuring spatial spread), the Average Team Centroid, or the total inter-agent distance, could explicitly quantify concepts like defensive compactness versus offensive over-extension. By feeding these high-level macro-features alongside the raw node data, the model could more easily learn to penal-

ize poor structural positioning without needing to infer it solely from raw coordinates.

Finally, the model operates under a deaf constraint. In high-level competitive play, team rotations are often coordinated via voice communication (e.g., calling out a "fake" or a specific pass). This intent channel is invisible to the replay parser. Consequently, the model may penalize a player for being out of position when they were actually executing a coordinated feint, which represents an irreducible error floor in any model trained solely on kinematic data. Although this voice data is a challenge to acquire, it may lead to even more future research directions for analyzing team plays.

6.1.2. Temporal Horizons and Goal Definitions

The binary classification task (Goal vs. No Goal within 5 seconds) imposes a rigid temporal horizon. While 5 seconds provides a practical window for tactical analysis, it is ultimately an arbitrary threshold.

Future iterations could evaluate performance across multiple horizons (e.g., 3s, 10s, 15s). A 3-second window might better capture immediate "striker" mechanics, while a 10-second window could highlight build-up play and midfield control. Additionally, reformulating the problem from binary classification to a regression task (predicting the continuous time-to-goal) would resolve the boundary issues where a goal scored at $t = 5.1s$ is currently penalized as a false negative. While this formulation represents the theoretical ideal for continuous threat assessment, it introduces significant stochastic noise, as the exact duration of a scoring sequence is inherently more variable than the binary outcome of the play.

6.1.3. Data Resolution and Scalability

The decision to sample data at 5Hz was a necessary trade-off between computational efficiency and fidelity. However, Rocket League's physics engine operates at 120Hz. The 5Hz sampling rate inevitably acts as a low-pass filter that smooths out

high-frequency mechanical inputs.

Furthermore, questions remain regarding data scalability. While the Primary Model Dataset was substantial, a larger corpus could potentially smooth the volatility observed in the Ranked Generalization Dataset analysis. The volatility in mid-rank performance (Platinum/Diamond) suggests that 30 replays per rank may be insufficient to capture the high variance of skill in those tiers as well. Similarly, increasing the Historical Augmentation Dataset might have further regularized the statistical ledger predictions by exposing the model to a wider variance of non-standard play patterns.

6.2. Future Work and Broader Implications

Beyond immediate technical refinements, this research opens several pathways for interdisciplinary exploration, ranging from advanced machine learning architectures to human-computer interaction (HCI) applications.

6.2.1. Advanced Architectural Paradigms

While GNNs effectively capture spatial relationships, alternative architectures could offer superior handling of complex temporal dependencies. One example is the Transformer architectures; the "attention is all you need" paradigm has shown promise in trajectory prediction and sports analytics. A Transformer-based model could treat the match as a sequence of tokens, potentially learning long-range dependencies that LSTM layers struggle to capture. In addition, moving from predictive (what will happen) to prescriptive (what should happen), future systems could utilize state-of-the-art Reinforcement Learning (RL) agents. Bots like Seer [59] have demonstrated superhuman performance; their policy networks could be distilled to provide users with the "optimal move" in any given scenario. However, this introduces a critical interpretability dilemma: disentangling execution from intent. It remains an open research question whether these agents dominate due to a genuine superior strategy or simply because they possess "frame-perfect" mechanical precision that physically

outpaces the strategic depth of human players.

6.2.2. Interactive and Immersive Analytics

The Unity-based user portal demonstrates the potential of 3D visualization, yet the medium of interaction offers vast room for expansion.

Porting the analysis interface to extended reality (XR) headsets could allow coaches and teams to "step inside" the replay. A collaborative XR environment would enable multiple users to stand on the virtual field, pointing out spatial gaps and passing lanes in a shared, immersive 3D space. This would potentially be more immersive than interaction over 2D screens or tangible tools like tactical boards. Additional features, such as saving and loading state snapshots, a more detailed difference visualization system (e.g., ghosts), would enable even more detailed analysis for the users.

Beyond manual exploration, the tokenization of the game state opens the door for powerful Case-Based Reasoning. By encoding match situations into search-optimized embeddings, future systems could allow users to query the database for historical analogues. Instead of manually constructing a scenario, a coach could select a specific formation and ask, "Show me how professional teams usually resolve this specific geometric setup." This effectively transforms the dataset from a static archive into a semantic search engine for tactical decision-making.

While the current user portal allows users to modify a static state, integrating temporal models (like LSTMs) into the user portal introduces significant user experience (UX) challenges. Modifying a sequence for counterfactual analysis is far more complex than moving a static object, as the user would need to define a trajectory (or multiple states) rather than a point. This complexity suggests that static graph models may remain superior for intuitive "what-if" exploratory tools.

To create a well-calibrated human benchmark, future studies could utilize a gam-

ified web interface of the guessing scene in the user portal. By collecting thousands of binary (or slider-based probability estimate) predictions from the community (crowd-sourcing), we could construct a robust "Human Oracle" baseline to compare against the model performances.

Ideally, the system should carry both descriptive and prescriptive capabilities. By integrating a Monte Carlo sampling approach, the portal could autonomously generate thousands of slight variations of the current state, perturbing the player's position and velocity, to identify the local maximum for scored goal probability, or the local minimum for conceded goal probability. This would allow the system to visualize the optimal trajectory, where, for instance, it explicitly shows the user: "If you had been 10 units to the left, your team's scoring probability would have increased by 15%."

While the current system offers visual interpretability through the user portal, future work should focus on mathematical explainability. The analysis of raw attention coefficients could quantify exactly which edge interactions (e.g., 'Player A $\rightarrow$ Ball' vs. 'Player A $\rightarrow$ Player B') contributed most to a specific threat prediction. This is useful for validating whether the model's logic aligns with human tactical intuition.

6.2.3. Operational Viability and Scalability

Moving from a research prototype to a production system involves several logistical hurdles regarding deployment and scope.

The ultimate application of this framework is real-time analysis. Implementing this pipeline for live matches would benefit broadcasters and coaches instantly. However, this requires reading memory directly from the game client rather than parsing replay files, a task that may introduce significant anti-cheat compliance issues. Additionally, deployment introduces a strict latency budget. While the simpler models (XGBoost, MLP) offer near-instantaneous inference suitable for high-frequency game loops, as the models get more complex, they incur higher computational costs. A

production-grade real-time overlay would likely require model quantization or knowledge distillation, compressing the heavy models into lighter ones to ensure the analytics do not impact the client's frame rate.

Beyond real-time prediction, the proposed framework holds significant potential for analysis over a long time. Future research could aggregate these predictive metrics over the course of a full season to evaluate the evolution of a team's performance. By analyzing trends in Average Goal Probability Created or Defensive Threat Mitigated, analysts could objectively quantify whether a team is improving its structural efficiency over time. Furthermore, analyzing the variance in these metrics could define a metric for "Teamness" or synergy, which quantifies how well specific player combinations collaborate to maximize expected value compared to the sum of their individual mechanical outputs, and how they evolve over the span of a predetermined period of time.

A more direct user application could be an in-game plugin (e.g., via BakkesMod) that overlays win probability in real-time while viewing a replay. Although technically feasible, such a system is harder to maintain than a standalone portal due to the strict limitations on third-party code injection and the lack of UI freedom compared to a Unity environment.

This study focused exclusively on standard 3v3 play. Adapting the system for 1v1 (Duel), 2v2 (Doubles), or 4v4 (Chaos) would require a complete retraining of the pipeline. The spatial dynamics of each gamemode are fundamentally different, meaning a model trained on 3v3 data would likely fail to generalize.

Periodic game updates sometimes alter the replay file structure or naming of the parameters, which breaks the parsers that lead to the whole system's dysfunction. This means that if the system is not maintained properly, future replay files may not be compatible with the system. A robust production system would require an automated CICD pipeline for data ingestion that detects version mismatches and triggers retrain-

ing. Moreover, the model faces the challenge of "Skill Inflation" or Concept Drift. As the competitive player base evolves, the mechanical baseline for a given rank increases; an RLCS level player in 2025 is significantly more skilled than an RLCS level player from 2020. Consequently, a static model trained on historical data will eventually lose calibration, as the definition of a "high-threat" situation shifts with the meta-game (the prevailing community strategies). Ensuring long-term viability requires a continuous retraining loop to capture these evolving standards of play.

6.2.4. Rocket League as a Proxy for Sports Analytics

Finally, this work reinforces the position of Rocket League as an ideal sandbox for broader sports analytics research. While categorized as an esport, its continuous physics-based interactions and absence of fantasy abilities make it structurally comparable to traditional invasion sports, such as hockey, soccer, basketball, American football, rugby, handball. Unlike other esports genres that rely on discrete cooldowns or rigid lane structures, these games all share a common objective: managing spatial control to propel a ball (or puck) into a goal within a bounded field.

The framework developed in this thesis, converting raw spatial data into graph-based predictive insights, is largely agnostic to the specific domain. The concepts of permutation invariance (handling teams without fixed ordering) and dynamic edge modeling (defining interactions by proximity) are directly applicable to physical sports. It stands as a proof-of-concept that could be adapted for analyzing positional risk assessments in soccer, defensive rotations in basketball, or puck possession in hockey, which can be useful as a bridge between virtual simulation and physical sports science.

Furthermore, the interactive 3D portal developed in this work addresses a universal challenge in sports analytics, which is the disconnect between complex models and human interpretation. By translating abstract probabilistic outputs into an exploratory visual interface, we demonstrate a workflow that empowers domain experts (coaches) to interact directly with the data. This pipeline establishes a precedent for

how optical tracking data in professional sports could be visualized, which allows for intuitive scenario planning without requiring deep technical expertise.

7. CONCLUSION

This thesis presented peaRL (Predictive & Exploratory Analytics for Rocket League), a full-stack analytical framework developed to extract, model, and visualize high-dimensional gameplay data. The project moved beyond simple outcome prediction to address the end-to-end challenges of esports analytics: from the low-level parsing of binary replays to the high-level design of interactive user interfaces.

A foundational contribution of this work was the construction of a robust data pipeline capable of handling the chaotic physics-based nature of Rocket League. Significant effort was dedicated to the creation of the datasets, a process that necessitated rigorous data cleaning, spatial normalization, and feature engineering to transform raw telemetry into usable learning signals.

We addressed the "Token Sparsity" problem inherent in discrete state analysis. When constructing the Statistical Ledger for historical probability baselines, we encountered the challenge that highly similar 3D spatial configurations rarely repeat. By implementing state tokenization and handling rare-event dilution, we created a statistical benchmark that, while outperformed by our machine learning models, provided a necessary "ground truth" for calibrating our expectations of goal probability.

Our evaluation strategy moved beyond a simple leaderboard to a nuanced comparison of diverse architectures. We found that all full state-space models (MLP, XGBoost, GNN variants) decisively outperformed the heuristic baselines, the simplified Logistic Regression, and the Statistical Ledger in both AUPRC and F1 scores. This confirms that high-fidelity prediction requires the complete 92-dimensional feature set (or its graph equivalent) rather than limited kinematic subsets.

However, the results indicate that there is no single "silver bullet" architecture. The Gradient Boosted Trees (XGBoost) proved to be a highly competitive baseline,

achieving ranking performance (AUPRC) comparable to the best deep learning models. However, it exhibited higher Log Loss, suggesting that while it effectively ranks threat, its probability calibration is less stable than the graph-based counterparts. Not all deep learning approaches succeeded. The GCN and direct LSTM models showed visibly inferior performance compared to the GAT and MLP variants. This suggests that simple spectral convolution or pure temporal sequence modeling (without attention mechanisms) struggles to capture the sharp and discrete state changes typical of Rocket League. Ultimately, the best-performing models (GAT-Att, GAT-Avg, GAT-LSTM) converged to similar performance ceilings. This indicates that, given the current feature set, we may be approaching the limit of extractable signal from raw replay data, where architectural tweaks yield diminishing returns.

The analysis of the Ranked Generalization Dataset provided critical validation of model robustness. Despite being trained primarily on professional-level play, the models maintained comparable AUPRC values when tested on a dataset with a lower skill base. This confirms that the learned features, such as spatial control and intercept vectors, are fundamental to the game's physics engine and generalize effectively across the entire skill spectrum, rather than being overfit to specific high-level meta-strategies.

Two specific technical findings further refined our understanding of the state space. With an ablation study, we observed that explicitly encoding kinematic relationships (e.g., relative velocity alignment) on graph edges provided little to no benefit. The models were able to infer these dynamics implicitly from node attributes, which suggests that simpler graph topologies may be more effective. Additionally, by segmenting the test set into Low, Medium, and High threat buckets, we confirmed that while the models maintain a slight edge in "High Threat" scenarios, the noise inherent in "Low Threat" midfield play depresses performance universally across all architectures.

Finally, the development of the Unity-based Analysis Portal shifts the project's value from theoretical modeling to practical utility. By embedding these models into an

interactive 3D environment, we unlock distinct use cases for different beneficiaries. For analysts, players, and coaches, the tool serves as a mechanism for strategy validation. The "Counterfactual Editor" allows them to manipulate player positions in real-time to answer specific questions (e.g., "If the defender had been 50 units closer, would the goal probability have dropped?"), a capability that have the potential to move analysis from anecdotal debate to data-driven inquiry. On the other hand, for researchers, the modular design demonstrates how predictive models can be decoupled from their training pipelines and deployed as real-time inference engines within game engines. Such a system offers a template for future HCI research in sports and esports analytics.

In conclusion, this thesis demonstrates that while advanced GNNs offer theoretical advantages in calibration and representation, they are part of a broader ecosystem of viable predictive tools. The true advancement lies not in a single superior algorithm, but in the rigorous engineering of the data pipeline and the successful translation of these mathematical outputs into an accessible and interactive format for human decision-making.

REFERENCES

1. Newzoo, "Global Games Market Report 2023 – Free Version", 2023, https://newzoo.com/resources/trend-reports/newzoo-global-games-market-report-2023-free-version, accessed on November 13, 2025.

2. Jenny, S. E., R. D. Manning, M. C. Keiper and T. W. Olrich, "Virtual(ly) Athletes: Where eSports Fit Within the Definition of "Sport"", *Quest*, Vol. 69, No. 1, pp. 1–18, 2017.

3. Data Sports Group, "Esports in 2025: Data Insights and Trends Shaping the Future", 2025, https://datasportsgroup.com/news-article/160807/esports-in-2025:-data-insights-and-trends-shaping-the-future/, accessed on November 13, 2025.

4. Gudmundsson, J. and M. Horton, "Spatio-Temporal Analysis of Team Sports", *ACM Computing Surveys*, Vol. 50, No. 2, pp. 1–34, 2018.

5. Can't Fly, "ballchasing.com", 2025, https://ballchasing.com/, accessed on November 12, 2025.

6. LNDR, "CARL2: Your Ultimate Rocket League Analysis & Coaching Tool", 2025, https://lndrlndr.github.io/, accessed on November 12, 2025.

7. Rubin, A., "Predicting Round and Game Winners in CSGO", OSF Preprints: u9j5g, 2022.

8. Smithies, T. D., M. J. Campbell, N. Ramsbottom and A. J. Toth, "A Random Forest approach to identify metrics that best predict match outcome and player ranking in the esport Rocket League", *Scientific Reports*, Vol. 11, No. 1, p. 19285, 2021.

9. Hodge, V. J., S. Devlin, N. Sephton, F. Block, P. I. Cowling and A. Drachen, "Win Prediction in Multiplayer Esports: Live Professional Match Prediction", *IEEE Transactions on Games*, Vol. 13, No. 4, pp. 368–379, 2021.

10. Anzer, G. and P. Bauer, "A Goal Scoring Probability Model for Shots Based on Synchronized Positional and Event Data in Football (Soccer)", *Frontiers in Sports and Active Living*, Vol. 3, p. 624475, 2021.

11. Robberechts, P., J. Van Haaren and J. Davis, "A Bayesian Approach to In-Game Win Probability in Soccer", *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pp. 3512–3521, Virtual Event, Singapore, 2021.

12. Lee, K., H. Leo and L. Wang, "Rocket League esports", *Wharton Sports Business Journal*, Vol. 4, pp. 1–15, 2022.

13. Xenopoulos, P., H. Doraiswamy and C. Silva, "Valuing Player Actions in Counter-Strike: Global Offensive", *IEEE International Conference on Big Data (Big Data)*, pp. 1283–1292, Atlanta, GA, USA, 2020.

14. Sun, H., Y. Xiong, R. Wu, K. Wang, L. Zhang, C. Fan, S. Tang and X.-Y. Li, "MVP-Shapley: Feature-based Modeling for Evaluating the Most Valuable Player in Basketball", arXiv:2506.04602, 2025.

15. Grotentraast, J., *Decision support for Rocket League player improvement*, B.S. Thesis, University of Twente, 2021.

16. Yang, Y., T. Qin and Y.-H. Lei, "Real-time eSports Match Result Prediction", arXiv:1701.03162, 2016.

17. Stein, M., D. Seebacher, R. Marcelino, T. Schreck, M. Grossniklaus, D. A. Keim and H. Janetzko, "Where to go: Computational and visual what-if analyses in soccer", *Journal of Sports Sciences*, Vol. 37, No. 24, pp. 2774–2782, 2019.

18. Delibas, E., A. Uzun, M. F. Inan, O. Guzey and A. Cakmak, "Interactive exploratory soccer data analytics", *INFOR: Information Systems and Operational Research*, Vol. 57, No. 2, pp. 141–164, 2019.

19. Olsen, A. H., *The Evolution of eSports: An Analysis of its origin and a look at its prospective future growth as enhanced by Information Technology Management tools*, M.S. Thesis, Coventry University, 2015.

20. Grand View Research, "Esports Market Size, Share & Trends Analysis Report", 2024, `https://www.grandviewresearch.com/industry-analysis/esports-market`, accessed on October 19, 2025.

21. Kumar, N., "eSports Viewership Statistics 2025 (Growth & Demographics)", 2025, `https://www.demandsage.com/esports-statistics/`, accessed on October 19, 2025.

22. EsportStats.co, "The Ultimate List of Esports Statistics in 2025", 2025, `https://esportstats.co/`, accessed on October 19, 2025.

23. Chugani, V., "The Analytics Behind Esports: Data-Driven Strategies in Competitive Gaming", 2024, `https://www.statology.org/the-analytics-behind-esports-data-driven-strategies-in-competitive-gaming/`, accessed on January 30, 2026.

24. Lewis, M., *Moneyball: The Art of Winning an Unfair Game*, W. W. Norton & Company, New York, 2003.

25. Market Intelo, "Esports Analytics Market Research Report 2033", 2025, `https://marketintelo.com/report/esports-analytics-market`, accessed on October 19, 2025.

26. Suncho, S., "Importance of Data Analytics in eSports", 2020, `https://esportslane.com/data-analytics-esports`, accessed on October 19, 2025.

27. Esports Charts, "Top esports games in 2025", 2025, https://escharts.com/top-games, accessed on October 20, 2025.

28. Psyonix, "Rocket League Esports", Rocket League, 2025, https://www.rocketleague.com/competitive, accessed on October 20, 2025.

29. Tracker Network, "Rocket League Season 17 Skill Rating Distribution - Tracker Network", 2025, https://rocketleague.tracker.network/rocket-league/distribution?playlist=13, accessed on March 12, 2025.

30. Babcock, N., "Boxcars", 2025, https://github.com/nickbabcock/boxcars, accessed on October 18, 2025.

31. Bakkes, "CPPRP", 2025, https://github.com/Bakkes/CPPRP, accessed on October 18, 2025.

32. jjbott, "RocketLeagueReplayParser", 2025, https://github.com/jjbott/RocketLeagueReplayParser, accessed on October 18, 2025.

33. Fausak, T., "Rattletrap", 2025, https://github.com/tfausak/rattletrap, accessed on October 18, 2025.

34. SaltieRL, "carball", 2025, https://github.com/SaltieRL/carball, accessed on October 18, 2025.

35. Unity, "Unity", 2025, https://unity.com/, accessed on November 16, 2025.

36. Unreal Engine, "Unreal Engine", 2025, https://www.unrealengine.com/en-US, accessed on November 16, 2025.

37. AIS Technolabs, "Unity vs Unreal Engine: Pros, Cons, and Key Differences Explained", 2025, https://www.aistechnolabs.com/blog/unity-vs-unreal-engine, accessed on November 16, 2025.

38. Drachen, A. and M. Schubert, "Spatial game analytics and visualization", *IEEE Conference on Computational Inteligence in Games (CIG)*, pp. 1–8, Niagara Falls, ON, Canada, 2013, iSSN: 2325-4289.

39. RL DataCoach, "RL DataCoach", 2025, `https://rldatacoach.com/`, accessed on November 12, 2025.

40. trophi.ai, "trophi.ai", 2025, `https://www.trophi.ai/rocket-league`, accessed on November 12, 2025.

41. Bonny, J. W., "Using Collective Metrics to Assess Team Dynamics and Performance in eSports:", *International Journal of Gaming and Computer-Mediated Simulations*, Vol. 14, No. 1, pp. 1–13, 2022.

42. Fujimura, A. and K. Sugihara, "Geometric analysis and quantitative evaluation of sport teamwork", *Systems and Computers in Japan*, Vol. 36, No. 6, pp. 49–58, 2005.

43. Bialkowski, A., P. Lucey, P. Carr, Y. Yue, S. Sridharan and I. Matthews, "Large-Scale Analysis of Soccer Matches Using Spatiotemporal Tracking Data", *IEEE International Conference on Data Mining*, pp. 725–730, Shenzhen, China, 2014.

44. Cervone, D., A. D'Amour, L. Bornn and K. Goldsberry, "A Multiresolution Stochastic Process Model for Predicting Basketball Possession Outcomes", *Journal of the American Statistical Association*, Vol. 111, No. 514, pp. 585–599, 2016.

45. Mahlmann, T., M. Schubert and A. Drachen, "Esports Analytics Through Encounter Detection", *Proceedings of the MIT Sloan Sports Analytics Conference*, pp. 1–18, Boston, MA, USA, 2016.

46. Mathonat, R., J.-F. Boulicaut and M. Kaytoue, "A Behavioral Pattern Mining Approach to Model Player Skills in Rocket League", *IEEE Conference on Games (CoG)*, pp. 267–274, Osaka, Japan, 2020.

47. Kleinman, E., S. Ahmad, Z. Teng, A. Bryant, T.-H. D. Nguyen, C. Harteveld and M. Seif El-Nasr, ""And then they died": Using Action Sequences for Data Driven, Context Aware Gameplay Analysis", *International Conference on the Foundations of Digital Games*, pp. 1–12, Bugibba, Malta, 2020.

48. Rocha, E., H. Maio, D. S. Menasché and C. Miceli, "Extracting and Composing a Dataset of Competitive Counter-Strike Global Offensive Matches", *Anais do III Dataset Showcase Workshop (DSW)*, pp. 33–42, Rio de Janeiro, Brasil, 2021.

49. Hitar-García, J. A., L. Morán-Fernández and V. Bolón-Canedo, "Machine Learning Methods for Predicting League of Legends Game Outcome", *IEEE Transactions on Games*, Vol. 15, No. 2, pp. 171–181, 2023.

50. Xenopoulos, P., W. R. Freeman and C. Silva, "Analyzing the Differences between Professional and Amateur Esports through Win Probability", *Proceedings of the ACM Web Conference*, pp. 3418–3427, New York, NY, USA, 2022.

51. Li, J., "Winning Duels in VALORANT: A Visualization of Optimal Positioning", *Proceedings of the MIT Sloan Sports Analytics Conference*, pp. 1–14, Boston, MA, USA, 2022.

52. Jang, J., J. Y. Woo and H. K. Kim, "Action2Score: An Embedding Approach to Score Player Action", *Proceedings of the ACM on Human-Computer Interaction*, Vol. 6, pp. 1–23, 2022.

53. Xenopoulos, P., B. Coelho and C. Silva, "Optimal Team Economic Decisions in Counter-Strike", arXiv:2109.12990, 2021.

54. Petri, G., M. H. Stanley, A. B. Hon, A. Dong, P. Xenopoulos and C. Silva, "Bandit Modeling of Map Selection in Counter-Strike: Global Offensive", arXiv:2106.08888, 2021.

55. Semenov, A., P. Romov, S. Korolev, D. Yashkov and K. Neklyudov, "Performance

of Machine Learning Algorithms in Predicting Game Outcome from Drafts in Dota 2", D. I. Ignatov, M. Y. Khachay, V. G. Labunets, N. Loukachevitch, S. I. Nikolenko, A. Panchenko, A. V. Savchenko and K. Vorontsov (Editors), *Analysis of Images, Social Networks and Texts*, pp. 26–37, Cham, 2017.

56. Xenopoulos, P. and C. Silva, "Graph Neural Networks to Predict Sports Outcomes", *IEEE International Conference on Big Data (Big Data)*, pp. 1757–1763, Orlando, FL, USA, 2021.

57. Bisberg, A. J. and E. Ferrara, "GCN-WP – Semi-Supervised Graph Convolutional Networks for Win Prediction in Esports", *IEEE Conference on Games (CoG)*, pp. 449–456, Beijing, China, 2022.

58. Osei, F., "Graph-Based Modeling and Prediction of Match Outcomes in International Football Tournaments", DOI: 10.13140/RG.2.2.30652.65928, 2024.

59. Walo, N., *Seer: Reinforcement Learning in Rocket League*, M.S. Thesis, ETH Zurich, 2022.

60. Moschopoulos, V., P. Kyriakidis, A. Lazaridis and I. Vlahavas, "Lucy-SKG: Learning to Play Rocket League Efficiently Using Deep Reinforcement Learning", arXiv:2305.15801, 2023.

61. Pleines, M., K. Ramthun, Y. Wegener, H. Meyer, M. Pallasch, S. Prior, J. Drogemuller, L. Buttinghaus, T. Rothemeyer, A. Kaschwig, O. Chmurzynski, F. Rohkrahmer, R. Kalkreuth, F. Zimmer and M. Preuss, "On the Verge of Solving Rocket League using Deep Reinforcement Learning and Sim-to-sim Transfer", *IEEE Conference on Games (CoG)*, pp. 253–260, Beijing, China, 2022.

62. Omi, N., L. Caccia, A. Sarkar, J. T. Ash and S. Sen, "Generative Modeling of Individual Behavior at Scale", arXiv:2502.14998, 2025.

63. Kipf, T. N. and M. Welling, "Semi-Supervised Classification with Graph Convo-

lutional Networks", *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, Toulon, France, 2017.

64. Xu, J., Y. Cao, Z. Zhang and H. Hu, "Spatial-Temporal Relation Networks for Multi-Object Tracking", *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 3987–3997, Seoul, South Korea, 2019.

65. Li, J., D. Cai and X. He, "Learning Graph-Level Representation for Drug Discovery", arXiv:1709.03741, 2017.

66. Li, Z., L. Xia, L. Shi, Y. Xu, D. Yin and C. Huang, "Open Spatio-Temporal Foundation Models for Traffic Prediction", *ACM Transactions on Intelligent Systems and Technology*, pp. 1–20, 2025.

67. Liu, Z., X. Xie, M. He, W. Zhao, Y. Wu, L. Cheng, H. Zhang and Y. Wu, "Smartboard: Visual Exploration of Team Tactics with LLM Agent", *IEEE Transactions on Visualization and Computer Graphics*, Vol. 31, No. 1, pp. 23–33, 2025.

68. Liu, G., Y. Luo, O. Schulte and T. Kharrat, "Deep soccer analytics: learning an action-value function for evaluating soccer players", *Data Mining and Knowledge Discovery*, Vol. 34, No. 5, pp. 1531–1559, 2020.

69. Liu, G., W. Zhu and O. Schulte, "Interpreting Deep Sports Analytics: Valuing Actions and Players in the NHL", U. Brefeld, J. Davis, J. Van Haaren and A. Zimmermann (Editors), *Machine Learning and Data Mining for Sports Analytics*, pp. 69–81, Cham, 2019.

70. Sun, X., J. Davis, O. Schulte and G. Liu, "Cracking the Black Box: Distilling Deep Sports Analytics", *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 3154–3162, Virtual Event, CA, USA, 2020.

71. Cheng, F., Y. Ming and H. Qu, "DECE: Decision Explorer with Counterfactual

Explanations for Machine Learning Models", arXiv:2008.08353, 2020.

72. Kaul, S., D. Borland, N. Cao and D. Gotz, "Improving Visualization Interpretation Using Counterfactuals", *IEEE Transactions on Visualization and Computer Graphics*, Vol. 28, No. 1, pp. 998–1008, 2022.

73. Wallner, G. and S. Kriglstein, "PLATO: A visual analytics system for gameplay data", *Computers & Graphics*, Vol. 38, pp. 341–356, 2014.

74. Andersen, E., Y.-E. Liu, E. Apter, F. Boucher-Genesse and Z. Popović, "Gameplay analysis through state projection", *Proceedings of the Fifth International Conference on the Foundations of Digital Games*, pp. 1–8, Monterey, CA, USA, 2010.

75. Weixelbaum, W. and K. Matkovic, "Rumble Flow++ Interactive Visual Analysis of Dota2 Encounters", *EuroVis Workshop on Visual Analytics (EuroVA)*, pp. 37–41, 2021.

76. Fu, Y. and J. Stasko, "Supporting Data-Driven Basketball Journalism through Interactive Visualization", *CHI Conference on Human Factors in Computing Systems*, pp. 1–17, New Orleans, LA, USA, 2022.

77. Sha, L., P. Lucey, Y. Yue, P. Carr, C. Rohlf and I. Matthews, "Chalkboarding: A New Spatiotemporal Query Paradigm for Sports Play Retrieval", *Proceedings of the 21st International Conference on Intelligent User Interfaces*, pp. 336–347, Sonoma, CA, USA, 2016.

78. Xenopoulos, P., J. Rulff and C. Silva, "ggViz: Accelerating Large-Scale Esports Game Analysis", *Proceedings of the ACM on Human-Computer Interaction*, Vol. 6, pp. 1–22, New York, NY, USA, 2022.

79. Seidenschwarz, P., A. Jonsson, F. Rauschenbach, M. Rumo, L. Probst and H. Schuldt, "Combining Qualitative and Quantitative Analysis in Football with SportSense", *Proceedings Proceedings of the 2nd International Workshop on Mul-*

timedia Content Analysis in Sports, pp. 34–41, Nice, France, 2019.

80. Horst, R., L. Wehenkel and R. Dörner, "3D Hexglyph Maps: An Immersive Analytics Technique Combining Hexbin Maps with Space-Time Cubes for Visualizing eSports Data", *IEEE Games, Entertainment, Media Conference (GEM)*, pp. 1–6, St. Michael, Barbados, 2022, iSSN: 2766-6530.

81. Mane, S. V. and N. Elmqvist, ""Wichita 1-1, Fox Three" - The Role of 3D Telemetry Analysis in Combat Flight Simulation", *Proceedings of the ACM on Human-Computer Interaction*, Vol. 8, pp. 1–27, New York, NY, USA, 2024.

82. Kejstova, M., T. Stastna and S. Kriglstein, "Play and Viz: Using Entertainment Games for Exploring Data Visualizations", *Proceedings of the 19th International Conference on the Foundations of Digital Games*, pp. 1–12, Worcester, MA, USA, 2024.

83. Li, Q., Z. Wu, P. Xu, H. Qu and X. Ma, "A Multi-Phased Co-design of an Interactive Analytics System for MOBA Game Occurrences", *Proceedings of the Designing Interactive Systems Conference*, pp. 1321–1332, Hong Kong, China, 2018.

84. Cheng, L., D. Deng, X. Xie, R. Qiu, M. Xu and Y. Wu, "SNIL: Generating Sports News From Insights With Large Language Models", *IEEE Transactions on Visualization and Computer Graphics*, Vol. 31, No. 7, pp. 3973–3986, 2025.

85. BLAST.tv, "RLCS Birmingham Major 1 2025", 2025, `https://blast.tv/rl/tournaments/birmingham-major-1-2025`, accessed on November 18, 2025.

86. RLCS Referee, "Split 1 (Birmingham Major)", 2025, `https://ballchasing.com/group/split-1-birmingham-major-yfp3oy8hz6`, accessed on November 18, 2025.

87. BLAST.tv, "RLCS Copenhagen Major 1 2024", 2024, `https://blast.tv/rl/tournaments/major-1-2024`, accessed on November 18, 2025.

88. RLCS Referee, “Major 1 (Cophenagen Major)”, 2024, https://ballchasing.com/group/major-1-nuaen2ts44, accessed on November 18, 2025.

89. BLAST.tv, “RLCS London Major 2 2024”, 2024, https://blast.tv/rl/tournaments/major-2-2024, accessed on November 18, 2025.

90. RLCS Referee, “Major 2 (London Major)”, 2024, https://ballchasing.com/group/major-2-dm8bqwqj09, accessed on November 18, 2025.

91. BLAST.tv, “RLCS World Championship 2024”, 2024, https://blast.tv/rl/tournaments/rlcs-2024-world-championship, accessed on November 18, 2025.

92. RLCS Referee, “World Championship (Fort Worth)”, 2024, https://ballchasing.com/group/world-championship-md058mxx2x, accessed on November 18, 2025.

93. Arslan, Oğuz, “Boysle’s Season 2024-2025 Standard Rank Seperated Random Picks”, 2025, https://ballchasing.com/group/boysle-s-season-2024-2025-standa-w4vbrqmci1, accessed on November 18, 2025.

94. RLBot, “Useful game values”, 2025, https://wiki.rlbot.org/v5/botmaking/useful-game-values/, accessed on November 19, 2025.

APPENDIX A: EXTENDED RANKED EVALUATION BY THREAT CHARACTERIZATION



Figure A.1. Detailed performance evolution across the ranked generalization dataset, segmented by threat category, for XGBoost, GAT-7n-avg-e2 (GAT-Avg), GAT-7n-att-e2 (GAT-Att), and GAT-LSTM hybrid models.